



Moltaqa Tech

E-learning website for tech industry community

Project Team

- Ahmed Usama
- Ahmed Anwar
- Omar Nabil
- Ahmed Talaat
- Ahmed Tarek
- Moaz El-yamany
- Mohamed Konsowa
- Mohammed Elalimy
- Mostafa Osman

Computer Engineering And Automatic Control Department
Faculty of Engineering, Tanta University

Supervised By
Dr. Hany Elghaish

Tanta, 2024

Acknowledgment

First and foremost, we would like to thank our advisor, Dr. Hany El-Ghaish. His support and guidance have been invaluable throughout this project. We are grateful for his expertise and the time he dedicated to helping us succeed.

We also extend our gratitude to the professors and teaching assistants in our department. Their contributions and efforts in providing us with theoretical and experimental support have been crucial to our project's success. We appreciate their dedication and commitment to our education.

Lastly, we want to express our heartfelt thanks to our parents, sisters, and brothers. Their unwavering support and constant encouragement have been instrumental in our academic journey. We are truly grateful for their belief in us and their presence throughout our undergraduate work.

To everyone mentioned above, we offer our sincere appreciation for their contributions to this graduation project. Your support has been invaluable, and we are grateful for the opportunity to have worked on this project.

Thank You

Table of Contents

Acknowledgment	ii
Table of Contents	iii
1 Project Plan	1
1.1 Executive Summary	1
1.2 Project Objectives	2
1.3 Success Factors	2
1.4 Key Stakeholders	2
1.5 Project Scope	2
1.5.1 Business Cases	2
1.5.2 Deliverables	3
1.5.3 Acceptance Criteria	3
2 Business Requirements	5
2.1 Business Requirements	5
2.2 Constraints	5
2.3 Schedule and Resources	5
2.3.1 Functional Requirements	6
2.4 Non-Functional Requirements	6
2.4.1 Speed	6
2.4.2 Ease of Use & Usability	7
2.4.3 Reliability	7
2.4.4 Robustness	7
2.5 Cost-Benefit Analysis	7
2.6 Limitations	7
2.6.1 Unrealistic Expectations	7
2.6.2 Communication Gaps	7
2.6.3 Lack of Project Visibility	8
2.7 SWOT Analysis	8
3 Software Requirements Specification	9
3.1 Overall Description	9
3.2 Product Perspective	9
3.3 Product Features	9
3.4 User Characteristics	10
3.5 Design and Implementation Constraints	10
3.6 Assumptions and Dependencies	11

3.7	Specific Requirements	11
3.8	Interface Requirements	11
3.8.1	External Hardware Interfaces Requirements	11
3.8.2	Internal Hardware Requirements	11
3.8.3	External Software Interfaces Requirements	11
3.8.4	Internal Software Requirements	12
3.8.5	Communication Interfaces Requirements	12
4	Functional Requirements	13
4.1	Unsigned User	13
4.2	Student User	14
4.3	Instructor User	14
5	Non-Functional Requirements and Theoretical NFR Framework	17
5.1	Non-Functional Requirements	17
5.1.1	Why Non-Functional Requirements	17
5.1.2	Theoretical NFR Framework	18
5.2	Security	18
5.3	Usability	18
5.4	Confidentiality	19
5.5	Reliability	19
5.6	Operability	19
5.7	Traceability	19
5.8	Recoverability	20
5.9	Visibility	20
6	Software Architecture and Design	21
6.1	Onion Architecture	21
6.1.1	Layers in Our Project	22
6.1.1.1	API Layer	22
6.1.1.2	Core Layer	23
6.1.1.3	Repository Layer	23
6.1.1.4	Service Layer	23
6.2	Design Patterns	24
6.2.1	Unit of Work Pattern	24
6.2.2	Repository Pattern	25
6.2.3	Specification Pattern	25
6.2.4	Dependency Injection	25
6.3	SQL Server and Entity Framework	26
6.3.1	Advantages of SQL Server	26
6.3.2	Entity Framework	27
6.4	Database Design	27
7	Performance Requirements	29
7.1	Performance Requirements	29
7.1.1	Databases	29
7.1.2	Networks	29
7.1.3	Technology Stacks	30

8	Smart Typing: Autocorrect and Autocomplete with AI	33
8.1	Introduction	33
8.2	The Importance in Web Design	34
8.3	Practical Use Cases	34
8.4	Implementation Approaches	35
8.5	Autocorrect Task	36
8.5.1	Power of BART	37
8.5.1.1	Architecture	37
8.5.1.2	Applications	37
8.5.2	Dataset Preparation	38
8.5.2.1	Data Augmentation and Cleaning Functions:	38
8.5.3	BART Fine-tune Process	39
8.5.3.1	Data Preparation	40
8.5.3.2	Model Configuration	40
8.5.3.3	Training Procedure	40
8.5.3.4	Evaluation Metrics	40
8.5.3.5	Results and Analysis	40
8.6	Autocomplete Task with GPT-2	41
8.6.1	Model Initialization	41
8.6.2	Model Used : GPT-2 (Generative Pre-trained Transformer 2)	41
8.6.3	Prediction Function	42
8.7	Future Directions	43
8.7.1	Emerging Trends in Autocomplete and Autocorrect Research	43
8.7.2	Potential Advancements and Challenges	44
8.7.3	Speculations on Future Integration with AI and Natural Language Understanding	44
8.8	Deployment with Flask	45
8.9	Conclusion	46
9	Development of an AI-Powered Chatbot Assistant: Advanced Features and Implementation	49
9.1	Introduction	49
9.2	Chatbot System Overview	50
9.3	Key Functionalities	50
9.3.1	Imports and Environment Setup	50
9.3.2	File Reading and Content Extraction	51
9.3.3	Web Scraping for Dynamic Content Retrieval	51
9.3.4	Text Processing and Normalization	51
9.3.5	Integration of AI Models	52
9.4	Model Selection	52
9.4.1	Introduction	52
9.4.2	Model Suitability	52
9.4.3	Inference Time Speed	53
9.4.4	Context Window Size	53
9.4.5	Memory vs Performance Trade-off	53
9.4.6	Model Update Frequency	53

9.4.7	Domain-Specific Competence	53
9.4.8	Selected Model	54
9.5	Setting Template and Memory	54
9.5.1	Challenges of Traditional Models	54
9.5.2	Model Parameters	54
9.6	Modes Implementation	55
9.6.1	Traditional Chatting Mode	55
9.6.2	Chart Generator Mode	55
9.6.3	Internet Search Mode	55
9.6.4	Link Scratch Mode	55
9.6.5	File Interact Mode	56
9.6.6	Explore Website Mode	56
9.7	Framework Construction	56
9.7.1	Frontend Development	56
9.7.2	Input and Output Setup	56
9.7.3	Launch Gradio App	56
10	Optimizing AI Models for E-Learning: Machine Translation, Question Generation, and Text Summarization	57
10.1	Introduction	57
10.2	Effective Methods for Fine Tuning AI Models	58
10.2.1	Full Fine-Tuning in 6 Steps	59
10.2.2	What Makes Full Fine-Tuning Worthwhile?	60
10.2.3	The Downside of Full Fine-Tuning	60
10.2.4	Parameter-Efficient Fine-Tuning	61
10.3	LORA (Low-Rank Adaptation).	62
10.3.1	Advantages of LoRA	62
10.3.2	Where PEFT beats full fine-tuning:	63
10.4	Machine Translation Feature	64
10.4.1	Model Architectural Overview	64
10.4.2	Dataset Preparation	67
10.4.2.1	Dataset Collection	67
10.4.2.2	Dataset processing	68
10.4.3	Training Process	68
10.4.3.1	Loading the Pre-trained Model	68
10.4.3.2	Fine-Tuning Process	69
10.4.3.3	Evaluation and Optimization	70
10.4.4	Understanding BLEU Score in Machine Translation	70
10.4.4.1	What is BLEU Score?	70
10.4.4.2	How Does BLEU Score Work?	71
	1. N-gram Precision:	71
	2. Brevity Penalty:	71
	3. BLEU Score Formula:	71
10.4.4.3	Why is BLEU Score Important?	72
10.4.4.4	Limitations of BLEU Score	72
10.4.5	Training Results for Marian Model (Helsinki-NLP)	73
10.4.5.1	Arabic to English Translation	73

10.4.5.2 English to Arabic Translation	74
10.5 MCQA Quiz Generation	75
10.6 Text Summarization Feature	81
10.7 Data Collection	81
10.8 Model Selection and Architecture	81
10.8.1 BART Architecture	82
10.9 Fine-Tuning with LoRA and PEFT	82
10.9.1 Low-Rank Adaptation (LoRA)	82
10.9.2 Parameter-Efficient Fine-Tuning (PEFT)	82
10.10 Evaluation Metrics	83
10.10.1 ROUGE-1	84
10.10.2 ROUGE-2	84
10.10.3 ROUGE-L	84
10.10.4 ROUGE-LSUM	84
10.11 Results	85
10.12 Deployment with FastAPI and Docker	85
10.12.1 FastAPI Implementation	85
10.12.2 Containerization with Docker	86
10.12.3 Hosting on Hugging Face Spaces	86
10.13 Conclusion	86
11 Website Services	87
11.1 Security: Authentication & Authorization and Logout	87
11.1.1 Authentication	89
11.1.2 Authorization	89
11.1.3 Logout	89
11.2 Courses Module	90
11.2.1 Course Creation	90
11.2.2 Course Structure	91
11.2.3 Tracking Progress	92
11.2.4 Reviews	93
11.3 BlogPosts Module	93
11.3.1 Posting and Categories	94
11.3.2 Comments and Views	94
11.3.3 Sharable Links	94
11.4 Events Module	94
11.4.1 Event Creation	95
11.4.2 Participation	95
11.5 Zoom Meeting Module	95
11.5.1 Creating and Joining Meetings	96
11.6 Dashboard	96
11.6.1 Student Dashboard	97
11.6.2 Instructor Dashboard	97
11.7 Shop Module	97
11.7.1 Cart and Checkout	98
11.7.2 Order History	98

12 Angular Framework	99
12.1 Overview	99
12.2 History and Evolution	99
12.3 Key Features	100
12.4 Advantages of Using Angular 16	100
12.5 E-Learning Project with Angular 16: A Case Study	101
12.6 Integration with .NET Backend	101
12.7 Comparison with Other Frameworks	102
12.8 Real-world Applications	102
12.9 Conclusion	103
13 Front-End Project Architecture	105
13.1 Overview	105
13.2 App Folder Structure	105
13.3 Layout Folder	107
13.4 Modules Folder	107
13.5 Integration of Angular and .NET	120
13.6 Design Patterns and Principles	121
13.7 Security Measures	121
13.8 Database and API Interactions	121
14 Libraries Used in the Front-End Project	123
14.1 Font Awesome	123
14.1.1 Description of Font Awesome	124
14.1.2 Usage in Our Project	124
14.2 Bootstrap	125
14.2.1 Description of Bootstrap	125
14.2.2 Usage in Our Project	126
14.3 jQuery	127
14.3.1 Description of jQuery	127
14.3.2 Usage in Our Project	128
14.4 AOS.js	129
14.4.1 Description of AOS.js	129
14.4.2 Usage in Our Project	130
14.5 Parallax.js	131
14.5.1 Description of Parallax.js	131
14.5.2 Usage in Our Project	132
14.6 Perfect Scrollbar	132
14.6.1 Description of Perfect Scrollbar	133
14.6.2 Usage in Our Project	133
14.7 Swiper	134
14.7.1 Description of Swiper	134
14.7.2 Usage in Our Project	135
14.8 jQuery PowerTip	135
14.8.1 Description of jQuery PowerTip	136
14.8.2 Usage in Our Project	136
14.9 JWT-decode	137

14.9.1	Description of JWT-decode	137
14.9.2	Usage in Our Project	138
14.10	Angular CDK	139
14.10.1	Description of Angular CDK	139
14.10.2	Usage in Our Project	139
14.11	@abacritt/angularx-social-login	140
14.11.1	Description of @abacritt/angularx-social-login	141
14.11.2	Usage in Our Project	141
14.12	Mermaid	142
14.12.1	Description of Mermaid	142
14.12.2	Usage in Our Project	143
15	Building a Content-Based Recommendation System	145
15.1	Introduction to Recommendation Systems	145
15.2	Data Sources	146
15.3	Feature Selection	146
15.4	Text Vectorization with TF-IDF	147
15.5	Calculating Similarity with Cosine Similarity	148
15.6	Building the Recommendation System	148
15.7	Deployment with FastAPI and Docker	149
15.7.1	FastAPI Implementation	150
15.7.2	Containerization with Docker	150
15.7.3	Hosting on Hugging Face Spaces	150
15.8	Conclusion	151
16	Visual Learning: AI-Powered Explanations and Diagrams	153
16.1	Introduction to Visual Learning	153
16.2	Types of Diagrams Generated by the Tool	153
16.2.1	Flowcharts	153
16.2.2	Sequence Diagrams	154
16.2.3	Mind Maps	155
16.2.4	Class Diagrams	155
16.2.5	ER Diagrams	155
16.2.6	Gantt Charts	155
16.2.7	State Diagrams	156
16.3	Mermaid Diagramming Language	156
16.3.1	Overview of Mermaid Syntax	157
16.3.2	Integration with AI Tools	158
16.4	Building the AI Tool	158
16.4.1	Prompt Engineering	158
16.4.2	Few-Shot Learning	160
16.5	The LLaMA3-70B Model	161
16.5.1	Architecture and Design	161
16.5.2	Training and Capabilities	161
16.5.3	Applications in Visual Learning	162
16.5.4	Langchain Library	163
16.5.4.1	Prompt Management	163

16.5.4.2	Integration with Groq API	163
16.5.4.3	Workflow Orchestration	163
16.6	Deployment with FastAPI and Docker	164
16.6.1	FastAPI Implementation	165
16.6.2	Containerization with Docker	165
16.6.3	Hosting on Hugging Face Spaces	165
16.7	Conclusion	166
17	Conclusion and Future Work	167
17.1	Conclusion	167
17.2	Future Work	168
17.2.1	Enhanced Analytics and Reporting	168
17.2.2	Personalized Learning Paths	168
17.2.3	Gamification	168
17.2.4	Mobile Application	168
17.2.5	Integration with Other Tools	169
17.2.6	Advanced Security Features	169
17.2.7	Expanded Payment Options	169
17.2.8	Collaborative Learning Features	169
17.2.9	Content Localization	169
17.2.10	Continuous Improvement and Feedback Loops	169
A	Appendix	171
A.1	Source Code Links	171
A.2	References	171

Chapter 1

Project Plan

1.1 Executive Summary

MoltaqaTech is a learning website specialized in technology topics and subjects. It offers courses where instructors can provide videos, assignments, and quizzes to students. Additionally, students can purchase courses, add them to a cart or wishlist, and participate in live meetings. The platform also features a blogging section where users can write posts and comment on others' posts. MoltaqaTech incorporates several AI features, including:

- Text summarization model: Provides a quick recap of video contents.
- Question generation model: Generates questions to help students test their understanding of the video.
- Course recommendation system: Personalizes learning experiences by suggesting courses and blogs based on user preferences.
- Machine translation model: Allows users to translate unknown words directly on the website without external tools.

The AI technologies used include Python, Flask, PyTorch, and the Hugging Face library. The backend is developed using .NET Core, while the frontend utilizes the Angular framework.

1.2 Project Objectives

1. Maximize learning outcomes and engagement for students in technology-related subjects.
2. Provide a seamless user experience for instructors to create and manage courses effectively.
3. Enhance the platform's functionality with AI-driven features for content summarization, question generation, course recommendations, and machine translation.
4. Foster a vibrant community through blogging and discussion features.
5. Utilize a robust tech stack to ensure scalability, reliability, and performance.

1.3 Success Factors

- Achieve 10,000+ active users on the platform.
- Receive positive feedback from users regarding the effectiveness and usability of AI features.
- Maintain a high level of user engagement through courses, blogs, and community interactions.

1.4 Key Stakeholders

Dr. Hany Al-Ghaish - Project Manager

1.5 Project Scope

1.5.1 Business Cases

MoltaqaTech offers a range of features for both instructors and students:

- Courses: Instructors can upload videos, assignments, and quizzes for students.
- E-commerce: Students can purchase courses, add them to a cart, or save them to a wishlist.
- Live Meetings: Interactive sessions for real-time collaboration.

- Blogging: Users can write and comment on blog posts.
- AI Features: Text summarization, question generation, course recommendations, and machine translation.

1.5.2 Deliverables

- Web Front-End: Angular framework for a responsive and interactive user interface.
- Back-End: .NET Core for robust server-side functionality.
- AI Models: Python-based models using Flask, PyTorch, and Hugging Face library for text summarization, question generation, course recommendations, and machine translation.

1.5.3 Acceptance Criteria

- Reliability: Platform should function smoothly without major disruptions.
- Scalability: Ability to handle a growing user base and increased usage of AI features.

Chapter 2

Business Requirements

2.1 Business Requirements

Cloud hosting and storage are to be implemented on either AWS or Microsoft Azure.

2.2 Constraints

- The number of users must not exceed 100,000 due to limited server resources.
- The project timeline is constrained to 9 months.

2.3 Schedule and Resources

The project schedule includes the following phases:

1. Fancy UI design
2. Requirement analysis phase
3. UI/UX design & prototyping using Figma
4. Front-End phase
5. Back-End learning (.NET Core)
6. Back-End implementation
7. AI study phase

8. AI implementation (Python / Flask / PyTorch)

The project scope must include:

- Learning website specializing in technology topics (MoltaqaTech)
- Courses with sections, lectures, and quizzes
- Cart and wishlist functionality
- Live meetings feature
- Blog post creation and commenting
- AI features including text summarization, question generation, course recommendation, and machine translation
- Backend using .NET Core with Onion Architecture
- Frontend using Angular framework

2.3.1 Functional Requirements

- Text/Video conferencing
- AI features integration
- Educational courses with videos, assignments, and quizzes
- Registration, login, and user management
- Cart and wishlist functionality
- Blog post creation, commenting, and moderation
- Live meetings scheduling and participation

2.4 Non-Functional Requirements

2.4.1 Speed

- High-speed video streaming service optimization
- Efficient web server configuration for high traffic loads
- Proactive notification system for network issues

2.4.2 Ease of Use & Usability

- User-friendly interface
- Intuitive navigation and layout
- Responsive design across devices
- Comprehensive user support resources

2.4.3 Reliability

- 97% system availability
- Mean failure time not exceeding 2 seconds

2.4.4 Robustness

- Fault tolerance with Onion Architecture
- Local data storage for client-server communication issues
- Network issue notifications for users

2.5 Cost-Benefit Analysis

The project involves learning new technologies and architectures, which may have initial time costs but long-term benefits in scalability and maintainability.

2.6 Limitations

2.6.1 Unrealistic Expectations

Prevent with realistic expectations and stakeholder understanding.

2.6.2 Communication Gaps

Prevent with effective communication tools and project management software.

2.6.3 Lack of Project Visibility

Prevent with centralized document management and status tracking.

2.7 SWOT Analysis

Strengths	Weaknesses
Engaged existing followers	Lack of brand recognition
Original messaging and tone of voice	Need for funding
High-quality design	Poor user experience
Application design fits various needs	Modest advertising budget

Opportunities	Threats
Grow social media following	Competition from similar brands
Build e-commerce sales	Busy holiday season
Target audience growth	Emergence of new technologies

Chapter 3

Software Requirements Specification

3.1 Overall Description

MoltaqaTech is an innovative learning platform focused on technology-related topics and subjects. This section provides an overview of the entire system, including its context, basic functionality, stakeholders, and system constraints and assumptions.

3.2 Product Perspective

MoltaqaTech is a comprehensive learning website that offers various features to enhance the learning experience. It provides courses with video lectures, assignments, and quizzes. Students can purchase courses, add them to their cart or wishlist, and participate in live meetings. The platform also includes a blogging feature where users can create posts and interact through comments. Additionally, MoltaqaTech integrates AI features such as text summarization, question generation, course recommendation, and machine translation using Python, Flask, PyTorch, and the Huggingface library. The backend is developed in .NET Core, while the frontend utilizes the Angular framework. The system follows the Onion architecture for high scalability, loose coupling, and ease of deployment.

3.3 Product Features

- **Courses:** Instructors can upload videos, assignments, and quizzes for their courses.

- **E-commerce:** Students can purchase courses, add them to their cart or wishlist, and proceed to checkout.
- **Live Meetings:** Users can participate in live meetings for interactive learning experiences.
- **Blogging:** Users can create and share blog posts and engage in discussions through comments.
- **AI Features:**
 - Text Summarization: Provides a quick recap of video contents.
 - Question Generation: Generates questions to test students' understanding.
 - Course Recommendation: Personalizes learning by suggesting courses and blogs based on user preferences.
 - Machine Translation: Translates unknown words within the website without external tools.

3.4 User Characteristics

There are two primary user roles in MoltaqaTech:

- **Students:** They can enroll in courses, participate in live meetings, purchase courses, interact with blog posts, and benefit from AI features.
- **Instructors:** They can create and manage courses, conduct live meetings, and engage with students through the platform.

3.5 Design and Implementation Constraints

- **Technology Stack:** The system relies on Python, Flask, PyTorch, Huggingface library, .NET Core, and Angular framework.
- **Onion Architecture:** The backend architecture follows the Onion architecture paradigm for scalability and ease of deployment.
- **Integration of AI Models:** AI models for text summarization, question generation, course recommendation, and machine translation are integrated into the system.
- **Hardware and Software Dependencies:** The system requires sufficient hardware resources for AI processing and a stable internet connection for seamless operation.

3.6 Assumptions and Dependencies

- **AI Model Dependencies:** The AI features depend on Python, Flask, PyTorch, and the Huggingface library.
- **User Interaction:** Users are assumed to have basic knowledge of technology to interact effectively with the platform.
- **Internet Connectivity:** A stable internet connection is assumed for users to access and utilize all features.

3.7 Specific Requirements

This section details the functional and quality requirements of MoltaqaTech, including user authentication, course management, e-commerce functionalities, live meeting capabilities, blogging features, and AI model integrations.

3.8 Interface Requirements

3.8.1 External Hardware Interfaces Requirements

As MoltaqaTech is a web-based platform, there are no specific external hardware requirements. Users can access the system using standard devices with internet connectivity.

3.8.2 Internal Hardware Requirements

The backend infrastructure should have sufficient CPU, RAM, and storage capabilities to handle concurrent user requests, AI processing, and database interactions effectively.

3.8.3 External Software Interfaces Requirements

MoltaqaTech's frontend is built using the Angular framework and communicates with the backend services via secure APIs. Users interact with the system through a web browser, and standard web development practices are followed for code and UI design.

3.8.4 Internal Software Requirements

The backend architecture follows the Onion architecture paradigm, ensuring high scalability, loose coupling, and ease of deployment. The system utilizes AI models for various features, and communication between services is optimized for performance and reliability.

3.8.5 Communication Interfaces Requirements

Communication within the system components and with external services is crucial for MoltaqaTech's functionality. Secure API gateways are used for communication between frontend and backend services, while AI models interact seamlessly with the core system for text summarization, question generation, course recommendation, and machine translation functionalities.

Chapter 4

Functional Requirements

4.1 Unsigned User

- Can view the website home page that contains:
 - The trending (most viewed) courses.
 - Most viewed profile pages (like popular instructors).
 - **Search Bar**: They can search for courses or instructors by name and view their content.
 - **Join Zoom Meeting** button: Takes them to a page where they can join a Zoom meeting.
 - They can't enroll in courses unless they sign up or log in.
 - They can't post reviews or comments unless they sign up or log in.
- User Registration: They should provide:
 - Username
 - Email
 - Password
- User Login: They should provide:
 - Email
 - Password
 - **Retrieve password**: If the user has registered and chosen "I forgot my password", the password is sent to them by email.

4.2 Student User

- Can view the home page that contains:
 - **Feed:** A tab that contains the activities of the courses or instructors they follow.
 - **Recommendations:** A tab that contains recommended courses or instructors to follow.
 - **Search Bar:** They can search for courses or instructors by name.
 - **Join a Zoom Meeting** button: Takes them to a page where they can join a Zoom meeting.
- Can enroll in and view courses they have purchased.
- Can put courses in a wishlist.
- Can post reviews on courses they have purchased.
- Can access their personal profile and data.
- Can change their profile picture.
- Can change their name, email, and password.
- Can view all the courses they have enrolled in.
- Can view all the instructors they follow.

4.3 Instructor User

- Can do all the things that Student Users can do.
- Normal student user but when request to be an instructor it's required from him to enter
 - bio
 - Profile picture if wasn't provided before
- Can create courses that:
 - Have learning objectives, descriptions, and costs.
 - Are divided into sections, with each section containing items that can be either lectures or quizzes.
 - Track the progress of students enrolled in them.

- Can access an instructor dashboard that provides:
 - Overview of the courses they have created.
 - Statistics on student progress and course performance.

Chapter 5

Non-Functional Requirements and Theoretical NFR Framework

5.1 Non-Functional Requirements

5.1.1 Why Non-Functional Requirements

Functional requirements exhibit the behavior of the system that fulfills the client's objectives or tasks, whereas non-functional requirements consolidate constraints and qualities. Qualities are properties of the system that are examined by it and, in this manner, will impact their level of satisfaction with the system. Constraints are not subject to negotiation and, unlike qualities, are (theoretically at any rate) off-limits throughout design trade-offs. Contextual constraints are characteristics of the "super-system" or the advancement affiliation that oblige the change somehow. Examples include the target operating system or hardware platform by virtue of the customer environment, or the inclination set of open visionaries by virtue of the improvement association. Non-functional requirements (NFRs) are in general called quality requirements. The business asks for the more non-viable perspectives to be satisfied in information schemas other than their functionality. Mistakes in light of the omission of NFRs or not suitably administering them are among the most costly and generally hard to reexamine. Non-functional requirements are always related to a viable need.

5.1.2 Theoretical NFR Framework

The NFR Framework offers a systematic methodology for characterizing NFRs. It offers an extraordinary capacity to see all significant NFRs and their interdependencies. Additionally, it helps software designers to comprehend the vital activities for ensuring quality. It captures and documents design decisions and rationales in addition to providing traceability for derived specifications and requirements. The NFR Framework offers a precise approach to describing NFRs. It helps software designers understand essential activities for ensuring quality, captures and documents design decisions, and provides traceability for derived specifications and requirements.

5.2 Security

Security is the feature of the system which ensures that the system must be protected from unintentional or malignant harm. Unauthorized access to the data is not permissible. For safety purposes, the data must be backed up regularly, and the backed-up data must be stored in a secure location. In an online learning platform, the application must send or receive information to or from the server and client in an encrypted way. Security must adhere to some standards and plans. Security is a significant subject as the client is more concerned about the security of the account, personal data, and transactions. The information kept in the system ought to be precise and complete.

5.3 Usability

The online learning platform will be used by various types of clients, whether they have knowledge of computers or not. The application must be easy to use and enable the client to manage their accounts or courses with simplicity. The application must have a graphical user interface and provide informative error messages. Measurable qualities of usability include learning time, the number of errors while working, and client satisfaction. The interfaces of the system should be clear, easy, and simple to use and understand. To enhance usability, online help and customer care should be incorporated into the system.

5.4 Confidentiality

Clients should access the online learning account after successful authentication. The data entered by the client is not accessible to other clients. Confidentiality means maintaining the secrecy of the client's information, and clients should have their own passwords and usernames. These must automatically become inaccessible to unauthorized users.

5.5 Reliability

Reliability reflects the capacity of the software to maintain its performance over time. It implies how well the system performs during peak hours. A robust system can handle bugs without failure and recover quickly from failures. The application must be self-contained, consistent, and complete within itself. The failure rate should be minimal, and the system should be reliable, handling failures effortlessly.

5.6 Operability

Operability concerns how well the software will work in a distinctive environment. The online learning application must operate on any device, such as handheld gadgets, desktops, or laptops, without failure. Changes in the environment should not hamper the software's operability. The application should be operational on any handheld gadget and have the capacity to show different types of content. While in operation, the system should provide consistency and ease of operation and control.

5.7 Traceability

Traceability refers to the capability of tracing the status of transactions and account activities. In the learning industry, it makes tracking user activities possible. The online learning platform should enable users to trace the state of their activities at any time.

5.8 Recoverability

Recoverability implies the ability to restore software to the point when a failure occurred. It depends not just on having a backup of the data but also on having a predefined plan for recovering that information. The software must be tested to check if it can recover from failures, such as internet problems or other reasons. The system should recover the affected information after failure and restore the desired level of performance.

5.9 Visibility

Visibility refers to the condition of having the capacity to see online meetings and account activities clearly. The application must be designed so that every single feature is obvious to the client. The features should be self-contained so that even a novice can use the system without hesitation. All features must be visible, easy to understand, and use.

Chapter 6

Software Architecture and Design

6.1 Onion Architecture

Onion architecture is an architectural pattern that addresses the challenges of traditional layered architecture by ensuring a separation of concerns and promoting a clear dependency rule: code can depend only on layers inward. This architecture offers several advantages:

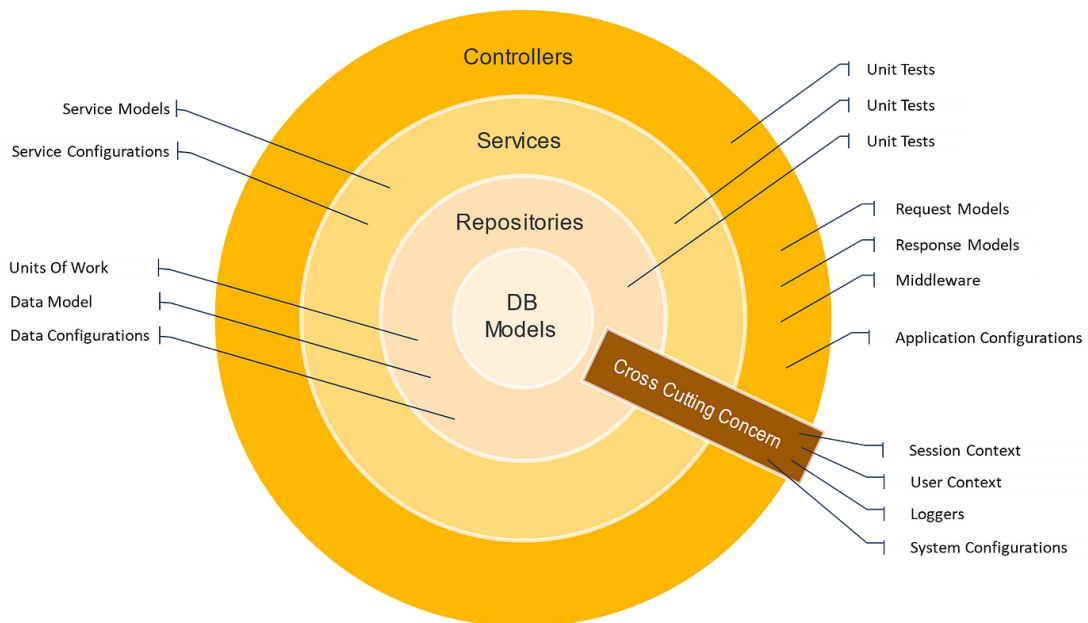


FIGURE 6.1: Onion Architecture

- **Separation of Concerns:** Each layer in the onion architecture has a specific responsibility, which makes the system more modular and easier to understand.

- **Testability:** Because the core logic is separated from external concerns, such as data access and user interfaces, it is easier to write unit tests for the core logic.
- **Maintainability:** Changes in one layer often do not affect other layers, making the system easier to maintain.
- **Flexibility:** The system is more flexible and adaptable to changes in technology or requirements since external dependencies are kept on the outer layers.
- **Scalability:** The architecture supports scalability as each layer can be scaled independently based on the needs of the application.

6.1.1 Layers in Our Project

Our project is structured into four layers, each with distinct responsibilities:

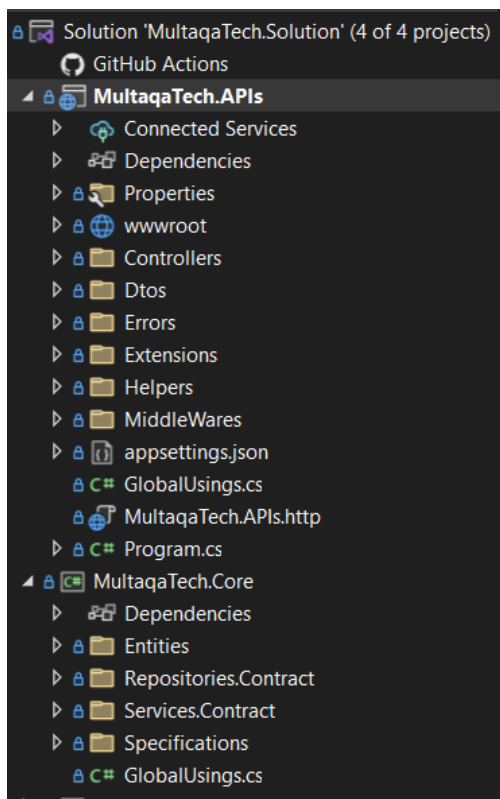


FIGURE 6.2: Solution Structure 1

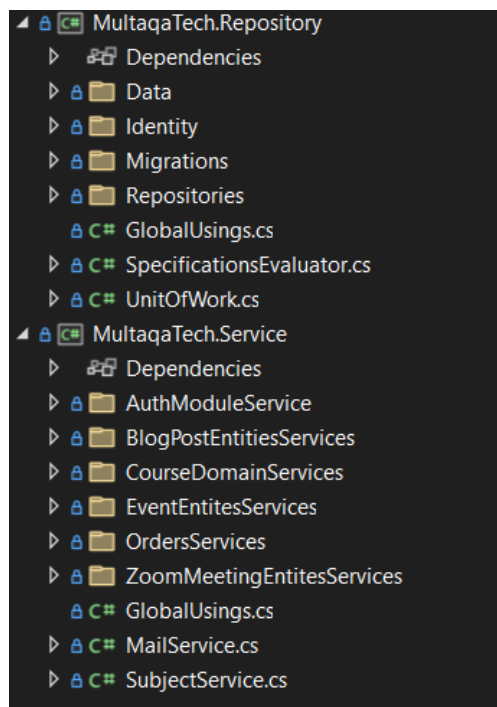


FIGURE 6.3: Solution Structure 2

6.1.1.1 API Layer

The API layer serves as the entry point for client requests. It contains controllers (endpoints) and Data Transfer Objects (DTOs) that are exposed to the clients. This layer also includes:

- **Mapping Profiles:** Used for mapping between domain models and DTOs.
- **Custom Middleware:** Includes middleware for handling exceptions, rate limiting, and other cross-cutting concerns.
- **Authentication and Authorization:** Ensures that only authenticated and authorized users can access specific endpoints.

6.1.1.2 Core Layer

The core layer defines the core business logic and entities that reflect the tables in the database. This layer includes:

- **Entities:** Represent the business objects and are directly mapped to the database tables.
- **Contracts (Interfaces):** Define the contracts for the repository and service layers, ensuring a clear separation of concerns and promoting the Dependency Inversion Principle.

6.1.1.3 Repository Layer

The repository layer handles data access logic. It includes:

- **Identity DbContext:** Manages all identity-related tables and the business database, facilitating user authentication and authorization.
- **Generic Repository Class:** Provides a generic implementation of common data access operations such as CRUD (Create, Read, Update, Delete).
- **Unit of Work Class:** Manages transactions across multiple repositories, ensuring atomic operations and maintaining data integrity.

6.1.1.4 Service Layer

The service layer contains the business logic of the application. It implements the contracts defined in the core layer and is used by the controllers in the API layer to process requests and return responses. This layer includes:

- **Business Services:** Implement business logic and handle operations related to business rules and processes.

- **Helper Services:** Provide utility functions and common operations used across different services and layers.

6.2 Design Patterns

In our project, we utilized several design patterns to enhance the architecture and design:

6.2.1 Unit of Work Pattern

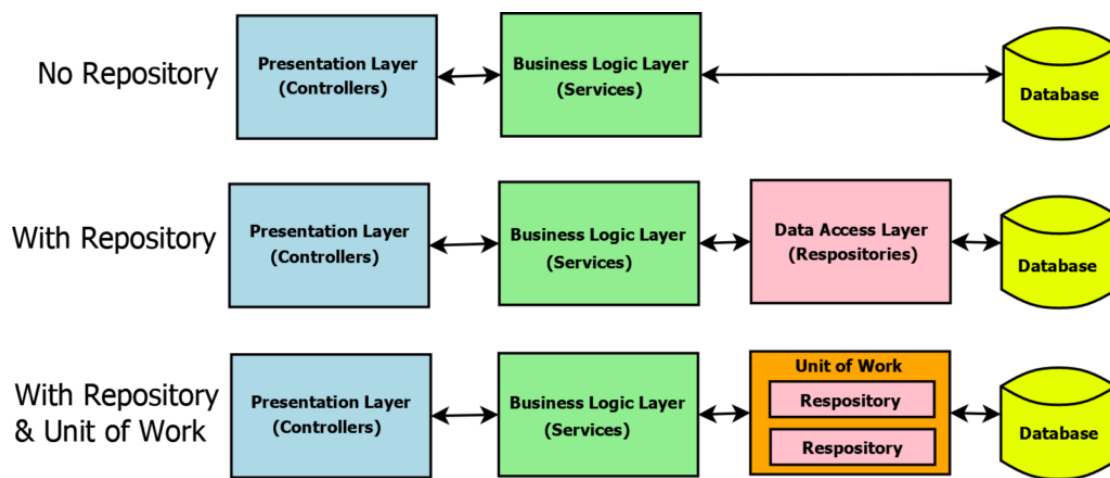


FIGURE 6.4: UnitOfWork and Repository

The Unit of Work pattern maintains a list of objects affected by a business transaction and coordinates the writing out of changes and the resolution of concurrency problems.

Its advantages include:

- **Transaction Management:** Ensures that all database operations within a single business transaction are completed successfully.
- **Optimized Performance:** Reduces the number of database calls by batching updates and committing them as a single transaction.
- **Concurrency Handling:** Manages changes in a way that reduces the likelihood of conflicts and ensures data consistency.

In our project, the Unit of Work pattern is implemented in the repository layer, where it coordinates the work of multiple repositories and manages transactions.

6.2.2 Repository Pattern

The Repository pattern abstracts the data access layer, providing a cleaner separation of concerns and a more manageable codebase. Its advantages include:

- **Decoupling:** Separates the business logic from the data access logic, making the system more modular and easier to maintain.
- **Testability:** Facilitates unit testing by allowing mock repositories to be used in tests, enabling isolated testing of the business logic.
- **Reusability:** Promotes reuse of data access code across multiple services and layers, reducing code duplication.

In our project, the Repository pattern is implemented in the repository layer, where generic repositories handle common data access operations and specific repositories manage more complex queries.

6.2.3 Specification Pattern

The Specification pattern encapsulates the logic for querying data, allowing for flexible and reusable query definitions. Its advantages include:

- **Flexibility:** Allows for complex query logic to be reused and composed dynamically, supporting advanced filtering, sorting, and pagination.
- **Readability:** Improves the readability of query logic by encapsulating it in dedicated classes, making it easier to understand and maintain.
- **Reusability:** Promotes reuse of query logic across different parts of the application, reducing code duplication and simplifying maintenance.

In our project, the Specification pattern is used in the repository layer to define complex queries in a reusable and maintainable manner.

6.2.4 Dependency Injection

Dependency Injection (DI) is a design pattern used to implement IoC, allowing for the creation of dependent objects outside of a class and providing those objects to a class in different ways. Its advantages include:

- **Flexibility:** Allows for different implementations of a dependency to be used interchangeably, promoting a more modular and adaptable system.
- **Testability:** Facilitates unit testing by allowing mock dependencies to be injected, enabling isolated testing of the system components.
- **Maintainability:** Reduces coupling between classes, making the system easier to maintain and evolve over time.

In our project, Dependency Injection is implemented throughout the layers, ensuring that dependencies are managed in a consistent and maintainable way.

6.3 SQL Server and Entity Framework

We used SQL Server as our database management system due to its robust features and seamless integration with .NET and Entity Framework.

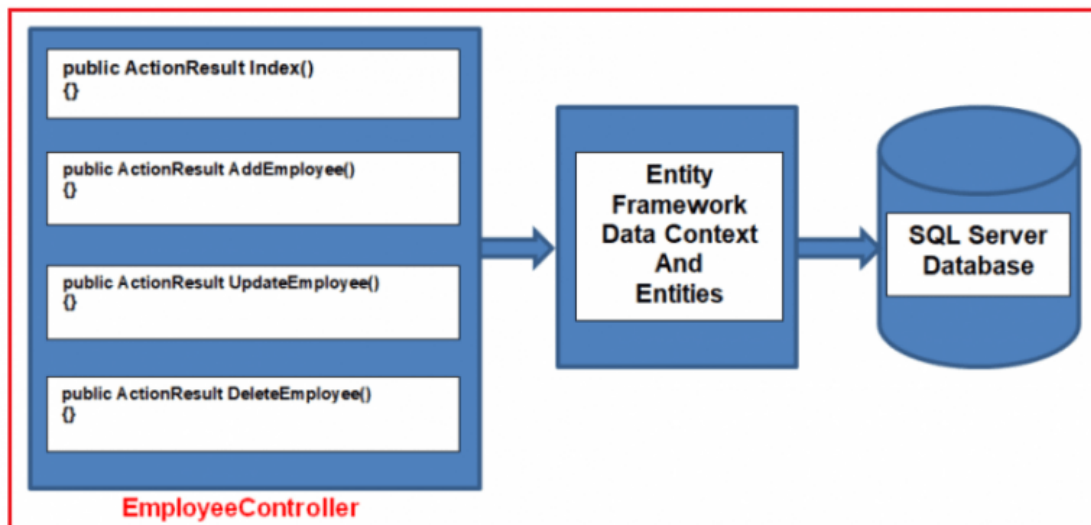


FIGURE 6.5: SQL Server and Entity Framework

6.3.1 Advantages of SQL Server

SQL Server offers numerous advantages that make it an ideal choice for our project:

- **Scalability:** SQL Server can handle large volumes of data and high transaction loads, making it suitable for enterprise-level applications.
- **Security:** SQL Server provides advanced security features, including encryption, authentication, and authorization, ensuring the safety and integrity of the data.

- **Performance:** SQL Server includes various optimization tools and techniques to enhance query performance, such as indexing, partitioning, and query optimization.
- **Integration:** SQL Server integrates well with other Microsoft products and services, including .NET and Azure, providing a seamless development and deployment experience.
- **High Availability:** SQL Server supports features like clustering, database mirroring, and Always On availability groups, ensuring high availability and disaster recovery.

6.3.2 Entity Framework

Entity Framework (EF) is an open-source ORM framework for .NET applications. It enables developers to work with data using .NET objects, eliminating the need for most data-access code. Its advantages include:

- **Productivity:** Reduces the amount of boilerplate code required for data access operations, allowing developers to focus on the business logic.
- **Maintainability:** Automatically manages the database schema based on the model, reducing the likelihood of schema mismatch issues and simplifying database migrations.
- **Flexibility:** Supports multiple database providers and can be easily switched without major changes to the codebase, promoting a more adaptable and resilient system.
- **Querying Capabilities:** Provides a rich querying capability with LINQ, enabling complex queries to be written in a type-safe manner and integrated seamlessly into the application.

In our project, Entity Framework is used to manage the data access layer, providing a clean and efficient way to interact with the SQL Server database.

6.4 Database Design

The database design for our project is intensive, catering to the needs of an e-learning system. Below is the design of our SQL Server database:

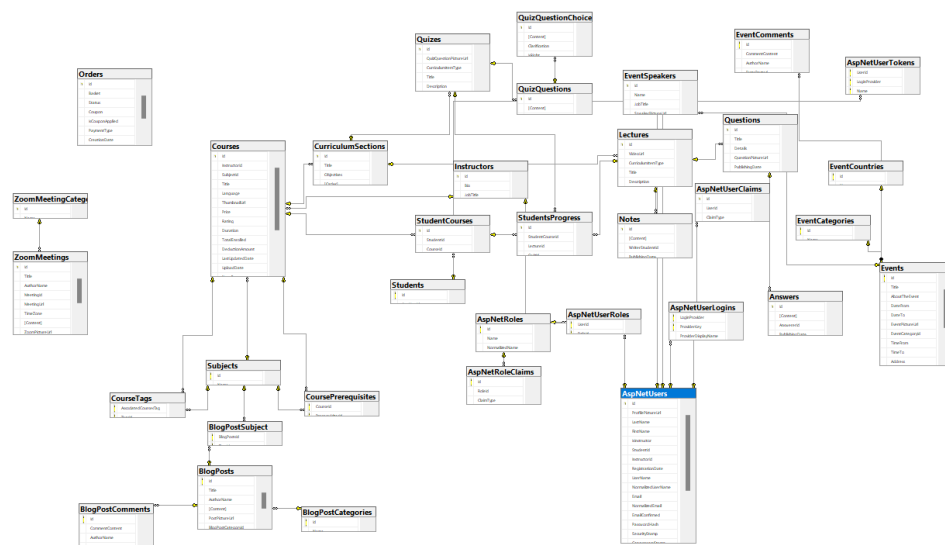


FIGURE 6.6: Intensive Database Design

Chapter 7

Performance Requirements

7.1 Performance Requirements

To specify performance requirements, it's a good idea to specify what impacts the performance.

7.1.1 Databases

Databases are accessed from the hard drive which can cause bottlenecks with network and memory. On average, database access can take in the scale of milliseconds (ms). The choice of the database stack will depend on how the data is stored and connected. For example, relational databases have low performance if the data is too connected, leading to a high number of joins. Therefore, it's a good idea to use a graph database instead.

7.1.2 Networks

Networking is a critical part of the system as its services are distributed. In terms of performance, the network average access time with REST API can be in the scale of microseconds (s). gRPC is considered faster in terms of decoding and encoding; it is 7 times faster in decoding or receiving and 10 times faster in terms of encoding or sending data compared to REST or JSON-based APIs.

Using server-based communication is acceptable, but to achieve lower server load, we plan to use P2P (peer-to-peer) to handle streaming between clients. This can boost performance by migrating the graphics processing to clients' machines.

7.1.3 Technology Stacks

The technology stacks used can significantly affect system performance. In our project, we utilize ASP.NET Web API for the backend. ASP.NET Web API is known for its high performance and scalability, particularly for handling multiple concurrent requests efficiently.

ASP.NET Core offers several features to enhance performance:

- **Asynchronous Programming:** ASP.NET Core supports asynchronous programming using the `async` and `await` keywords. This allows non-blocking operations, ensuring that the server can handle more requests simultaneously.

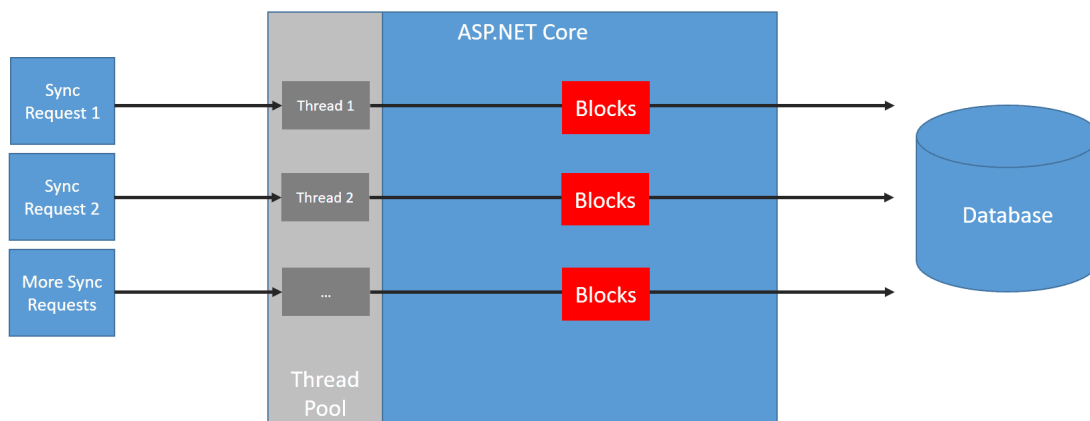


FIGURE 7.1: Asynchronous Programming

- **Dependency Injection:** Built-in dependency injection helps manage object lifetimes and improve application performance by reducing memory usage.
- **Kestrel Web Server:** The default web server for ASP.NET Core is Kestrel, which is highly optimized for speed and can handle a large number of connections with low latency.

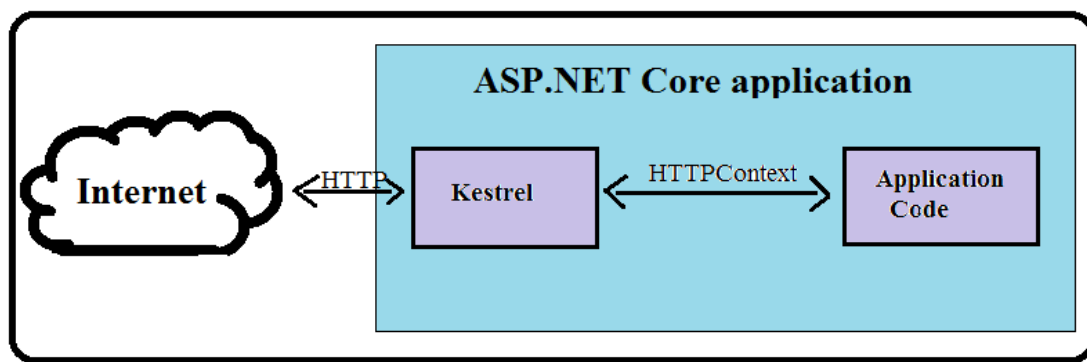


FIGURE 7.2: Kestrel Web Server

- **Parallelism:** ASP.NET Core can efficiently manage parallel tasks using the Task Parallel Library (TPL), which provides a powerful set of APIs for parallel programming.

These features ensure that our system can efficiently manage and process numerous requests concurrently, providing a responsive and robust user experience.

Chapter 8

Smart Typing: Autocorrect and Autocomplete with AI

8.1 Introduction

In the digital age, text input efficiency and accuracy are crucial, significantly enhanced by advances in artificial intelligence and deep learning. Autocomplete and autocorrect systems, now ubiquitous in our daily interactions, have evolved from simple rule-based algorithms to sophisticated models powered by deep learning. These systems predict and correct text in real-time, significantly improving our typing experience on various devices.

Deep learning, using multi-layered artificial neural networks, has greatly enhanced these systems' accuracy and functionality by analyzing language patterns to predict the next word or correct typographical errors. Innovations in neural network architectures, such as RNNs, LSTMs, and transformer models, have driven this evolution, enabling systems to understand and generate human language with high precision.

This chapter explores the principles, history, and technological advancements of autocomplete and autocorrect, highlighting the role of deep learning. It also discusses the challenges, limitations, ethical considerations, and future trends in text prediction and correction technologies.

These technologies enhance productivity and bridge communication gaps, making digital interactions more seamless and intuitive. Understanding the power and potential of deep learning in this context helps us appreciate the technological marvels shaping our modern, connected world.

8.2 The Importance in Web Design

In the realm of web design, autocomplete and autocorrect features play a crucial role in enhancing user experience (UX) by improving efficiency, accuracy, and overall satisfaction. These features are particularly beneficial in various aspects of website interaction, from search bars to form inputs and content creation.

- **Enhancing User Efficiency** Autocomplete helps users complete queries and entries faster by predicting the rest of the text based on initial input. This functionality significantly reduces the time users spend typing and minimizes errors, leading to a smoother and more efficient browsing experience. For instance, in an e-commerce website, autocomplete can assist users in quickly finding products by suggesting relevant items as they type in the search bar.

- **Improving Data Accuracy**

Autocorrect ensures that user inputs are accurately interpreted and processed by automatically correcting spelling mistakes and typos. This feature is indispensable in web forms where accurate data submission is critical, such as during user registration or when making online purchases. By preventing input errors, autocorrect helps maintain data integrity and improves the overall reliability of interactions on the website.

- **Enhancing User Experience**

From a UX perspective, autocomplete and autocorrect contribute to a more intuitive and user-friendly interface. By anticipating user intentions and providing real-time suggestions or corrections, these features reduce cognitive load and frustration associated with manual text input. This seamless interaction fosters a positive perception of the website and encourages users to engage more deeply with its content and functionalities.

8.3 Practical Use Cases

- **E-commerce Websites**

Autocomplete and autocorrect are indispensable in e-commerce websites for enhancing product search capabilities and streamlining the shopping experience. By suggesting relevant products and correcting search queries, these features help users find desired items quickly and accurately. This not only improves user satisfaction but also increases conversion rates as users can navigate and complete purchases more efficiently.

- **Content Management Systems (CMS)**

In content management systems, autocomplete and autocorrect assist content creators and editors in writing and editing articles, blog posts, or other forms of content. These features facilitate faster content creation by suggesting commonly used phrases, correcting spelling errors, and maintaining consistent language across the website. This enhances productivity and ensures that published content meets high editorial standards.

8.4 Implementation Approaches

1. **Rule-based Methods:** These methods rely on predefined dictionaries and algorithms that apply fixed rules and patterns to correct or suggest words. They are straightforward to implement and computationally efficient. Rule-based systems typically use techniques such as:

- **Pattern Matching:** Matching input against a set of predefined patterns to identify and correct errors.
- **Edit Distance Algorithms:** Calculating the minimum number of operations (insertions, deletions, substitutions) required to transform one word into another, such as Levenshtein distance.
- **Phonetic Algorithms:** Mapping words based on their pronunciation, like Soundex or Metaphone, to suggest similar-sounding words.

Rule-based methods excel in handling simple spelling errors and predefined language rules but may struggle with context-dependent corrections and evolving language use.

2. **Statistical Approaches:** These methods analyze large text corpora to infer probabilities and predict the next word or correct spelling errors based on statistical patterns. Common statistical techniques include:

- **n-gram Models:** Predicting the likelihood of a word based on the preceding n-1 words in the sequence.
- **Hidden Markov Models (HMMs):** Modeling sequences of observations (words) under the assumption that each word depends on a hidden state (context).
- **Probabilistic Context-Free Grammars (PCFGs):** Assigning probabilities to different parse trees of a sentence to predict the next word or correct syntactic errors.

Statistical approaches require extensive training data to build accurate models and can handle a wide range of language contexts. However, they may struggle with rare or unseen words and can be computationally intensive during training.

3. Machine Learning Approaches: These approaches leverage deep learning models to learn from large datasets and provide contextually accurate predictions and corrections. Key deep learning architectures include:

- **Recurrent Neural Networks (RNNs):** Suitable for sequential data, RNNs process inputs in a sequential manner, retaining information about previous inputs through hidden states.
- **Long Short-Term Memory Networks (LSTMs):** A type of RNN that addresses the vanishing gradient problem and can retain information over longer sequences.
- **Transformer Models:** Such as BERT (Bidirectional Encoder Representations from Transformers) and GPT (Generative Pre-trained Transformer), use self-attention mechanisms to capture relationships between words in a sentence without relying on sequential processing.

These machine learning approaches excel in capturing complex language patterns, understanding context, and providing accurate predictions. They require substantial computational resources for training large models but offer high adaptability and performance in real-world applications.

Advances in neural network architectures and training techniques continue to improve the capabilities of machine learning-based autocomplete and autocorrect systems, making them increasingly effective in enhancing user experience and productivity across digital platforms.

8.5 Autocorrect Task

In this work, we implemented an advanced autocorrect system using the pretrained Facebook BART model, a powerful transformer-based architecture known for its efficacy in text generation and correction tasks. The BART model was fine-tuned on a custom dataset specifically generated for autocorrect tasks, ensuring it learns the nuances of common typographical and grammatical errors. The fine-tuning process involved feeding the model pairs of incorrect and correct text samples, allowing it to adapt to the specific patterns of errors and corrections. This deep learning approach leverages the robust language understanding capabilities of BART, resulting in a highly accurate and context-aware autocorrect system. The system not only corrects

simple typographical errors but also improves overall sentence coherence and fluency, significantly enhancing user typing experience across various digital platforms.

8.5.1 Power of BART

The Bidirectional and Auto-Regressive Transformers (BART) is a transformer-based model introduced by **lewis-et al-2020-bart**. It combines the advantages of bidirectional and auto-regressive approaches in sequence-to-sequence modeling. BART is particularly well-suited for tasks such as text generation, summarization, and denoising.

8.5.1.1 Architecture

BART consists of an encoder-decoder architecture similar to other transformer models like BERT and GPT. The key components of BART include:

- **Encoder:** The encoder utilizes bidirectional self-attention mechanisms to capture contextual information from the input sequence.
- **Decoder:** The decoder uses auto-regressive attention mechanisms during generation, ensuring that each token is generated based on previously generated tokens.
- **Masked Language Modeling (MLM):** BART is pretrained using a masked language modeling objective, similar to BERT, to learn robust representations of text.

8.5.1.2 Applications

Due to its architecture, BART has shown promising results in various natural language processing tasks, including:

- Text summarization
- Document understanding
- Machine translation
- Data augmentation

In conclusion, BART represents a powerful transformer model that integrates bidirectional and auto-regressive capabilities. Its versatility makes it suitable for a wide range of tasks in natural language processing, contributing significantly to advancements in AI and language understanding.

8.5.2 Dataset Preparation

The dataset used for fine-tuning the BART model was specifically curated to reflect the text patterns and common errors found in the field of computer science and course reviews. This domain-specific dataset ensures that the autocorrect system is well-versed in the terminology, syntax, and typical mistakes associated with technical documents and academic reviews.

The dataset was compiled as a text file containing numerous sentences and paragraphs extracted from course reviews, lecture notes, and related academic content. This text data included a variety of typographical errors, grammatical mistakes, and context-specific anomalies that the autocorrect system would need to correct. By focusing on this specialized field, the fine-tuned model is tailored to provide more accurate and contextually appropriate corrections for users engaged in computer science education and review activities.

The preparation process involved the following steps:

- **Collecting a large corpus of text data from relevant sources.**
- **Introducing controlled errors to simulate common typing mistakes.**
- **Structuring the data into pairs of incorrect and correct text samples for model training.**

8.5.2.1 Data Augmentation and Cleaning Functions:

- **insert_characters(text, augmentation_probability):**
 - Randomly inserts characters into the text based on the given probability, excluding digits.
- **replace_characters(text, augmentation_probability):**
 - Randomly replaces characters in the text with other alphabetic characters based on the specified probability, excluding digits.
- **swap_characters_case(text, augmentation_probability):**

- Swaps the case (upper to lower or vice versa) of characters in the text based on the provided probability.
- **lower_case_words(text, augmentation_probability):**
 - Converts words in the text to lowercase based on the given probability, except for words that already start with lowercase letters.
- **cleanup(text):**
 - Cleans up the text by removing unwanted characters and preserving specific symbols, digits, and punctuation.
- **remove_punctuation(text):**
 - Removes all punctuation marks from the text, except those that appear between digits.
- **combine_sentences(text, sentences, augmentation_probability):**
 - Combines the input text with randomly selected sentences from a list based on the given probability.
- **delete_word(text, augmentation_probability):**
 - Deletes a randomly selected word from the text based on the specified probability, ensuring that short texts retain sufficient context.

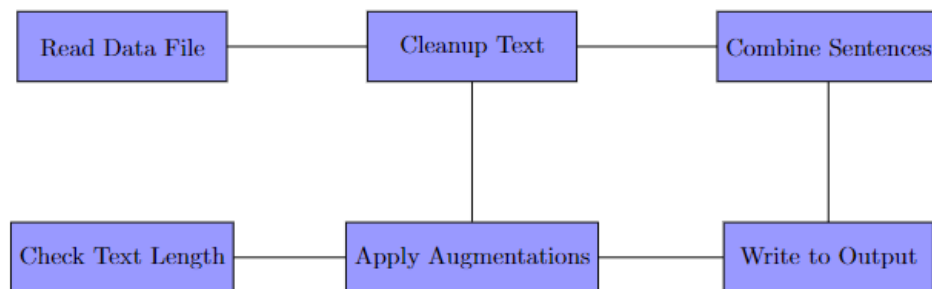


FIGURE 8.1: Sequence of Operations for Dataset Preparation

8.5.3 BART Fine-tune Process

Fine-tuning BART involves several key steps to adapt the pretrained model to a specific task or dataset. This process typically includes data preparation, model configuration, training, evaluation, and analysis of results.

8.5.3.1 Data Preparation

The first step in fine-tuning BART is preparing the training data. This involves selecting or curating a dataset that is relevant to the target task, preprocessing it into a format suitable for the model input, and dividing it into training, validation, and possibly test sets.

8.5.3.2 Model Configuration

Once the data is prepared, the next step is to configure the BART model for fine-tuning. This includes setting hyperparameters such as learning rate, batch size, number of training epochs, and any task-specific modifications to the model architecture or objective function.

8.5.3.3 Training Procedure

Training begins with initializing the BART model using the pretrained weights. The model is then optimized using a suitable optimization algorithm, such as Adam, and fine-tuned on the training data. During training, checkpoints may be saved periodically to monitor progress and prevent overfitting.

8.5.3.4 Evaluation Metrics

To assess the performance of the fine-tuned BART model, various evaluation metrics are employed. These metrics may include accuracy, BLEU score, ROUGE score, or task-specific metrics depending on the nature of the target task.

8.5.3.5 Results and Analysis

Finally, the results of fine-tuning BART are analyzed. This includes comparing the performance of the fine-tuned model against baseline models or previous state-of-the-art results, identifying strengths and weaknesses, and discussing potential areas for improvement or further research.

Metric	Value
Epoch	0.9998
Total FLOs	9,664,010 GF
Training Loss	0.152
Training Runtime	1:35:39.13
Training Samples	67,996
Samples per Second	11.848
Steps per Second	0.74

TABLE 8.1: Training Metrics

8.6 Autocomplete Task with GPT-2

The implementation showcases an advanced autocomplete system leveraging the capabilities of the GPT-2 (Generative Pre-trained Transformer 2) model. This section provides an in-depth overview of how the system operates and the model's significant role in enhancing user interaction and productivity.

8.6.1 Model Initialization

The `GPT2` class is designed to initialize the GPT-2 model and tokenizer, crucial components for transforming raw text inputs into meaningful predictions:

- **Tokenizer:** Converts input text into tokens that the GPT-2 model can process. It segments the text into smaller units, such as words or sub-words, to facilitate language understanding.
- **Model:** The GPT-2 model is a state-of-the-art transformer-based architecture developed by OpenAI. It excels in understanding and generating human-like text based on vast amounts of pre-existing data.
- **Device Management:** Utilizes GPU acceleration if available, ensuring efficient computation for rapid real-time predictions. This enhances the responsiveness of the autocomplete feature, crucial for seamless user interactions.

8.6.2 Model Used : GPT-2 (Generative Pre-trained Transformer 2)

- Developed by OpenAI, GPT-2 represents a breakthrough in natural language processing. It employs a transformer architecture that captures intricate language patterns and dependencies, enabling it to generate coherent and contextually relevant text.

- Pre-trained on diverse textual data, GPT-2 possesses a broad understanding of language semantics and syntax. This extensive training allows it to perform effectively across various text-based applications, including autocomplete tasks.
- The specific variant (`gpt2`) used in this implementation strikes a balance between model size and performance, making it suitable for real-time applications like autocomplete systems. Its ability to generate fluent text responses enhances user engagement and interaction quality.

This implementation exemplifies how advanced deep learning models like GPT-2 are revolutionizing user interactions through enhanced autocomplete capabilities. By leveraging sophisticated language understanding and generation techniques, GPT-2 significantly improves the efficiency and accuracy of text input across digital platforms.

Looking ahead, the continual evolution of transformer-based models promises further advancements in autocomplete systems. Future iterations may integrate more nuanced context awareness, multilingual support, and personalized user interactions, thereby elevating the user experience to new heights.

Integrating GPT-2 into applications empowers developers to create intelligent and responsive interfaces that anticipate user needs and streamline communication. As these technologies advance, they will continue to shape a future where digital interactions are more intuitive, efficient, and personalized.

8.6.3 Prediction Function

The `predict_next` method is pivotal in the autocomplete process:

- **Text Encoding:** Converts the input text into numerical tokens using the tokenizer. This step prepares the text for input into the GPT-2 model.
- **Model Inference:** Passes the encoded tokens through the GPT-2 model to generate predictions. The model predicts the likelihood of various word sequences following the input text.
- **Top- k Predictions:** Retrieves the top- k most probable next words or phrases based on their predicted probabilities. This approach ensures that the autocomplete suggestions are relevant and contextually accurate.

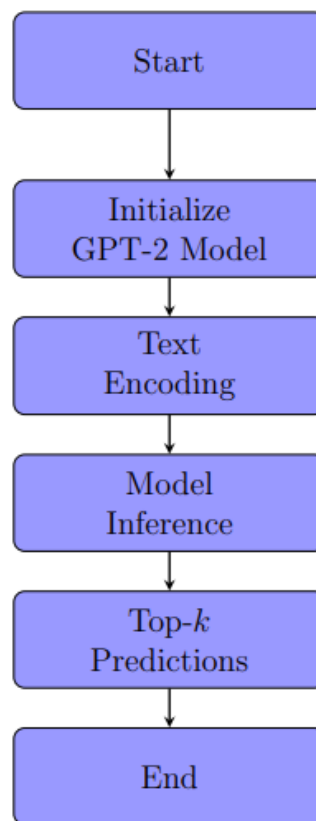


FIGURE 8.2: Flowchart of Autocomplete Task using GPT-2

8.7 Future Directions

The field of autocomplete and autocorrect systems is rapidly evolving, driven by advancements in artificial intelligence and natural language processing. Looking ahead, several emerging trends and potential advancements are shaping the future of these technologies, alongside notable challenges that must be addressed.

8.7.1 Emerging Trends in Autocomplete and Autocorrect Research

1. **Contextual Adaptability:** Future systems are expected to better understand and adapt to the context in which users operate. This includes considering user intent, historical context, and situational cues to provide more accurate predictions and corrections.
2. **Multilingual Support:** As digital interactions become increasingly global, there is a growing demand for autocomplete and autocorrect systems that can effectively handle multiple languages, dialects, and language varieties.

3. **Personalization and User Modeling:** Advances in machine learning techniques will enable systems to personalize suggestions based on individual user preferences, writing style, and behavioral patterns gathered from extensive user data.
4. **Real-time Feedback and Adaptation:** Systems may incorporate real-time feedback mechanisms to continuously improve suggestions based on immediate user interactions, enhancing responsiveness and accuracy.
5. **Integration with Voice Input:** With the rise of voice-enabled devices and applications, future systems may integrate seamlessly with voice input technologies, offering predictive text suggestions and real-time corrections during voice interactions.

8.7.2 Potential Advancements and Challenges

1. **Enhanced Accuracy and Precision:** Advancements in deep learning models, such as enhanced transformer architectures and more sophisticated training techniques, will likely lead to significant improvements in prediction accuracy and correction precision.
2. **Handling Ambiguity and Contextual Nuances:** Addressing challenges related to understanding and disambiguating complex language nuances, slang, idiomatic expressions, and context-dependent meanings will be crucial for future systems.
3. **Ethical and Fair AI Practices:** As these systems become more pervasive, ensuring fairness, transparency, and accountability in their design, implementation, and deployment will remain critical to mitigate biases and uphold user trust.
4. **Privacy and Data Security:** Stricter regulations and advancements in privacy-preserving techniques will be essential to safeguard user data collected for training and improving these AI-driven systems.
5. **User Empowerment and Control:** Providing users with greater control over how autocomplete and autocorrect features operate, including options for customization, privacy settings, and opt-out mechanisms, will become increasingly important.

8.7.3 Speculations on Future Integration with AI and Natural Language Understanding

1. **Conversational AI and Assistants:** Autocomplete and autocorrect systems may evolve to seamlessly integrate with conversational AI platforms, enhancing natural language understanding and response generation in virtual assistants.

2. **Semantic Understanding and Knowledge Graphs:** Future systems could leverage advancements in semantic understanding and knowledge graphs to provide more contextually relevant suggestions and corrections based on a deeper understanding of user queries and content.
3. **Augmented Writing Assistance:** Beyond basic correction and prediction, future systems may offer advanced writing assistance features, such as style improvement suggestions, content generation support, and collaborative writing tools.
4. **Cross-platform Integration:** Integration across diverse digital platforms, including social media, messaging apps, productivity tools, and IoT devices, will enable consistent and seamless user experiences across different contexts.
5. **Human-AI Collaboration:** There is potential for future systems to foster closer collaboration between humans and AI, supporting creative writing processes, enhancing productivity, and promoting more effective communication in digital environments.

8.8 Deployment with Flask

After training and fine-tuning your machine learning model, the next crucial step is deploying it into production. Flask, a lightweight web framework in Python, provides an efficient way to create web applications and APIs, making it suitable for serving machine learning models.

- **Setting Up the Flask Application** Begin by setting up your Flask application. Install Flask using pip if it's not already installed (`pip install Flask`). Create a directory structure for your project and initialize a Flask application.
- **Model Serialization** Serialize your trained machine learning model using libraries like Pickle or Joblib. This allows you to save the model in a file format that can be easily loaded within your Flask application.
- **Creating Flask Routes** Define routes within your Flask application to handle incoming HTTP requests and serve predictions. For instance, create a route `/predict` that accepts POST requests containing input data for prediction.
- **Loading the Model** Within your Flask route, load the serialized model. Deserialization typically involves reading the model file and any required preprocessing steps for input data.

- **Handling Predictions** Implement logic in your Flask route to preprocess incoming data, feed it into the loaded model for prediction, and return the predicted results as an HTTP response.
- **Deploying to a Web Server** Choose a web server for deploying your Flask application. Options include deploying to cloud platforms like Heroku or AWS EC2, or setting up a local deployment for testing purposes.
- **Testing and Validation** Thoroughly test your deployed Flask application to ensure it handles various inputs correctly and produces accurate predictions. Validate the integration of your machine learning model with the Flask API endpoints.
- **Monitoring and Maintenance** Set up monitoring mechanisms to track the performance of your deployed model in production. Plan for ongoing maintenance tasks such as updating dependencies and periodically retraining the model.

Flask simplifies the process of integrating machine learning models into scalable web applications by providing a flexible and robust framework for building APIs. Flask provides a straightforward way to create RESTful APIs for serving machine learning models, making it a popular choice for deploying predictive models into production environments. By following these steps, you can effectively deploy your trained machine learning model using Flask, enabling real-time predictions in production environments.

8.9 Conclusion

In this chapter, we have explored the evolution, functionalities, and impacts of autocomplete and autocorrect technologies, particularly in the context of advancements driven by deep learning. These technologies have revolutionized text input across digital platforms, from mobile devices to web applications, by predicting and correcting text in real-time, thereby enhancing user efficiency, accuracy, and overall experience.

Key points discussed include the foundational principles of autocomplete and autocorrect systems, their evolution from rule-based approaches to sophisticated deep learning models, and their widespread applications beyond text messaging. We examined how deep learning techniques, such as recurrent neural networks (RNNs), long short-term memory (LSTM) networks, and transformer models like BERT and GPT, have significantly improved the accuracy and contextual understanding of these systems.

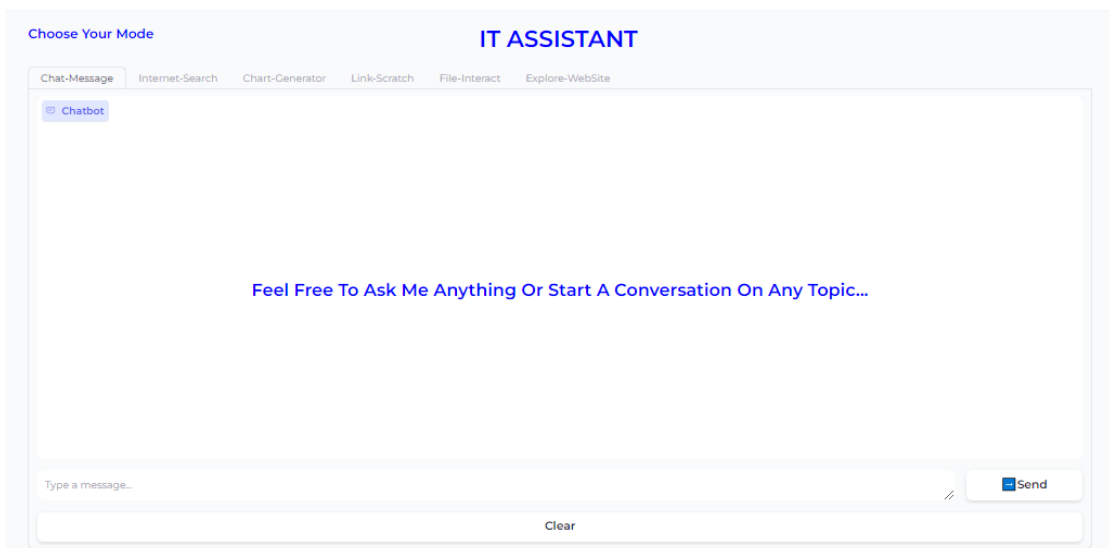
Looking forward, the future role of deep learning in enhancing autocomplete and autocorrect technologies appears promising. Emerging trends such as contextual adaptability, multilingual support, and real-time feedback mechanisms are poised to further refine these systems, making them more intuitive and responsive to user needs.

Challenges related to bias mitigation, privacy concerns, and ensuring user empowerment will need careful consideration to foster inclusive and ethical implementations.

In conclusion, deep learning continues to drive innovation in autocomplete and autocorrect technologies, offering unprecedented capabilities in understanding and predicting human language. As research progresses and technology evolves, these advancements will continue to shape a future where digital interactions are more seamless, efficient, and personalized. By leveraging the power of deep learning responsibly, we can unlock new possibilities and enhance the way we communicate and interact in the digital age.

Chapter 9

Development of an AI-Powered Chatbot Assistant: Advanced Features and Implementation



9.1 Introduction

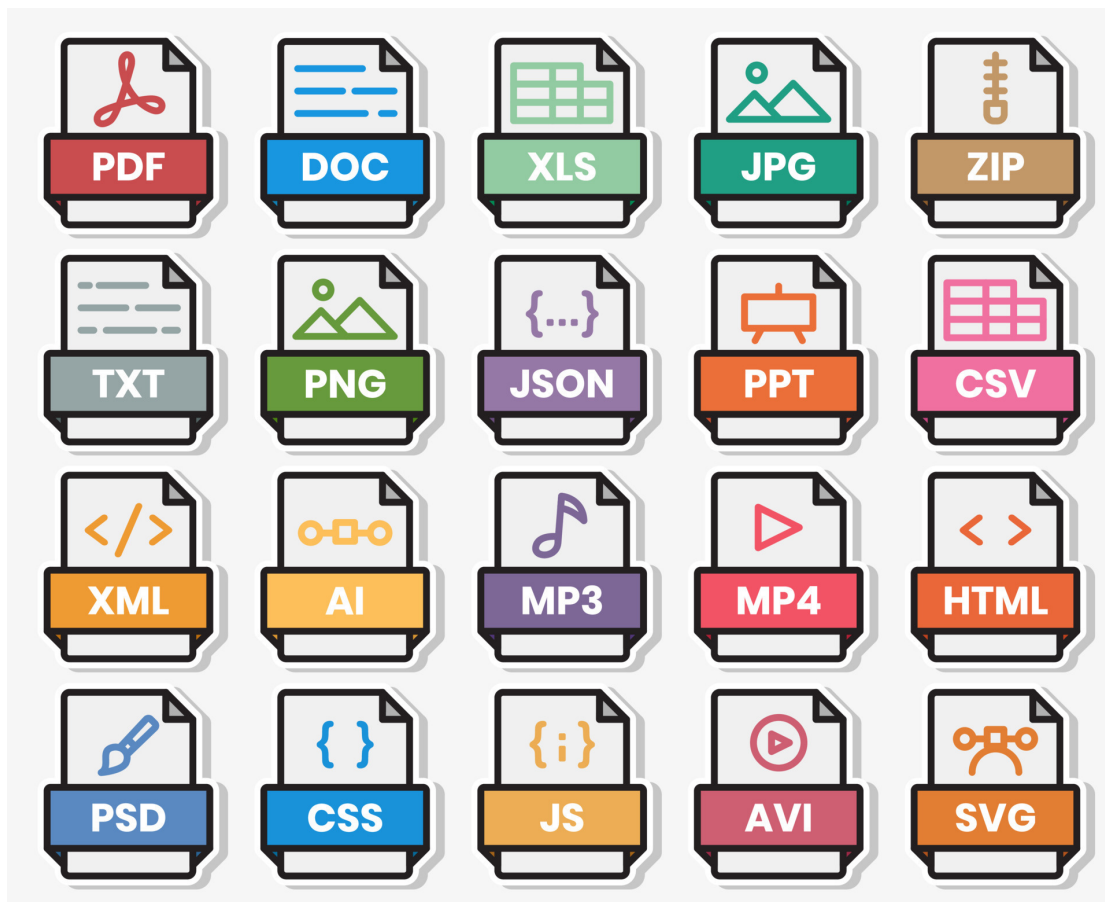
In this chapter, we delve deeper into the development and advanced capabilities of our AI-powered chatbot assistant, specifically focusing on enhancing text and code processing tasks. This Python-based implementation integrates cutting-edge artificial intelligence techniques, including natural language processing (NLP), machine learning models, and web scraping, to enable efficient data extraction, analysis, and user interaction.

9.2 Chatbot System Overview

The chatbot system is designed to cater to a broad spectrum of tasks crucial for contemporary data-driven applications. Its modular architecture emphasizes reusability and scalability, ensuring robust performance across diverse domains. By seamlessly integrating advanced AI techniques such as NLP, machine learning models, and web scraping, the chatbot excels in extracting, analyzing, and interacting with data, significantly enhancing productivity in text and code processing workflows.

9.3 Key Functionalities

9.3.1 Imports and Environment Setup



The script begins by importing essential libraries and configuring the environment to ensure compatibility and functionality across various platforms. These libraries encompass tools for comprehensive file handling (e.g., Pandas for CSV and Excel), robust NLP frameworks (NLTK for text processing), versatile web scraping tools

(BeautifulSoup for HTML parsing), and sophisticated AI model integration (HuggingFace Transformers for language modeling).

9.3.2 File Reading and Content Extraction

A primary functionality of the chatbot is its robust file handling capabilities, supporting diverse formats such as CSV, text, PDF, DOCX, PPTX, XLSX, JSON, HTML, and XML. Error handling mechanisms are implemented to ensure seamless data extraction from these formats, critical for processing a wide array of textual information sources.

9.3.3 Web Scraping for Dynamic Content Retrieval

Complementing static file handling, the chatbot utilizes advanced web scraping techniques powered by BeautifulSoup. This enables dynamic extraction of textual content from web pages, proficiently handling HTTP requests and parsing HTML to retrieve pertinent information.

9.3.4 Text Processing and Normalization



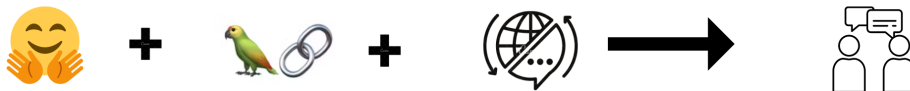
Ensuring consistency in subsequent processing tasks, the chatbot employs sophisticated text normalization techniques. These include converting text to lowercase, removing punctuation, and employing NLTK for tokenization. These preprocessing steps optimize textual data for deeper analysis and interaction with AI models.

9.3.5 Integration of AI Models

At the core of its functionality lies the integration of powerful AI models sourced from HuggingFace's repository. These include state-of-the-art language models facilitating conversational AI for interactive user sessions and SentenceTransformer for generating embeddings crucial for semantic search and similarity computations.

9.4 Model Selection

Open Source LLM's: Langchain and Hugging face



9.4.1 Introduction

Selecting an appropriate model for the chatbot involves evaluating multiple factors beyond its capacity to generate responses. This section explores the criteria used to select a model suitable for the envisioned application.

9.4.2 Model Suitability

Considerations for model suitability include adherence to chat templates, capability in managing dynamic interactions, and adaptation to diverse user contexts.

9.4.3 Inference Time Speed

The responsiveness of the chatbot to user inputs is pivotal for ensuring a seamless user experience. Models with faster inference times significantly enhance user satisfaction.

9.4.4 Context Window Size

The size of the context window influences the chatbot's ability to maintain coherence and relevance in conversations. A larger context window allows the chatbot to consider more preceding interactions, thereby improving response quality.

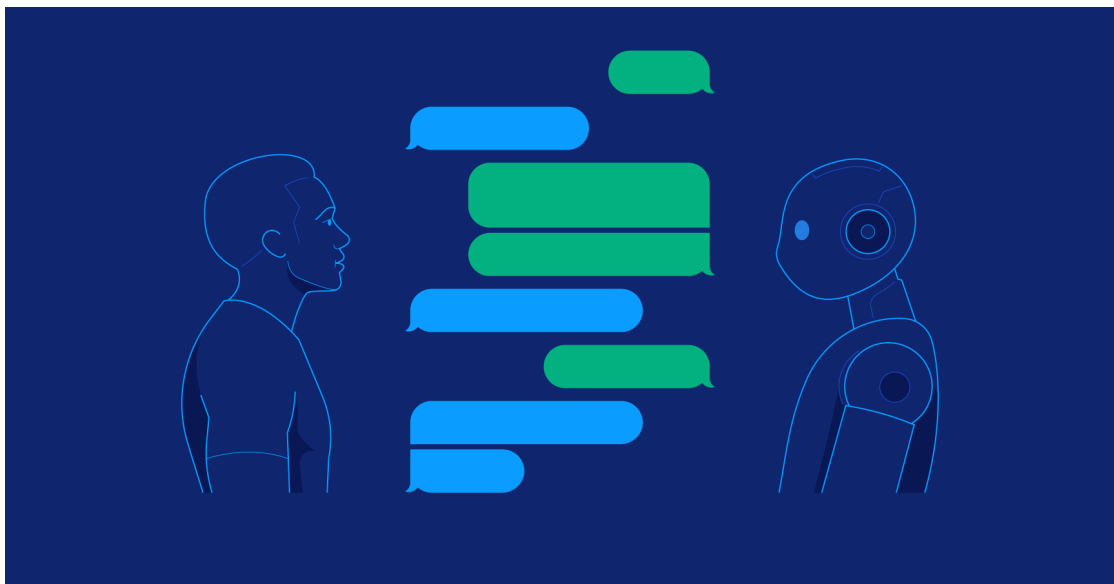
9.4.5 Memory vs Performance Trade-off

Optimal parameter settings balance model size (memory usage and number of parameters) with performance metrics like speed and accuracy, optimizing computational resource utilization without compromising functionality.

9.4.6 Model Update Frequency

Regular updates of models with the latest data and improvements are crucial for maintaining relevance and accuracy in responses.

9.4.7 Domain-Specific Competence



Not all models excel uniformly across different domains. While some may perform well in general topics, they may struggle with specialized fields like programming or IT.

Selecting a model aligned with the intended application context is essential.

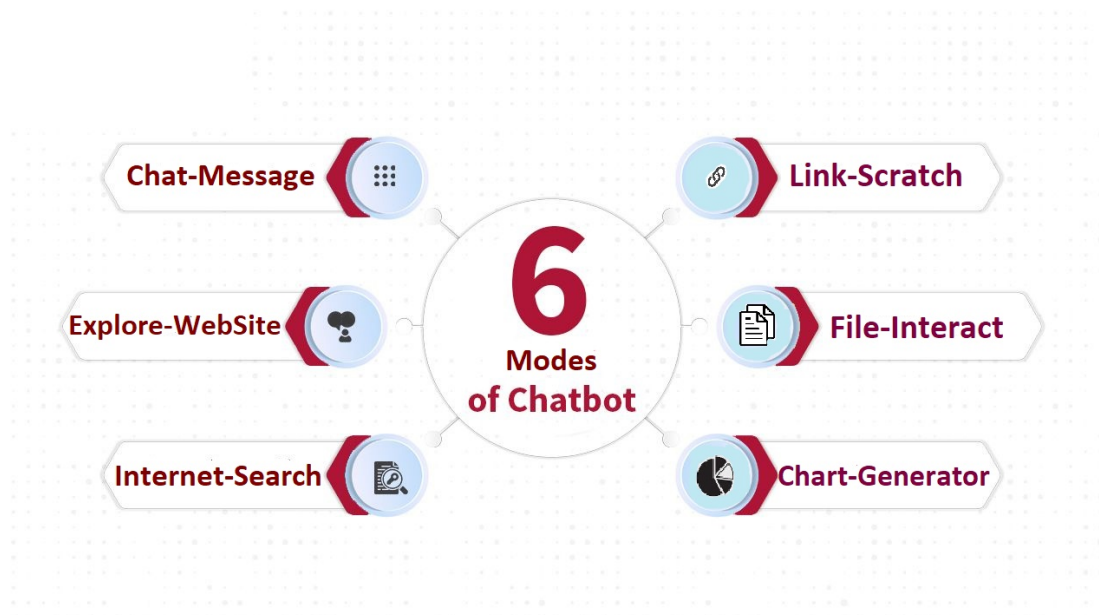
9.4.8 Selected Model

Detailed insights into the chosen model, such as `HuggingFaceH4/starchat2-15b-v0.1`, including its inference speed, context window size (e.g., 16k), and suitability for the application, are provided.

9.5 Setting Template and Memory

9.5.1 Challenges of Traditional Models

Traditional chatbot models often necessitate inputs and outputs in predefined formats, posing challenges such as fixed input-output structures, lack of memory for ongoing interactions, and limitations in contextual message comprehension.



9.5.2 Model Parameters

Critical parameters influencing the chatbot's behavior and output quality, including temperature (response randomness control), `top_k` and `top_p` (diversity and sampling control), `repetition_penalty` (repetitive response avoidance), and `max_new_tokens` (response length limitation), are detailed.

9.6 Modes Implementation

9.6.1 Traditional Chatting Mode

Explores the primary mode where the chatbot engages in general conversation topics, highlighting its functionality and typical use cases.

9.6.2 Chart Generator Mode

Outline of the chatbot's ability to generate graphs and charts based on user queries and inputs, extending its utility beyond text-based interactions to support data visualization needs.

9.6.3 Internet Search Mode



Implementation of an internet search mode utilizing the RAG (Retrieved Augmented Generation) technique, enabling real-time information retrieval from the web to enrich responses.

9.6.4 Link Scratch Mode

Details on a mode where users provide links for the chatbot to analyze and discuss content, explaining how the chatbot processes and extracts relevant information from provided links.

9.6.5 File Interact Mode

Functionality enabling users to upload diverse file types (e.g., documents, spreadsheets) for the chatbot's analysis and response to related queries concerning the uploaded content.

9.6.6 Explore Website Mode

Discussion on the chatbot's role in aiding users to navigate specific websites, efficiently retrieving information such as course details or blog posts.

9.7 Framework Construction

9.7.1 Frontend Development



Insight into the chatbot's frontend interface development using the Gradio framework, emphasizing design considerations and implemented user interface elements to facilitate seamless interaction.

9.7.2 Input and Output Setup

Considerations for setting up input types (e.g., buttons, text areas) and tailored output displays for each operational mode, ensuring user interaction clarity and usability.

9.7.3 Launch Gradio App

Optimization of Gradio app settings for concurrent processing, multi-user support, session management, and efficient thread handling, enhancing the chatbot application's scalability and responsiveness.

Chapter 10

Optimizing AI Models for E-Learning: Machine Translation, Question Generation, and Text Summarization

10.1 Introduction

In the digital age, the landscape of education has been transformed by technology, making learning more accessible and engaging for students worldwide. One of the most significant advancements in this realm is the integration of artificial intelligence (AI) into e-learning platforms. AI features not only enhance the delivery of educational content but also personalize the learning experience to meet the diverse needs of students. In this chapter, we will delve into three key AI features—machine translation, text summarization, and question generation—implemented in an e-learning website to revolutionize how courses are delivered and consumed.

- **Enhancing Accessibility with Machine Translation**

Language barriers have long been a challenge in the field of education, limiting the reach of quality educational content. The machine translation feature embedded in the e-learning website addresses this issue by allowing users to choose the language of the accompanying scripts for educational videos. Whether a student prefers Arabic or English, the AI-powered translation ensures that the content is accurately and swiftly converted, maintaining the original context and meaning.

This feature not only broadens the accessibility of courses but also empowers students from different linguistic backgrounds to learn without constraints, fostering a more inclusive educational environment.

- **Streamlining Learning with Text Summarization**

In today's fast-paced world, students often seek efficient ways to review and digest information. The text summarization feature leverages advanced AI algorithms to distill lengthy educational texts into concise and coherent summaries. After watching an educational video, students can quickly access the key points and main ideas, saving time and enhancing their understanding of the material. This feature is particularly beneficial for students who need to review large volumes of information or prepare for exams, as it provides a quick reference to the essential content without the need to rewatch entire videos or reread extensive texts.

- **Enhancing Comprehension with Question Generation**

Assessment and self-evaluation are crucial components of the learning process. The question generation feature uses AI to create relevant and thought-provoking questions based on the content of the educational videos. This allows students to test their knowledge and understanding of the material in a structured manner. By interacting with these AI-generated questions, students can identify areas where they need further study and reinforcement, leading to a deeper comprehension of the subject matter. This interactive approach to learning not only engages students but also promotes active recall, a proven technique for improving memory and learning outcomes.

By integrating these AI features, the e-learning website not only enhances the quality of education but also provides a more personalized and efficient learning experience. As we explore each of these features in detail, we will see how AI is transforming the educational landscape, making learning more accessible, streamlined, and effective for students around the globe.

10.2 **Effective Methods for Fine Tuning AI Models**

Fine-tuning is the process we use to further train an already pre-trained LLM on a smaller, task-specific, labeled dataset. In this way, we adjust some of the model parameters to optimize its performance for a particular task or set of tasks. In full fine-tuning, all the model parameters are updated, making it similar to pretraining—just that it's done on a labeled and much smaller dataset.

10.2.1 Full Fine-Tuning in 6 Steps

To illustrate, let's say we want to build a tool that generates abstracts for biotechnology research papers. For full fine-tuning, you'll need to go through the following steps:

1. Create the Dataset

Collect a set of research papers from the target domain of biotechnology. Ensure each paper comes with its original abstract. Split this collection into training, validation, and test sets.

2. Preprocess the Data

Convert each research paper into a format amenable to the model. Pair each processed paper content with its corresponding abstract to form input-output pairs for supervised training.

3. Configure the Model

Load the pre-trained LLM (e.g., a pre-trained version of GPT-4). Decide on hyperparameters for fine-tuning, such as learning rate, batch size, and number of epochs, based on preliminary tests or domain knowledge.

4. Train the Model

Feed the processed content to the LLM as input and train it to generate the corresponding abstract as the output. Monitor the model's performance on the validation set to prevent overfitting and to decide when to stop training or make adjustments.

5. Evaluate Performance

Once fine-tuning is complete, assess the model's performance on the test set, which it hasn't seen before. Metrics might include the BLEU score, ROUGE score, or human evaluations to measure the quality and relevance of the generated abstracts as compared to the original ones.

6. Iterate Until Performance is Satisfactory

Based on evaluation results, iterate on the above steps, possibly collecting more data, adjusting hyperparameters, or trying different model configurations to improve performance.

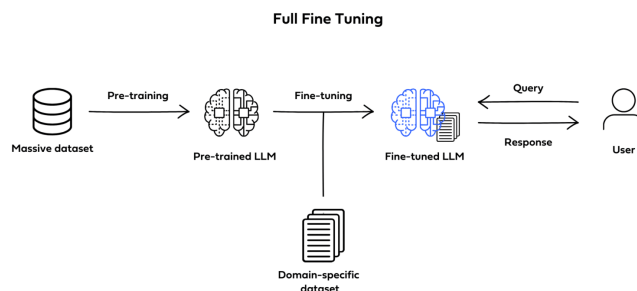


FIGURE 10.1: Full Fine Tuning Process.

10.2.2 What Makes Full Fine-Tuning Worthwhile?

- **Requires Less Data than Training from Scratch**

Full fine-tuning can be effective even with a relatively small task-specific dataset. The pretrained LLM already understands general language constructs. The fine-tuning process focuses primarily on adjusting the model's knowledge to the specifics of the new data. A pretrained LLM, initially trained on roughly 1 trillion tokens and demonstrating solid general performance, can be efficiently fine-tuned with just a few hundred examples, equivalent to several hundred thousand tokens.

- **Enhanced Accuracy**

By fine-tuning on a task-specific dataset, the LLM can grasp the nuances of that particular domain. This is especially vital in areas with specialized jargon, concepts, or structures, such as legal documents, medical texts, or financial reports. As a result, when faced with unseen examples from the specific domain or task, the model is likely to make predictions or generate outputs with higher accuracy and relevance.

- **Increased Robustness**

Fine-tuning allows us to expose the model to more examples, especially edge cases or less common scenarios in the domain-specific dataset. This makes the model better equipped to handle a wider variety of inputs without producing erroneous outputs.

10.2.3 The Downside of Full Fine-Tuning

- **High Computational Costs**

Full fine-tuning involves updating all the parameters of a large model. With large-scale LLMs boasting tens or hundreds of billions of parameters, training requires

enormous amounts of computing power. Even when the fine-tuning dataset is relatively small, the number of tokens can be huge and expensive to compute.

- **Substantial Memory Requirements**

Working with large models can necessitate specialized hardware, such as high-end GPUs or TPUs, with significant memory capacities. This is often impractical for many businesses.

- **Time & Expertise Intensive**

When the model is very large, you often need to distribute the computation over multiple GPUs and nodes. This requires appropriate expertise. Depending on the size of the model and the dataset, fine-tuning can take hours, days, or even weeks.

10.2.4 Parameter-Efficient Fine-Tuning

Parameter-efficient fine-tuning (PEFT) uses techniques to further tune a pretrained model by updating only a small number of its total parameters. A large language model pretrained on vast amounts of data has already learned a broad spectrum of language constructs and knowledge. In short, it already possesses much of the information needed for many tasks. Given the smaller scope, it's often unnecessary and inefficient to adjust the entire model. The fine-tuning process is conducted on a small set of parameters.

PEFT methods vary in their approach to determining which components of the model are trainable. Some techniques prioritize training select portions of the original model's parameters. Others integrate and train smaller additional components, like adapter layers, without modifying the original structure.

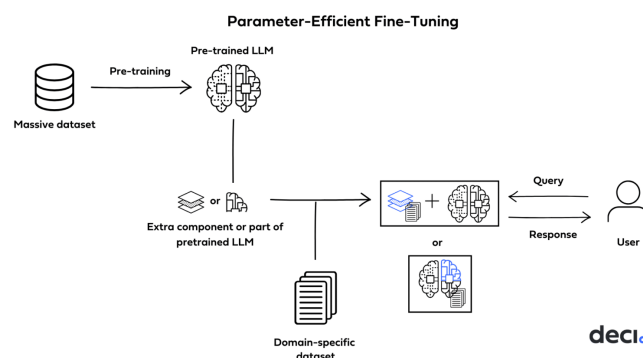


FIGURE 10.2: Parameter-Efficient Fine-Tuning Process.

10.3 LORA (Low-Rank Adaptation).

LoRA, short for Low-Rank Adaptation of Large Language Models, was introduced in early 2023. It has since become the most commonly used PEFT method, helping companies and researchers reduce their fine-tuning costs. Using reparameterization, this technique downsizes the set of trainable parameters by performing low-rank approximation.

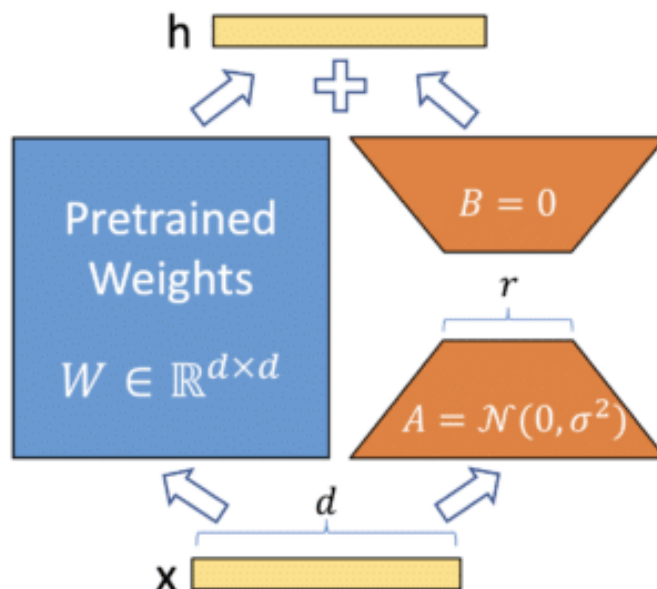


FIGURE 10.3: Illustration of the LoRA technique.

For example, if we have a $100,000 \times 100,000$ weight matrix, then for full fine-tuning we need to update 10,000,000,000 parameters. Using LoRA, we can capture all or most of the crucial information by using a low-rank matrix that contains the selected parameters updated during fine-tuning.

To get this low-rank matrix, we can reparametrize the original weight matrix into two matrices, A and B , each of low rank r . Our new low-rank matrix is then taken to be the product of A and B . If $r = 2$, we end up updating $(100,000 \times 2) + (100,000 \times 2) = 400,000$ parameters instead of 10,000,000. By updating a much smaller number of parameters, we reduce the computational and memory requirements needed for fine-tuning. The following graphic from the original paper on LoRA illustrates the technique.

10.3.1 Advantages of LoRA

The following are some of the advantages of LoRA:

- **Task Switching Efficiency** — Creating different versions of the model for specific tasks becomes easier. You can simply store a single copy of the pretrained weights and build many small LoRA modules. When you switch from task to task, you replace only the matrices, A and B , and keep the LLM. This significantly reduces storage requirements.
- **Fewer GPUs Required** — LoRA reduces GPU memory requirements by up to 3x since we don't calculate/retrain gradients for most parameters.
- **High Accuracy** — On a variety of evaluation benchmarks, LoRA performance proves to be almost equivalent to full fine-tuning—at a fraction of the cost. For example, Deci's DeciLM 6B was fine-tuned using LoRA for instruction following. The fine-tuned model, DeciLM 6B-Instruct matches the performance of top-tier models in its class.

Because it's so new, LoRA's efficacy in situations where you need to fine-tune a model on multiple tasks is still untested. In such situations, the pre-trained LLM needs to be fine-tuned on each task sequentially, and it remains to be seen whether LoRA can maintain the accuracy of full fine-tuning.

10.3.2 Where PEFT beats full fine-tuning:

- **More Efficient and Faster Training** — Fewer parameter tweaks mean fewer calculations, which translates directly into speedier training sessions that require less computing power and fewer memory resources. This makes fine-tuning in resource-tight scenarios more practical.
- **Preserving Knowledge from Pretraining** — Extensive pre-training on broad datasets packs models with invaluable general knowledge and capabilities. With PEFT, we ensure that this treasure trove isn't lost when adapting the model to novel tasks because we retain most or all of the original model weights.

Whether or not PEFT is an effective alternative to full fine-tuning depends on the use case and the particular PEFT technique selected. In PEFT you train a much smaller number of parameters than in full fine-tuning and, if the task is “hard enough,” the difference in the number of parameters trained will show.

10.4 Machine Translation Feature

In the realm of e-learning, effective communication across languages is crucial for reaching a diverse audience. To address this need, we implemented a machine translation feature on our e-learning platform that translates content between Arabic and English. For this task, we chose the Marian model, a state-of-the-art neural machine translation model, specifically the `helsinki/nlp-mt-en-ar` and `helsinki/nlp-mt-ar-en` models.

• Challenges and Solutions

Implementing a machine translation feature posed several challenges:

- **Handling Ambiguities:** Machine translation can struggle with ambiguous phrases. We addressed this by fine-tuning the models on a domain-specific dataset to improve accuracy.
- **Real-Time Translation:** Ensuring that translations occur in real time on the platform required optimizing the model's inference process and deploying efficient hardware solutions.

Overall, the Marian model provided a robust solution for our machine translation needs, enhancing the accessibility of our e-learning content across multiple languages.

10.4.1 Model Architectural Overview

• Transformer Architecture

At the heart of the Marian model is the transformer architecture, introduced by Vaswani et al. in their seminal paper “Attention is All You Need” (2017). The transformer architecture is the foundation for many modern NMT systems due to its ability to handle long-range dependencies and parallelize training processes.

- **Encoder-Decoder Structure:** Marian follows the classic encoder-decoder structure of the transformer model. The encoder processes the input text (source language), while the decoder generates the translated text (target language).
- **Self-Attention Mechanism:** The model uses self-attention mechanisms to weigh the importance of different words in a sentence, allowing it to capture contextual relationships effectively. The self-attention mechanism enables the

model to focus on relevant parts of the input text when generating translations.

- **Multi-Head Attention:** Marian employs multi-head attention, which allows the model to simultaneously attend to different parts of the input sequence. This approach enhances the model's ability to understand various aspects of the text.

- **Layers and Components**

- **Encoder Layers:** Marian's encoder consists of a series of identical layers, each containing a multi-head self-attention mechanism and a position-wise feed-forward network. These layers extract features from the input text and transform them into a high-dimensional representation.
- **Decoder Layers:** The decoder also consists of a series of identical layers, each containing multi-head self-attention, encoder-decoder attention, and a position-wise feed-forward network. These layers generate the output text based on the encoder's representation and previously generated tokens.
- **Positional Encoding:** Since transformers do not have an inherent understanding of token order, Marian uses positional encodings to inject information about the position of each token in the sequence.

- **Features and Capabilities**

- **High Translation Quality:** Marian has achieved impressive performance on various translation benchmarks. Its effectiveness stems from several key features:
 - * **Pre-trained Models:** Marian offers a range of pre-trained models for different language pairs, including `helsinki/nlp-mt-en-ar` for English-to-Arabic translation and `helsinki/nlp-mt-ar-en` for Arabic-to-English translation.
 - * **Fine-Tuning Flexibility:** The model's architecture supports fine-tuning on domain-specific datasets, allowing for adaptations to particular fields like computer science and programming.
- **Scalability and Efficiency:**
 - * **Scalable Training:** Marian is designed to handle large-scale training tasks efficiently. It supports distributed training across multiple GPUs, enabling the model to be trained on extensive datasets without prohibitive computational costs.

- * **Fast Inference:** Marian's architecture is optimized for fast inference, which is essential for real-time translation tasks on an e-learning platform.
- **Multilingual Capabilities:**
 - * **Multilingual Training:** Marian is capable of handling multiple language pairs simultaneously. This multilingual capability is facilitated by the model's architecture, which supports the translation of various languages through a shared representation space.
- **Open Source and Community Support:**
 - * **Open Source Framework:** Marian is an open-source project, which means that the model and its source code are freely available for modification and use. This openness has fostered a strong community of developers and researchers who contribute to the model's improvement.
- **Practical Application in the E-Learning Project**
 - **Selection of Pre-trained Models:** For our e-learning project, we selected the pre-trained models `helsinki/nlp-mt-en-ar` and `helsinki/nlp-mt-ar-en` from the Marian model suite. These models were chosen based on their high performance in translating between English and Arabic, as demonstrated by various benchmarks and evaluations.
 - * `helsinki/nlp-mt-en-ar`: This model translates from English to Arabic. We used this model to convert course materials, tutorials, and educational content from English into Arabic for our e-learning platform.
 - * `helsinki/nlp-mt-ar-en`: This model translates from Arabic to English. We employed this model to translate Arabic educational content into English, ensuring that Arabic-speaking users could access resources in both languages.
 - **Fine-Tuning the Marian Models:**
 - * **Custom Dataset:** To adapt the pre-trained models for our specific domain, we fine-tuned them using a custom dataset focused on computer science and programming. This process involved updating the models with new data to improve their accuracy and relevance for translating technical content.
 - * **Training Process:** The fine-tuning involved several steps, including data preprocessing, model training, and evaluation. We used the combined dataset of OPUS-100 and our custom data to achieve high-quality translations that met the needs of our e-learning platform.

10.4.2 Dataset Preparation

In the fine-tuning process for the Marian model in our machine translation (MT) task between Arabic and English, dataset collection and preparation were critical for achieving high-quality translations. This involved combining a custom dataset focused on computer science with the OPUS-100 public dataset to create a robust training corpus.

10.4.2.1 Dataset Collection

- Custom Dataset for Computer Science

Data Sources: We collected data from various sources to build a specialized dataset:

- **Educational Materials:** Included course content from platforms like Coursera and edX, and textbooks on algorithms, data structures, and software engineering.
- **Programming Documentation:** Gathered official documentation for languages (Python, Java) and libraries (TensorFlow, PyTorch).
- **Technical Articles and Blogs:** Sourced research papers and tutorials from academic journals and technical blogs.
- **Programming Tutorials:** Collected step-by-step programming tutorials from online resources.

Methods: We used web scraping for automated data extraction and manual collection for textbooks and selected resources.

- OPUS-100 Dataset :

Overview: OPUS-100 is a multilingual parallel corpus offering high-quality text data from diverse sources.

Data Extraction: We accessed the Arabic-English text pairs from the OPUS-100 dataset through its official website.

Preparation: We focused on extracting aligned Arabic-English text pairs and preprocessing them by tokenizing and normalizing the text.

10.4.2.2 Dataset processing

- **Cleaning and Structuring**

Data Cleaning: We removed irrelevant content and ensured consistency in the datasets. This involved verifying the relevance of the data for computer science and removing noise.

Structuring: We organized the data into parallel text pairs, ensuring proper alignment of Arabic and English sentences. We balanced the dataset to cover general translation needs and domain-specific content.

- **Combining Datasets**

Integration: We merged the OPUS-100 dataset with our custom dataset to create a comprehensive training corpus. This combined general translation data with domain-specific content.

Training Data: The merged dataset was used to fine-tune the Marian model, preparing the data for training by creating training, validation, and test sets. Dataset collection and preparation were essential for fine-tuning the Marian model for our Arabic-English MT task. By leveraging the OPUS-100 public dataset and creating a custom dataset focused on computer science, we ensured that the model could handle both general and specialized translation tasks effectively. This process included careful data collection, cleaning, structuring, and combining efforts, leading to a successful machine translation system for the e-learning platform.

10.4.3 Training Process

Fine-tuning the Marian model involves several crucial steps that adapt a pre-trained model for a specific translation task. This subsection provides a detailed overview of the training process, including model selection, fine-tuning strategies, and evaluation techniques.

10.4.3.1 Loading the Pre-trained Model

The first step in fine-tuning is selecting and loading a pre-trained Marian model. This model, pre-trained on general language data, provides a foundation of language knowledge that can be adapted for specific translation tasks.

- **Model Selection:** A pre-trained Marian model is selected as the starting point for fine-tuning. This model has already been trained on general data, so it contains a broad base of language knowledge that can be tailored for specific translation tasks. For instance, the `Helsinki-NLP/opus-mt-en-ar` model is chosen for translating from English to Arabic, while `Helsinki-NLP/opus-mt-ar-en` is selected for translating from Arabic to English.

10.4.3.2 Fine-Tuning Process

Once the pre-trained model is loaded, the fine-tuning process begins. This involves several key components, including setting hyperparameters, defining optimization strategies, and executing the training loop.

- **Setting Hyperparameters:** Hyperparameters are crucial for controlling the fine-tuning process. Key hyperparameters include:
 - **Learning Rate:** Determines how quickly the model's parameters are updated during training. A smaller learning rate ensures gradual improvements, while a larger rate speeds up learning but may cause instability.
 - **Batch Size:** Specifies the number of samples processed at one time. Larger batch sizes can lead to faster training but require more memory.
 - **Number of Epochs:** Defines how many times the entire dataset is used for training. More epochs allow the model to learn better, but too many can lead to overfitting.
- **Defining Optimization Strategies:** The optimization process involves:
 - **Optimizer Selection:** An optimizer like `AdamW` is chosen to adjust the model's parameters based on the computed loss. `AdamW` is preferred for its efficiency in handling sparse gradients.
 - **Learning Rate Scheduler:** A scheduler is set up to adjust the learning rate during training. It typically starts with a higher rate and decreases it over time to ensure effective learning.
- **Training Loop:** The model undergoes multiple iterations, or epochs, where:
 - The model processes batches of training data.
 - It calculates the loss, which measures the discrepancy between predicted and actual translations.
 - The model's parameters are updated to minimize this loss.

- The process repeats until the model’s performance stabilizes or achieves satisfactory results.

10.4.3.3 Evaluation and Optimization

After the initial training, the model’s performance is evaluated and refined to ensure it meets the desired standards for translation quality.

- **Evaluation Metrics:** The effectiveness of the fine-tuned model is assessed using metrics such as:
 - **BLEU Score:** Measures the quality of translated text by comparing it to reference translations. Higher BLEU scores indicate better performance.
- **Model Refinement:** Based on the evaluation results, several refinement strategies may be employed:
 - **Error Analysis:** Identifies specific issues or patterns in translation errors. This analysis helps refine the model by adjusting hyperparameters, modifying training settings, or adding more data.
 - **Hyperparameter Adjustment:** Adjustments to hyperparameters such as learning rate and batch size are made to optimize the model’s performance based on evaluation feedback.

The iterative nature of this process ensures that the model improves over time, achieving better translation accuracy and reliability.

10.4.4 Understanding BLEU Score in Machine Translation

BLEU (BiLingual Evaluation Understudy) is a widely used metric in natural language processing (NLP) and machine translation (MT) for evaluating the quality of translated text. It compares machine-generated translations to one or more high-quality reference translations and provides a quantitative measure of translation accuracy.

10.4.4.1 What is BLEU Score?

The BLEU score evaluates how similar a machine-generated translation is to reference translations created by humans. It focuses on the overlap of n -grams (sequences of n

consecutive words) between the generated text and reference texts, providing a measure of translation quality.

10.4.4.2 How Does BLEU Score Work?

1. N-gram Precision: Definition: BLEU measures the proportion of n-grams in the generated translation that match n-grams in the reference translations.

N-grams: These are sequences of consecutive words. For example, in a 2-gram (bigram) analysis, “I am” and “am happy” are considered n-grams.

Precision Calculation: BLEU computes precision for different n-gram levels (1-gram, 2-gram, 3-gram, 4-gram). For each n-gram level, the precision is the ratio of the number of n-grams in the generated text that appear in the reference texts to the total number of n-grams in the generated text.

$$\text{Precision} = \frac{\text{Number of matching n-grams}}{\text{Total number of n-grams in generated text}} \quad (10.1)$$

2. Brevity Penalty: Definition: To prevent the model from favoring shorter translations, BLEU includes a brevity penalty.

Penalty Calculation: If the generated text is shorter than the reference text, the brevity penalty penalizes the score to avoid bias towards shorter outputs.

$$\text{Brevity Penalty} = \begin{cases} 1 & \text{if Len of generated txt} \geq \text{Len of target txt} \\ \exp\left(1 - \frac{\text{Length of reference text}}{\text{Length of generated text}}\right) & \text{otherwise} \end{cases} \quad (10.2)$$

3. BLEU Score Formula: Combined Score: BLEU combines n-gram precision scores with the brevity penalty to compute the final score. It uses the geometric mean of the precision scores for different n-gram levels and multiplies this by the brevity penalty.

$$\text{BLEU} = \text{BP} \times \exp\left(\sum_{n=1}^N w_n \log p_n\right) \quad (10.3)$$

Simplified Version:

$$\text{BLEU} = \text{Brevity Penalty} \times \text{Precision}_1^{w_1} \times \text{Precision}_2^{w_2} \times \text{Precision}_3^{w_3} \times \text{Precision}_4^{w_4} \quad (10.4)$$

BLEU Score	Interpretation
< 0.1	Almost useless
0.1 – 0.19	Hard to get the gist
0.2 – 0.29	The gist is clear, but has significant grammatical errors
0.3 – 0.39	Understandable to good translations
0.4 – 0.49	High quality translations
0.5 – 0.59	Very high quality, adequate, and fluent translations
≥ 0.6	Quality often better than humans

TABLE 10.1: BLEU Score Interpretation

10.4.4.3 Why is BLEU Score Important?

- **Correlation with Human Judgment:** BLEU has been shown to correlate well with human judgment of translation quality, providing a reliable metric for evaluating MT systems.
- **Objective Measure:** It offers a quantitative, objective measure of translation quality that can be used for comparing different translation systems.

10.4.4.4 Limitations of BLEU Score

- **N-gram Focus:** BLEU only considers exact n-gram matches, which may not capture all aspects of translation quality, such as fluency or meaning.
- **Brevity Penalty:** The brevity penalty may not always accurately reflect translation quality, especially for very short or very long texts.

The BLEU score is a key metric in machine translation evaluation that measures the overlap of n-grams between generated translations and reference texts. By combining precision scores across different n-gram levels with a brevity penalty, it provides a balanced measure of translation quality. Despite some limitations, BLEU remains a valuable tool for assessing and comparing MT systems.

10.4.5 Training Results for Marian Model (Helsinki-NLP)

This subsection discusses the training results for the Marian model (Helsinki-NLP) for the tasks of translating from Arabic to English and vice versa. The training process involved multiple epochs, and the results were evaluated based on training loss, validation loss, BLEU scores, and general sequence length of generated translations.

10.4.5.1 Arabic to English Translation

The training process for the Arabic to English translation task was conducted over three epochs. The key metrics recorded at each epoch include the training loss, validation loss, and BLEU score. Below are the results:

- **Epoch 1:**

- Training Loss: 1.377000
- Validation Loss: 1.237711
- BLEU Score: 45.172500

- **Epoch 2:**

- Training Loss: 1.366300
- Validation Loss: 1.231334
- BLEU Score: 45.071200

- **Epoch 3:**

- Training Loss: 1.359100
- Validation Loss: 1.230445
- BLEU Score: 45.042000

Observations:

- The training loss decreased steadily across the epochs, indicating that the model was learning and improving its translation capabilities.
- Validation loss also showed a gradual decrease, demonstrating that the model was generalizing well to unseen data.
- BLEU scores remained relatively stable, with a slight decrease over the epochs, suggesting consistent translation quality.

10.4.5.2 English to Arabic Translation

The training process for the English to Arabic translation task was monitored over multiple steps, with key metrics recorded at every 500-step interval. Below are the results:

- **Step 500:**

- Training Loss: 1.943100
- Validation Loss: 1.770475
- BLEU Score: 26.928800
- Generated Length: 13.456000

- **Step 1000:**

- Training Loss: 1.914900
- Validation Loss: 1.769974
- BLEU Score: 27.582500

- **Step 1500:**

- Training Loss: 1.935600
- Validation Loss: 1.768939
- BLEU Score: 26.931400

untill

- **Step 10000:**

- Training Loss: 1.910900
- Validation Loss: 1.765509
- BLEU Score: 26.775100

Observations:

- Training loss fluctuated slightly across steps but showed a general downward trend, indicating model improvement.
- Validation loss remained relatively stable, suggesting consistent performance on unseen data.

- BLEU scores demonstrated minor variations, reflecting stable translation quality over time.

The training results for the Marian model (Helsinki-NLP) indicate successful fine-tuning for both translation tasks. The training and validation losses decreased over time, and the BLEU scores remained stable, demonstrating the model's ability to produce high-quality translations between Arabic and English.

10.5 MCQA Quiz Generation

A. Data Management

- **Data Collection**



- Collecting Transcripts from YouTube Videos
- Specifically focusing on IT fields and programming domains.
- **Text Preprocessing**
 - Text Preprocessing with GPT
 - Reformulating text and eliminating non-essential words.
- **Data Structuring**
 - Extracting Keywords, Distractors, and Questions
 - Utilizing GPT for extracting and structuring key elements into dictionaries.
- **Data Storage and Contribution**
 - Saving and Uploading Data
 - Storing results as CSV files and creating Hugging Face dataset objects for open-source contributions.



HUGGING FACE

- **Dataset Integration**
 - Collecting Existing Datasets
 - Including datasets such as QuAIL, RACE, Cosmos QA, squad-v2, and DROP.
- **Dataset Formatting**
 - Formatting and Consolidating Datasets
 - Limiting context length, filtering data, and unifying datasets into a standard format with columns: ID, CONTEXT, QUESTION, ANSWER, DISTRAC-TORS.
- **Dataset Management**

- Combining and Shuffling Datasets
- Merging multiple datasets and randomizing for uniformity.

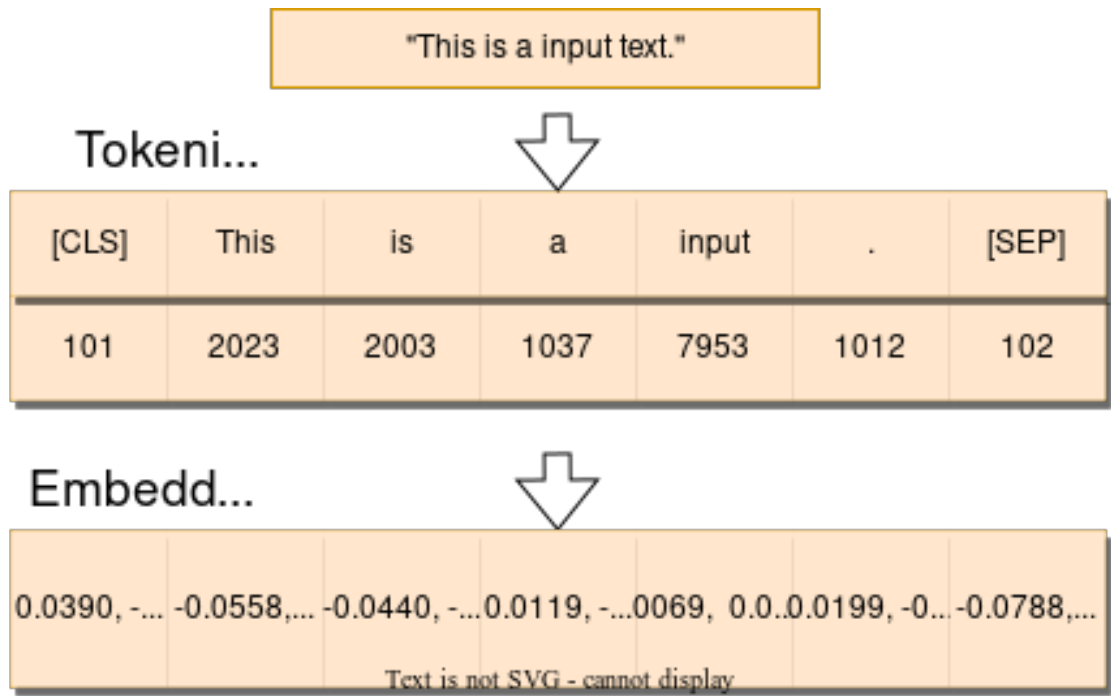
• Dataset Partitioning



- Splitting into Training and Validation Sets
- Segmenting data into training (60k examples) and validation (5k examples) sets.

• Data Preparation for Models

- Tokenization and Data Preparation
- Tokenizing data, padding/truncatin sequences, and converting text to numerical formats (INPUT-IDS, ATTENTION-MASK, LABELS).

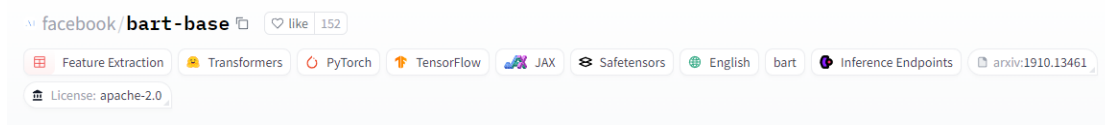


• Model Input Handling

- Model Input Preparation
- Implementing data collectors and setting batch sizes for model training.

B. Model Development

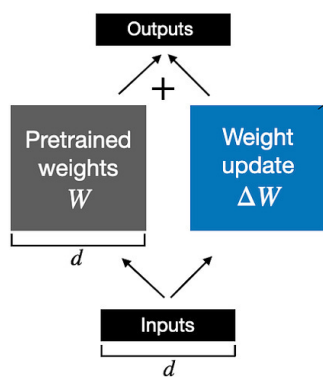
• Model Selection and Evaluation



- Testing and Evaluation
- Assessing various models, addressing issues like performance, context window size, and inference speed.
- Choosing the "facebook/bart-base" Model
- Opting for enhanced capabilities suitable for the project requirements.

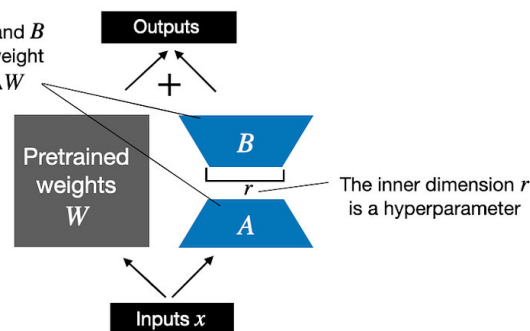
• Model Finetuning

Weight update in regular finetuning



LoRA matrices A and B approximate the weight update matrix ΔW

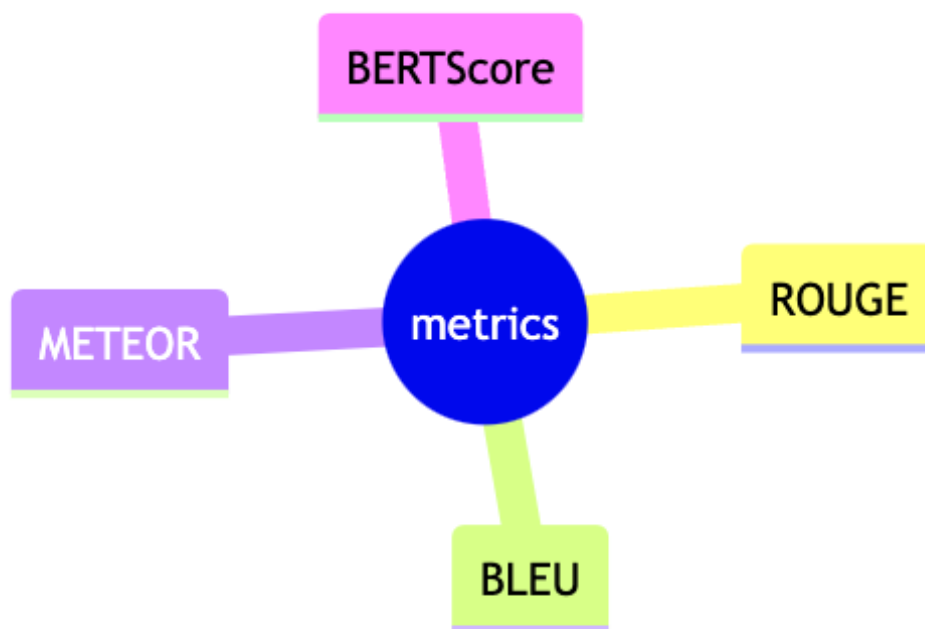
Weight update in LoRA



- Finetuning Strategies
- Exploring full, partial, and new part addition finetuning approaches.
- Adopting PEFT-LORA (Low Rank Adaptation)
- Freezing model parameters, adding new parameter matrices with regularization.

• Model Evaluation Metrics

- Metrics Selection
- Selecting appropriate evaluation metrics (e.g., ROGUE, BLEU, Bert score) based on task-specific needs.
- Achieving Performance Goals
- Demonstrating above-average results on a 5k evaluation dataset using specified evaluation criteria.



C. Inference Optimization

- **Text Normalization**

- Text Normalization with NLTK
- Sequential steps to standardize text, including normalization, case lowering, number substitution, whitespace and punctuation removal, email extraction, word extraction, and stopword removal.

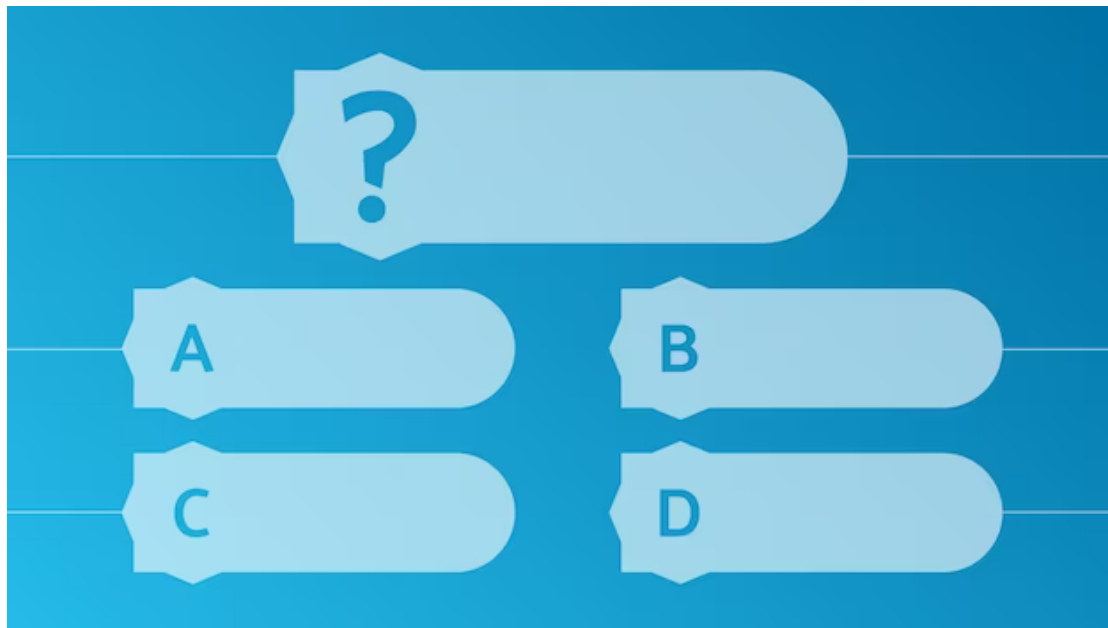
- **Keyword Extraction**

- Keyword Extraction with TopicRank
- Utilizing graph-based methods to extract unigrams, bigrams, and trigrams.
- Conducting similarity checks with PorterStemmer and vector methods.

- **Distractor Generation**

- Distractor Generation with Sense2Vec
- Generating distractors for MCQAs using Sense2Vec and the Reddit dataset.
- Employing similarity checks to differentiate keywords and distractors.

- **Question Generation**



- Question Generation with Finetuned Model
 - Using a finetuned model to generate questions of specific lengths.
 - Adjusting parameters based on normalized context, keywords, and distractors.
- **Deploying the Model**



- We use flask to prepare the backend and setting the routes out of my python code
- Deploy the code using huggingface space with docker configuration , you need to set any tokens or parameters

- Handle postman or javascript for web api not just python or just directly through the space itself
- settling the header and body :
- Set Headers=`key: Content-Type, value:`
- `application/json`
- Set body=`raw: json`
- `"context": " "`

10.6 Text Summarization Feature

Summarization is a crucial task in natural language processing (NLP) that aims to condense large volumes of text into shorter versions while retaining the essential information. In the context of an online learning platform for programming and computer science, summarization becomes especially important. Lecture videos, which often span several hours, contain valuable information that can be overwhelming for students to digest in one go. By summarizing the transcripts of these lecture videos, we can provide students with concise and relevant content, enhancing their learning experience and enabling them to review key concepts quickly.

10.7 Data Collection

The first phase of building the summarization model involved data collection. We used the `youtube_transcript_api` Python library to extract transcripts from approximately 7500 YouTube videos related to programming and computer science. To ensure variety and richness in the summaries, we utilized GPT to generate summaries for these transcripts. Different prompts were employed for different videos to generate diverse summaries, rather than relying on a single prompt for all videos.

To augment our dataset, we combined the collected summaries with a sample from the well-known CNN/DailyMail dataset, which is widely used for summarization tasks. After combining the data, we shuffled it and split it into training and validation sets.

10.8 Model Selection and Architecture

For the summarization task, we chose the `facebook/bart-base` model. BART (Bidirectional and Auto-Regressive Transformers) is a powerful transformer-based

model designed for text generation tasks. It combines the strengths of both bidirectional and auto-regressive models, making it highly effective for summarization.

10.8.1 BART Architecture

BART consists of an encoder-decoder architecture similar to that of sequence-to-sequence models. The encoder processes the input text bidirectionally, meaning it considers the entire context of the input sequence, while the decoder generates the output text auto-regressively, predicting one token at a time.

The key advantages of BART include:

- **Flexibility:** It can handle a variety of text generation tasks, including summarization, translation, and question answering.
- **Performance:** BART has achieved state-of-the-art results on several benchmark datasets.
- **Robustness:** The model is robust to noise in the input data, making it well-suited for real-world applications.

10.9 Fine-Tuning with LoRA and PEFT

To tailor the `facebook/bart-base` model to our specific summarization task, we fine-tuned it using Low-Rank Adaptation (LoRA) and Parameter-Efficient Fine-Tuning (PEFT) techniques.

10.9.1 Low-Rank Adaptation (LoRA)

LoRA is a technique that reduces the number of trainable parameters by approximating the weight matrices with low-rank factorization. This approach helps in speeding up the training process and reducing the computational cost without significantly compromising the model's performance.

10.9.2 Parameter-Efficient Fine-Tuning (PEFT)

PEFT focuses on updating only a subset of the model parameters during fine-tuning.

By selectively training the most relevant parameters, PEFT achieves efficient fine-tuning with fewer resources, making it suitable for large-scale models like BART.

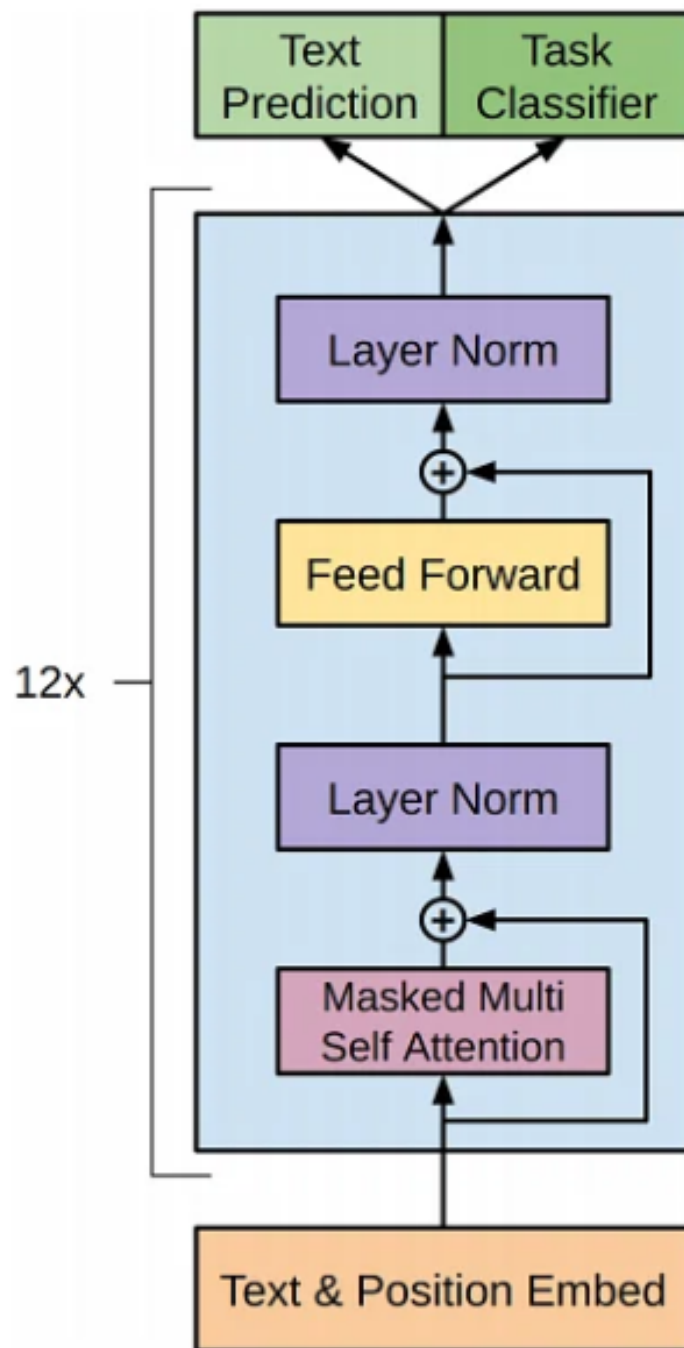


FIGURE 10.4: BART Model Architecture

10.10 Evaluation Metrics

To evaluate the performance of our summarization model, we used ROUGE scores, which are standard metrics for summarization tasks. ROUGE (Recall-Oriented Understudy for Gisting Evaluation) measures the overlap between the generated summary and a reference summary.

10.10.1 ROUGE-1

ROUGE-1 calculates the overlap of unigrams (single words) between the generated summary and the reference summary.

10.10.2 ROUGE-2

ROUGE-2 measures the overlap of bigrams (two consecutive words) between the generated summary and the reference summary.

10.10.3 ROUGE-L

ROUGE-L calculates the longest common subsequence (LCS) between the generated summary and the reference summary. This metric accounts for the order of words and is more sensitive to sentence structure.

10.10.4 ROUGE-LSUM

ROUGE-LSUM is similar to ROUGE-L but considers the summary as a whole rather than individual sentences, making it suitable for evaluating the overall coherence of the summary.

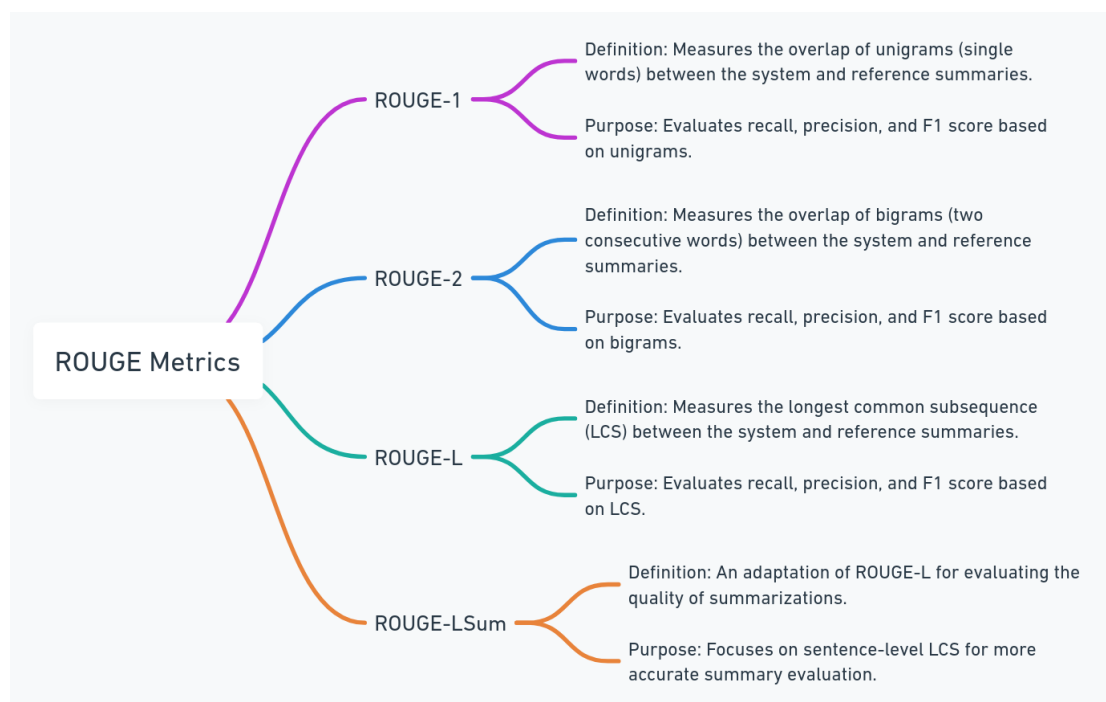


FIGURE 10.5: ROUGE Evaluation Metrics

10.11 Results

After fine-tuning the model, we evaluated it on the validation dataset. The final metrics were as follows:

- **ROUGE-1 F1:** 0.433
- **ROUGE-2 F1:** 0.191
- **ROUGE-L F1:** 0.292
- **ROUGE-LSUM F1:** 0.365

These results indicate that our model is effective in summarizing lecture transcripts, capturing a significant portion of the important information while maintaining coherence and readability.

10.12 Deployment with FastAPI and Docker

The recommendation system was deployed using FastAPI, a modern web framework for building APIs with Python. The API was containerized using Docker and deployed on Hugging Face Spaces, a platform for hosting machine learning models and applications.

10.12.1 FastAPI Implementation

FastAPI was chosen for its ease of use, performance, and automatic documentation generation. The API exposes an endpoint that takes a course name and returns the top 10 similar courses. Here is a brief overview of how FastAPI was implemented:

1. **Setup:** Install FastAPI and Uvicorn.
2. **Define the API:** Create endpoints for the recommendation functionality.
3. **Integrate the Recommendation System:** Call the recommendation function within the API endpoint.
4. **Run the API:** Use Uvicorn to run the FastAPI application.

10.12.2 Containerization with Docker

Docker was used to create a containerized environment for the API, ensuring consistency across different deployment platforms. The Docker setup involves:

1. **Dockerfile:** Create a Dockerfile to define the image.
2. **Build the Image:** Use Docker to build the image from the Dockerfile.
3. **Run the Container:** Start the container from the built image.

10.12.3 Hosting on Hugging Face Spaces

The containerized API was deployed on Hugging Face Spaces, providing a scalable and accessible interface for the recommendation system. The steps to deploy on Hugging Face Spaces include:

1. **Create a Space:** Set up a new Space on Hugging Face.
2. **Configure the Space:** Set the Space to use Docker.
3. **Upload Docker Image:** Push the Docker image to the Hugging Face Space.
4. **Run the Space:** Start the Space, making the API accessible online.

10.13 Conclusion

In this chapter, we have detailed the process of building a text summarization model for lecture transcripts in an online learning platform. Starting with data collection from YouTube videos and the CNN/DailyMail dataset, we fine-tuned the `facebook/bart-base` model using LoRA and PEFT techniques. The model's performance, evaluated using ROUGE metrics, demonstrates its capability to generate concise and informative summaries, aiding students in their learning journey.

Chapter 11

Website Services

11.1 Security: Authentication & Authorization and Logout

MoltaqaTech ensures secure access and usage through robust authentication and authorization mechanisms. The website uses JSON Web Tokens (JWT) for managing user sessions. Upon successful login, a token is generated and sent to the client. This token is required for accessing protected resources on the website.

Sign Up

Already have an account? [Log in](#)

First Name	Last Name
First Name	Last Name
Username	Email
username	Your Email
Password	Re-Enter Password
Password	Re-Enter Password

☐ Accept the Terms and Privacy Policy

Register

FIGURE 11.1: signup

Login

Don't have an account yet? [Sign up for free](#)

Username Or Email

Password

☐ Remember me [Forgot your password?](#)

Log In

or Log-in with

 Gacebook	 Google
---	---

FIGURE 11.2: login

11.1.1 Authentication

Users can sign up and log in using their Facebook or Gmail accounts. During the signup process, an email verification is sent to the provided email address. Users must verify their email to activate their accounts. This verification step enhances the security by ensuring that the email addresses are valid and belong to the users. The authentication process involves hashing passwords using secure algorithms before storing them in the database, ensuring that even in the event of a data breach, user passwords remain protected.

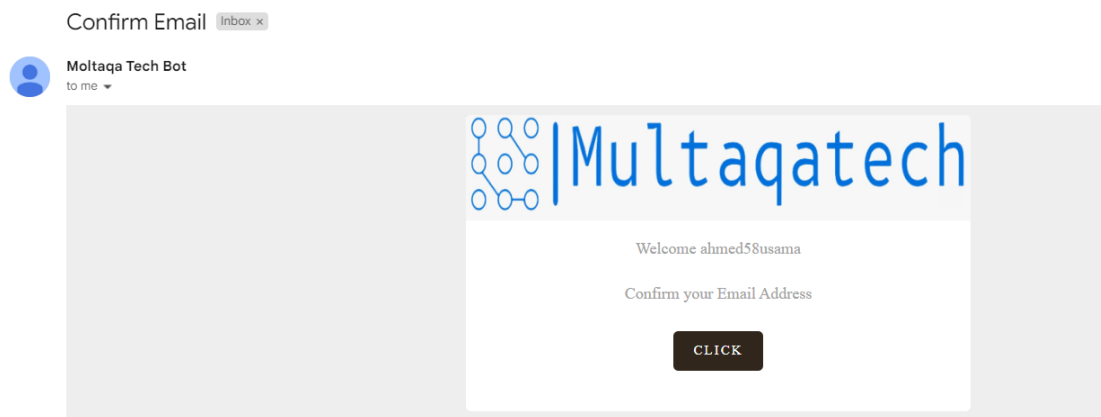


FIGURE 11.3: Confirmation Mail

11.1.2 Authorization

Authorization is handled through roles and claims. The website supports different roles such as student and instructor. Each role has specific permissions and access rights. For instance, instructors can create and manage courses, while students can enroll in and access courses they have purchased. Claims are used to fine-tune access control, allowing for more granular permissions within the roles.

11.1.3 Logout

When a user logs out, the JWT token is added to a Redis database blacklist, rendering it invalid for future use. This ensures that even if a token is stolen, it cannot be used after the user has logged out. Tokens expire after 24 hours, ensuring that old tokens cannot be reused. Tokens in the blacklist are automatically cleared after 24 hours as they are no longer useful. This approach provides a balance between security and performance, ensuring that the system remains responsive while maintaining high security standards.

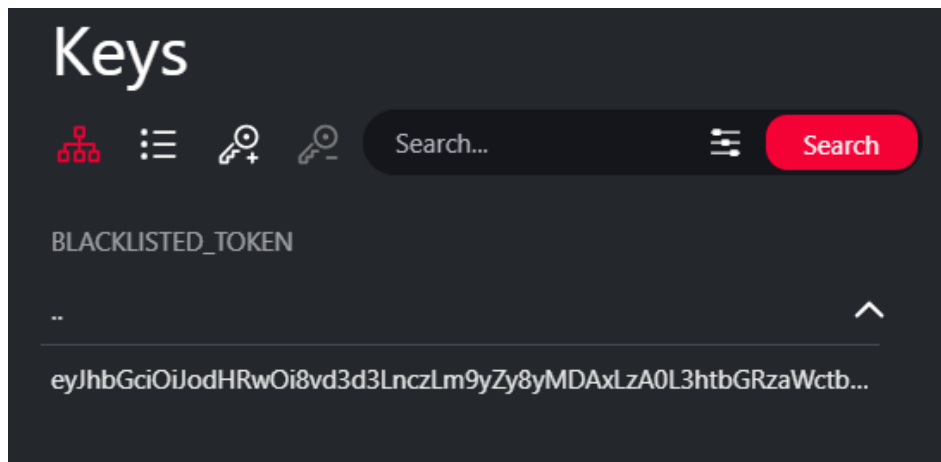


FIGURE 11.4: cachedBlacklistedTokens

```

3 2 references
4 public class JwtBlacklistMiddleware
5 {
6     private readonly RequestDelegate _next;
7     private readonly IConnectionMultiplexer _redis;
8
9     0 references
10    public JwtBlacklistMiddleware(RequestDelegate next, IConnectionMultiplexer redis)
11    {
12        _next = next;
13        _redis = redis;
14    }
15
16    0 references
17    public async Task Invoke(HttpContext context, IUnitOfWork unitOfWork)
18    {
19        // Check if the request contains authorization header
20        var token = context.Request.Headers["Authorization"].FirstOrDefault()?.Split(" ").Last();
21
22        if (token != null)
23        {
24            var redis = _redis.GetDatabase();
25            var key = $"blacklisted_token:{token}";
26
27            var isBlacklisted = await redis.KeyExistsAsync(key);
28
29            if (isBlacklisted) // Use either blacklistedToken.HasValue or isBlacklisted
30            {
31                context.Response.StatusCode = StatusCodes.Status401Unauthorized;
32                context.Response.ContentType = "application/json"; // Set content type
33
34                var errorResponse = new { message = "The provided token is invalid or blacklisted." };
35                await context.Response.WriteAsync(JsonConvert.SerializeObject(errorResponse));
36                return;
37            }
38
39            await _next(context);
40        }
41    }
42 }

```

FIGURE 11.5: JWT Blacklist Middleware

11.2 Courses Module

The core of MoltaqaTech is its Courses module. This section outlines the various features and functionalities associated with courses.

11.2.1 Course Creation

Users with instructor roles can create courses. Each course can include:

- **Learning Objectives:** Clearly defined goals that students are expected to achieve by the end of the course. These objectives help guide the course structure and provide students with a clear understanding of what they will learn.
- **Descriptions:** Detailed information about the course content, including what topics will be covered, prerequisites, and any additional resources that will be provided.
- **Costs and Discounts:** Pricing information, including any available discounts. Instructors can offer promotional discounts or bundle multiple courses together at a reduced price.

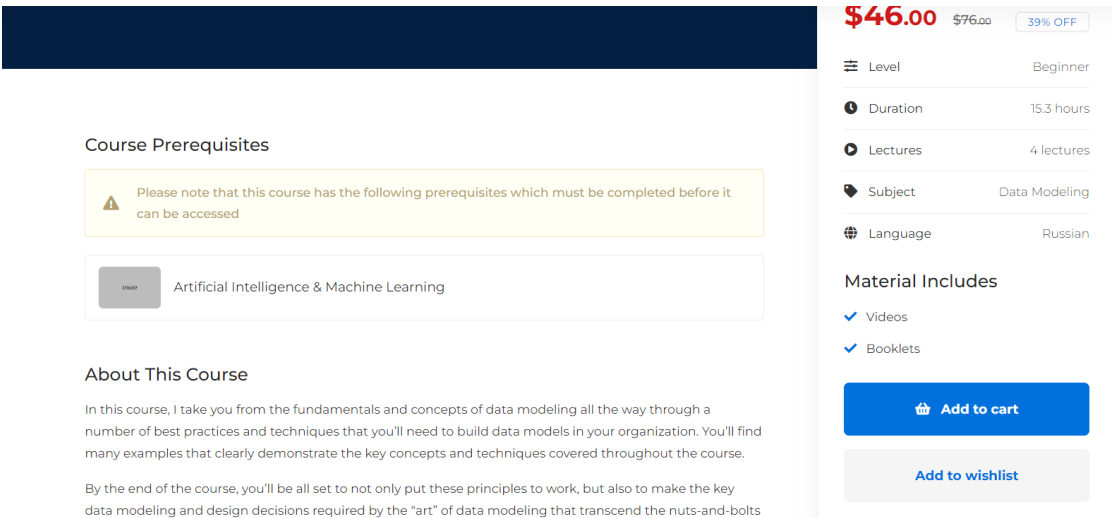


FIGURE 11.6: Course Layout

11.2.2 Course Structure

Courses are divided into sections, and each section comprises items that can either be lectures or quizzes.

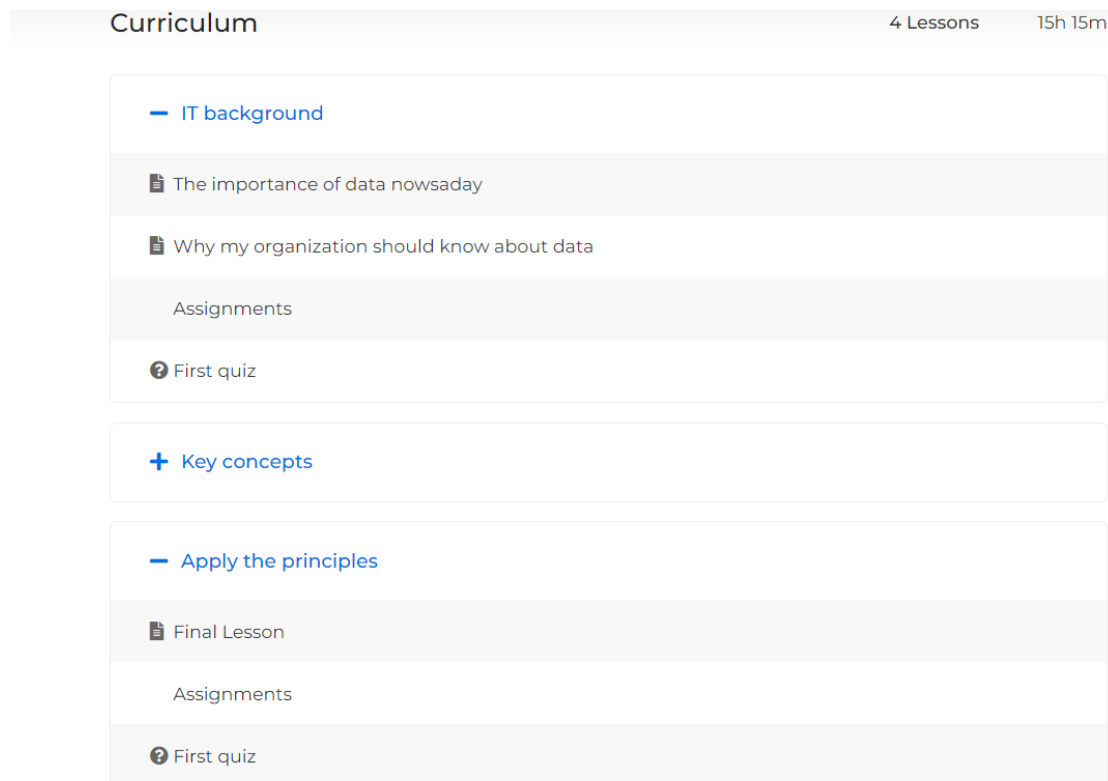


FIGURE 11.7: Curriculum

- **Lectures:** Instructional videos, reading materials, or interactive content. Lectures can include multimedia elements such as videos, audio clips, and downloadable resources to enhance the learning experience.
- **Quizzes:** Multiple-choice questions (MCQs) that students must pass to proceed. Quizzes are designed to test the student's understanding of the material and provide immediate feedback. Students cannot proceed to the next section until they pass the quiz, ensuring mastery of the content.

11.2.3 Tracking Progress

The system tracks the progress of students in each course they are enrolled in. Students can see their progress as they complete sections and quizzes. Instructors can also view the progress of their students, enabling them to provide additional support and guidance where needed. Progress tracking includes detailed analytics such as time spent on each section, quiz scores, and completion rates.

11.2.4 Reviews

Students can post reviews on courses they have purchased. Reviews help other students gauge the quality and relevance of the course content. Instructors can respond to reviews, providing additional context or addressing any concerns raised by students. Reviews are prominently displayed on the course page, contributing to the course's overall rating and visibility.

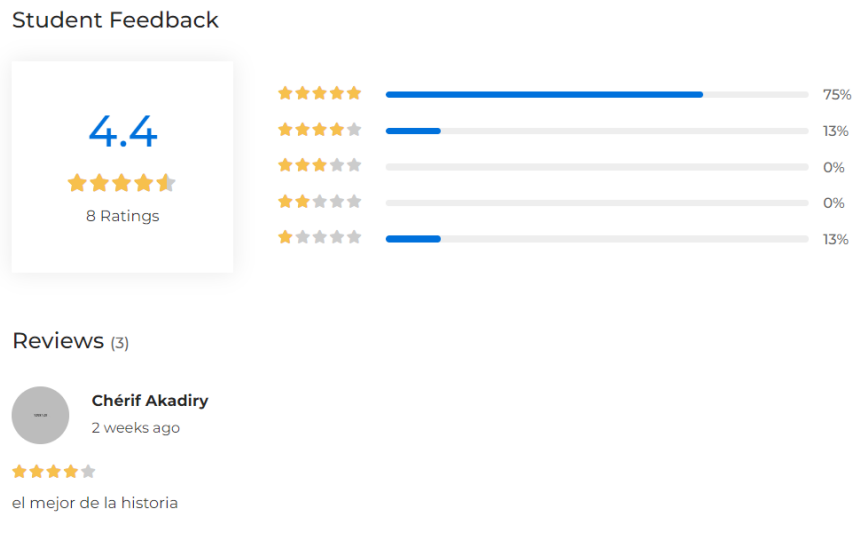


FIGURE 11.8: Courses Reviews

11.3 BlogPosts Module

The BlogPosts module allows users to share and discuss tech-related articles.

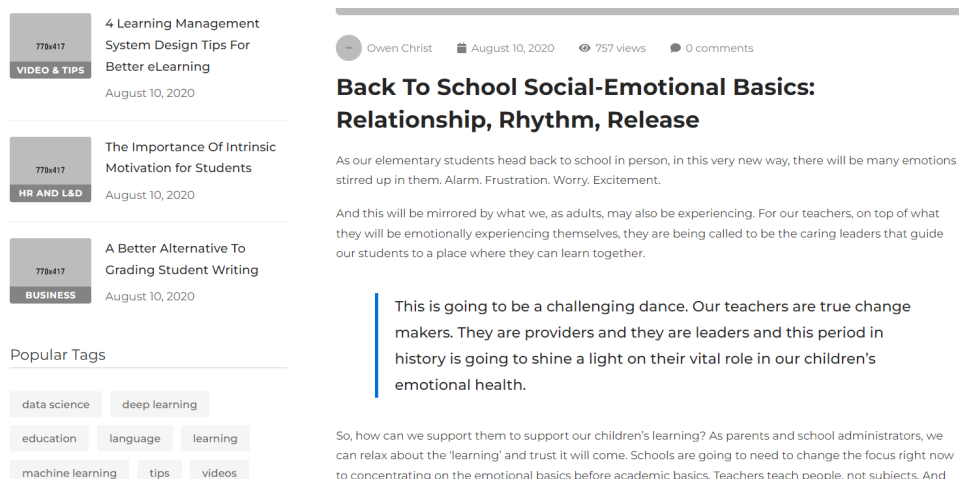


FIGURE 11.9: Blog Post

11.3.1 Posting and Categories

Users can post blog articles on various tech topics. Each post can be categorized and tagged with relevant subjects, making it easier for other users to filter and find content of interest. Categories and tags help organize the content and improve searchability.

Users can also add images, videos, and code snippets to their posts, making the content more engaging and informative.

11.3.2 Comments and Views

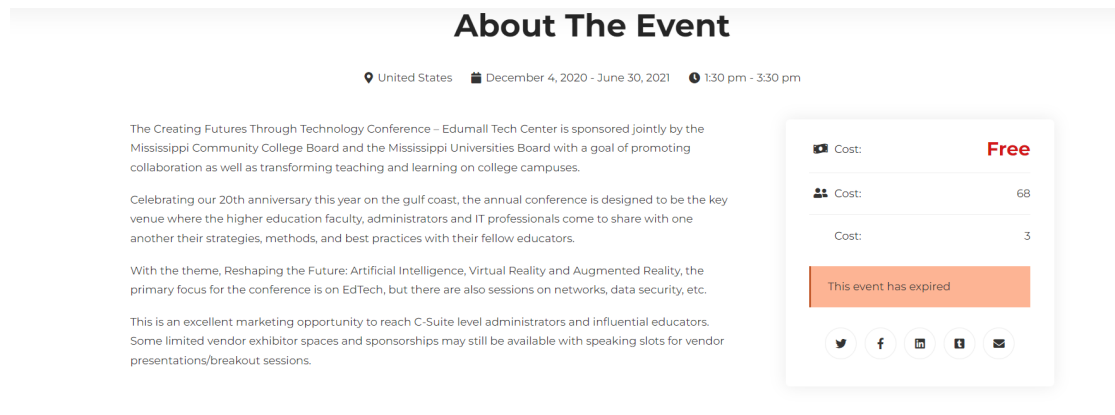
Users can comment on blog posts, fostering discussion and knowledge sharing. The system tracks the number of views each post receives and the posting date, allowing posts to be sorted and filtered accordingly. Comments can be upvoted or downvoted, helping highlight the most valuable contributions. The commenting system supports threaded discussions, enabling users to reply directly to specific comments.

11.3.3 Sharable Links

Each blog post has a sharable link, enabling users to share interesting articles with others outside the platform. Social media integration allows users to easily share posts on platforms such as Facebook, Twitter, and LinkedIn, increasing the reach and impact of the content.

11.4 Events Module

The Events module allows users to announce and participate in tech-related events.



Location

FIGURE 11.10: Event

11.4.1 Event Creation

Users can create events, specifying details such as:

- **Speakers:** Information about the event speakers, including their bios and links to their professional profiles.
- **Location:** Physical or virtual location details. For virtual events, users can specify the platform (e.g., Zoom, Microsoft Teams) and provide the necessary links.
- **Description:** What the event is about, including the agenda, target audience, and any prerequisites for attendance.

11.4.2 Participation

Other users can view and join events, fostering a sense of community and providing opportunities for networking and learning. Users can RSVP to events, add them to their personal calendars, and receive reminders as the event date approaches. Event organizers can manage attendance, send updates to participants, and gather feedback after the event.

11.5 Zoom Meeting Module

MoltaqaTech integrates with the Zoom API to facilitate live meetings.

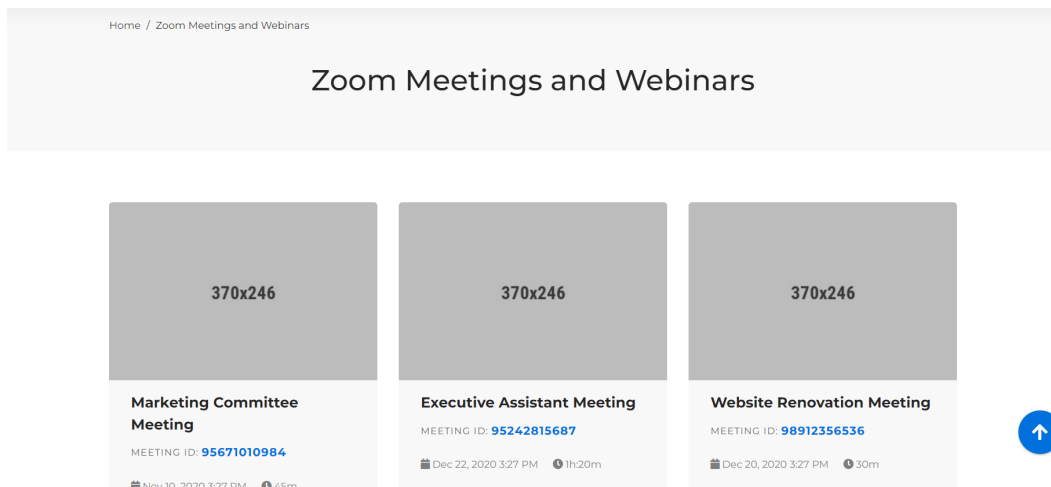


FIGURE 11.11: Zoom Meetings

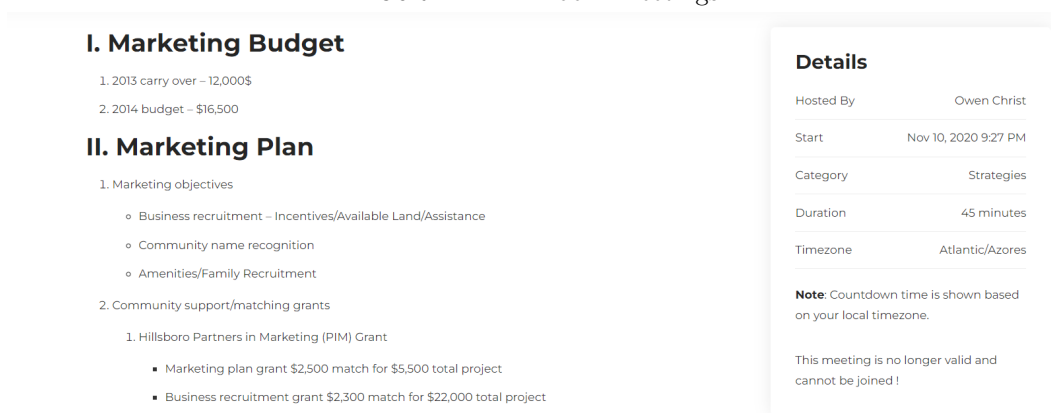


FIGURE 11.12: Zoom Meeting Single

11.5.1 Creating and Joining Meetings

Users can create Zoom meetings directly from the website. The integration allows users to schedule meetings, send invitations, and manage attendees without leaving the platform. Other users can join these meetings without leaving the platform, providing a seamless experience for live interactions and learning sessions. The system supports recording meetings, enabling users to review the sessions later or share them with those who couldn't attend.

11.6 Dashboard

The Dashboard provides a comprehensive overview for both students and instructors.

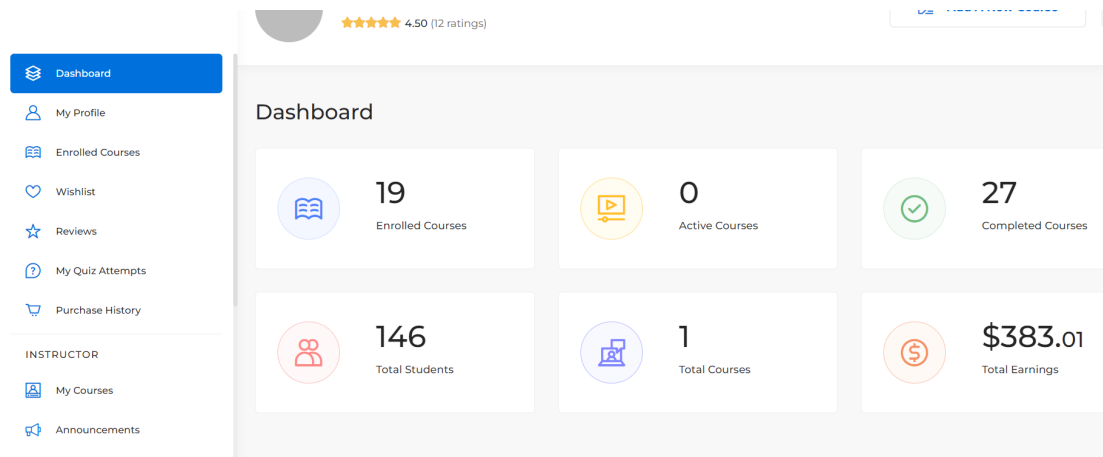


FIGURE 11.13: Dashboard

11.6.1 Student Dashboard

Students can track their progress, view courses they are enrolled in, manage their wishlist, and see their transaction history. The dashboard provides a personalized learning experience, showing recommendations based on the student's interests and progress. Students can also access their certificates of completion for courses they have finished.

11.6.2 Instructor Dashboard

Instructors have access to tools for managing their courses, including viewing student progress and performance, and managing course content. The dashboard provides insights into student engagement, quiz results, and overall course performance. Instructors can also see feedback from students and respond to queries directly through the platform.

11.7 Shop Module

The Shop module handles the purchasing and management of courses.

The screenshot displays a shopping cart with two items: 'Business Administration' priced at \$26.00 with a quantity of 1, and 'The Five Invitations' priced at \$42.00 with a quantity of 2. Each item has a 'Remove' link. Below the cart are buttons for 'Continue shopping', 'X Clear shopping cart', and 'Update cart'. The checkout section includes a 'Coupon Discount' field with an 'Apply coupon' button, a 'Calculate shipping' section with radio buttons for 'Free shipping' (selected) and 'Flat rate: \$5.00', and a shipping address field. A summary box on the right shows a subtotal of \$131.00, shipping of \$5.00, and a total of \$136.00, with a 'Checkout' button. A blue circular button with an upward arrow is also visible.

Item	Price	Quantity	Total Price	Action
Business Administration	\$26.00	1	\$26.00	Remove
The Five Invitations	\$42.00	2	\$84.00	Remove

Subtotal: \$131.00
Shipping: Free shipping \$5.00
Total: \$136.00

FIGURE 11.14: Shopping

11.7.1 Cart and Checkout

Students can add courses to their cart, proceed to checkout, and make payments using Stripe. The checkout process is secure and supports various payment methods, including credit cards and digital wallets. The module ensures a smooth and secure transaction process, with real-time payment verification and order confirmation.

11.7.2 Order History

Students can view their order history and access receipts for their purchases. The order history provides detailed information about each transaction, including the course details, purchase date, and payment status. Students can also request refunds or contact customer support directly from their order history page.

Chapter 12

Angular Framework

12.1 Overview

Angular is a comprehensive platform and framework developed and maintained by Google for building modern, single-page web applications. It combines declarative templates, dependency injection, end-to-end tooling, and integrated best practices to solve development challenges. Angular's key strengths lie in its robust architecture, extensive feature set, and strong community support, making it a popular choice for frontend development.

Angular is particularly known for its use of TypeScript, a statically-typed superset of JavaScript, which enhances code quality, maintainability, and scalability. The framework is designed to facilitate the development of large-scale applications by promoting a structured and organized approach to coding.

12.2 History and Evolution

Angular has undergone significant evolution since its inception:

- **AngularJS (2010):** Introduced by Google, AngularJS pioneered the use of the Model-View-Controller (MVC) pattern in frontend development, enhancing the structure and maintainability of web applications.
- **Angular 2+ (2016 onwards):** Angular 2 marked a complete rewrite, introducing TypeScript as its primary language, a component-based architecture, and a more modular approach. Subsequent versions, such as Angular 16, have continued

to refine and enhance these core principles, focusing on performance, scalability, and developer experience.

12.3 Key Features

Angular offers a multitude of features that make it a popular choice for developers:

- **Component-based Architecture:** Allows for reusable, modular, and maintainable code.
- **Dependency Injection:** Facilitates better organization and testing by injecting dependencies rather than hardcoding them.
- **Data Binding:** Provides two-way data binding, which keeps the model and the view in sync automatically.
- **Directives:** Extend HTML's capabilities by creating custom HTML tags and attributes.
- **Services:** Promote code reusability by sharing data and logic across different components.
- **Routing:** Enables the creation of single-page applications with multiple views and navigation.
- **TypeScript:** Enhances JavaScript with strong typing, making code more robust and easier to debug.
- **Forms:** Angular provides robust form handling capabilities through reactive forms (`FormGroup`, `FormControl`, etc.) and template-driven forms, facilitating complex form validations and user interactions.
- **Pipes:** Transform data before displaying it in the UI, such as formatting dates or currencies, enhancing the user experience.

12.4 Advantages of Using Angular 16

For your graduation project, Angular 16 provided several advantages:

- **Productivity:** Angular CLI (Command Line Interface) simplifies project setup, development, and deployment tasks, improving developer productivity.

- **Performance:** Angular's Ahead-of-Time (AOT) compilation and optimized change detection mechanism deliver faster rendering and improved application performance.
- **Scalability:** Angular's modular architecture and dependency injection system support the development of scalable applications, accommodating future growth and complexity.
- **Community and Ecosystem:** Angular benefits from a vibrant community and a rich ecosystem of libraries, tools, and resources, providing extensive support and solutions for developers.

12.5 E-Learning Project with Angular 16: A Case Study

Your e-learning website project leveraged Angular 16's capabilities effectively:

- **Data Binding:** Utilized two-way data binding to ensure real-time updates between the course content and the user interface, enhancing user interaction and experience.
- **Routing and Lazy Loading:** Implemented Angular's powerful routing system to navigate between different course modules and lessons seamlessly. Lazy loading was employed to load modules dynamically, optimizing initial page load times.
- **Services and Dependency Injection:** Services encapsulated business logic and data retrieval operations, promoting code reusability and maintainability. Dependency injection facilitated the seamless integration of services across components.
- **Forms and Validation:** Managed complex forms using Angular's reactive forms (`FormGroup`, `FormControl`, `Validators`), ensuring data integrity and providing a user-friendly form submission experience.
- **Pipes:** Utilized Angular pipes to transform data before displaying it in the UI, such as formatting dates or currencies, enhancing the user experience.

12.6 Integration with .NET Backend

In your e-learning project, Angular was chosen for the frontend while .NET was used for the backend. Here's why this combination was beneficial:

- **Full-stack Development:** Angular's frontend capabilities complement .NET's backend strengths, providing a full-stack solution for developing and deploying web applications.
- **Scalability and Performance:** .NET's performance and scalability features, combined with Angular's efficient rendering and data handling, ensure a smooth and responsive user experience, even with complex business logic and large datasets.
- **Integration Flexibility:** Angular's ability to consume RESTful APIs provided by .NET seamlessly, leveraging HTTP services for data retrieval and manipulation, simplifying communication between the frontend and backend layers.
- **Developer Expertise:** Leveraging Angular and .NET allowed you to capitalize on existing expertise within your team, ensuring efficient development and maintenance of the entire application stack.

12.7 Comparison with Other Frameworks

Choosing Angular over other frameworks, such as React or Vue.js, for your e-learning project was influenced by several factors:

- **Comprehensive Solution:** Angular provides an all-inclusive solution with built-in features like routing, forms, HTTP client, and state management, reducing the need for integrating multiple libraries or frameworks.
- **TypeScript Integration:** Angular's native support for TypeScript ensures type safety, improved code quality, and better maintainability compared to frameworks that rely solely on JavaScript or JSX.
- **Full-Featured CLI:** Angular CLI simplifies project setup, configuration, and deployment tasks, offering a standardized and efficient development workflow.
- **Large Community and Ecosystem:** Angular boasts a large and active community, extensive documentation, and a rich ecosystem of third-party libraries, tools, and components, providing robust support and solutions for developers.

12.8 Real-world Applications

Angular is widely adopted across various industries for building scalable and performant applications:

- **Enterprise Solutions:** Used by companies like Google, Microsoft, and Forbes for internal tools, enterprise applications, and content management systems.
- **E-commerce:** Powers interactive and responsive e-commerce platforms, enhancing user engagement and conversion rates.
- **Financial Services:** Provides secure and efficient solutions for banking and financial institutions to manage transactions and customer data.
- **Healthcare:** Supports healthcare providers with robust applications for patient management, scheduling, and medical records.

12.9 Conclusion

Angular 16 offers a robust framework for developing modern web applications, combining powerful features, scalability, and community support. Your experience with Angular 16 in developing the e-learning website underscores its effectiveness in building dynamic, responsive, and scalable applications. In the following chapters, we will explore in-depth the specific components, techniques, and challenges encountered during the development of your e-learning platform with Angular 16.

Chapter 13

Front-End Project Architecture

13.1 Overview

The architecture of our e-learning platform is meticulously designed to ensure scalability, maintainability, and ease of development. The project structure is divided into multiple modules and components, each serving a specific purpose, and follows the principles of modular design and separation of concerns. This chapter provides a detailed description of the architecture, explaining the roles of different modules, components, and services.

13.2 App Folder Structure

The root folder of the Angular application, commonly referred to as the "app folder," contains the main structural elements of the project. Here is a breakdown of its contents:

- **App HTML (`app.component.html`):** This file serves as the main HTML template for the root component of the application. It includes the base layout and acts as the entry point for other components.
- **App TypeScript (`app.component.ts`):** This file is the main TypeScript file for the root component, defining its logic and behavior.
- **App SCSS (`app.component.scss`):** This file contains the main styles for the root component, providing the base styling for the application.

- **App Routing (`app-routing.module.ts`):** This file contains the routing configuration for the main modules of the application. It defines the primary routes and their corresponding modules, facilitating navigation within the app.
- **Layout Folder:** The layout folder contains components that manage the overall layout of the application.
- **Modules Folder:** The modules folder is the backbone of the project, containing feature-specific modules that encapsulate related components, services, and other resources.

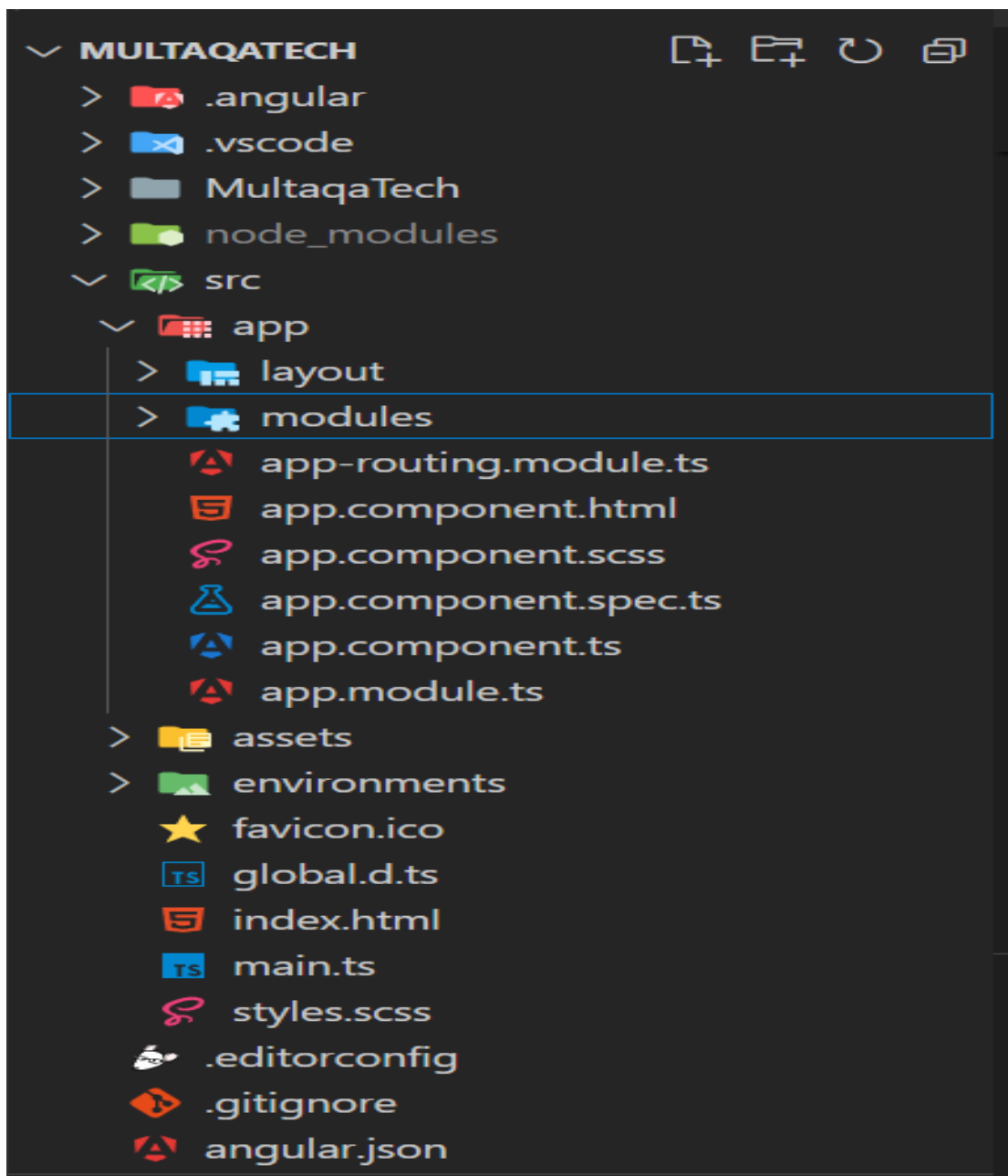


FIGURE 13.1: App Folder Structure

13.3 Layout Folder

The layout folder contains components that manage the overall layout of the application. Specifically:

- **Layout with Navbar:** This layout includes a navigation bar, which is used in pages where user navigation is required.
- **Layout without Navbar:** This layout excludes the navigation bar, providing a cleaner interface for pages where the navbar is unnecessary.

These layouts are used selectively across different pages to provide a consistent user experience while catering to the specific needs of each page.

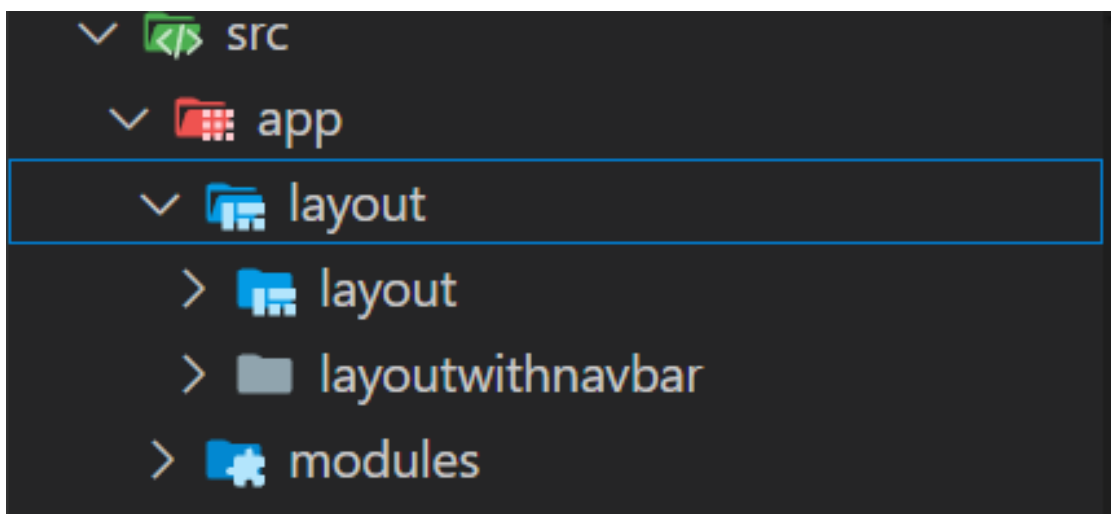


FIGURE 13.2: Layout Folder

13.4 Modules Folder

The modules folder is a pivotal part of the project structure, housing various submodules, each responsible for a distinct feature or section of the application. The key submodules include:

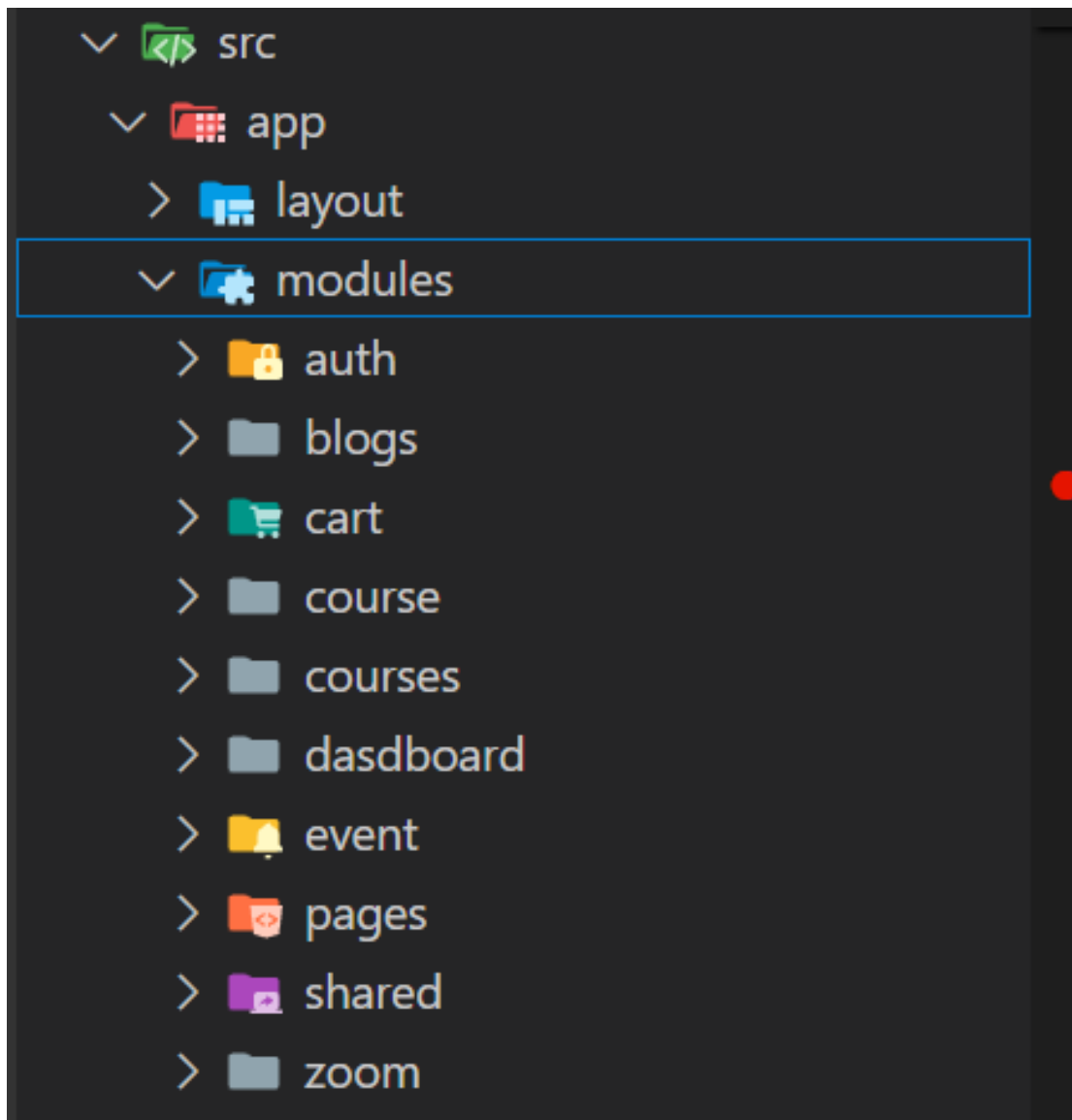


FIGURE 13.3: Modules Folder Architecture

- **Auth Module:** Manages authentication-related functionalities. It contains several components and services, detailed as follows:
 - **Components:**
 - * **confirmEmail Component:** Displays a message indicating whether the user's email has been confirmed.
 - * **forgotPassword Component:** Handles the process of changing the user's password.
 - * **welcome Component:** Serves as a welcoming page for guests, offering options to log in or sign up.

- **Auth Guard:** Protects routes from being accessed by users who are not logged in, ensuring that only authenticated users can access certain parts of the application.
- **Auth Service:** Handles all authentication-related API calls to the backend, including:
 - * **register API:** Registers a new user.
 - * **login API:** Authenticates a user.
 - * **forgotPassword API:** Initiates the password recovery process.
 - * **resetPassword API:** Resets the user's password.
 - * **getCurrentUser API:** Retrieves the currently logged-in user's information.
 - * **addInstructor API:** Adds a new instructor to the system.
 - * **Facebook API:** Facilitates Facebook login.
 - * **Google API:** Facilitates Google login.
 - * **logout API:** Logs out the current user.

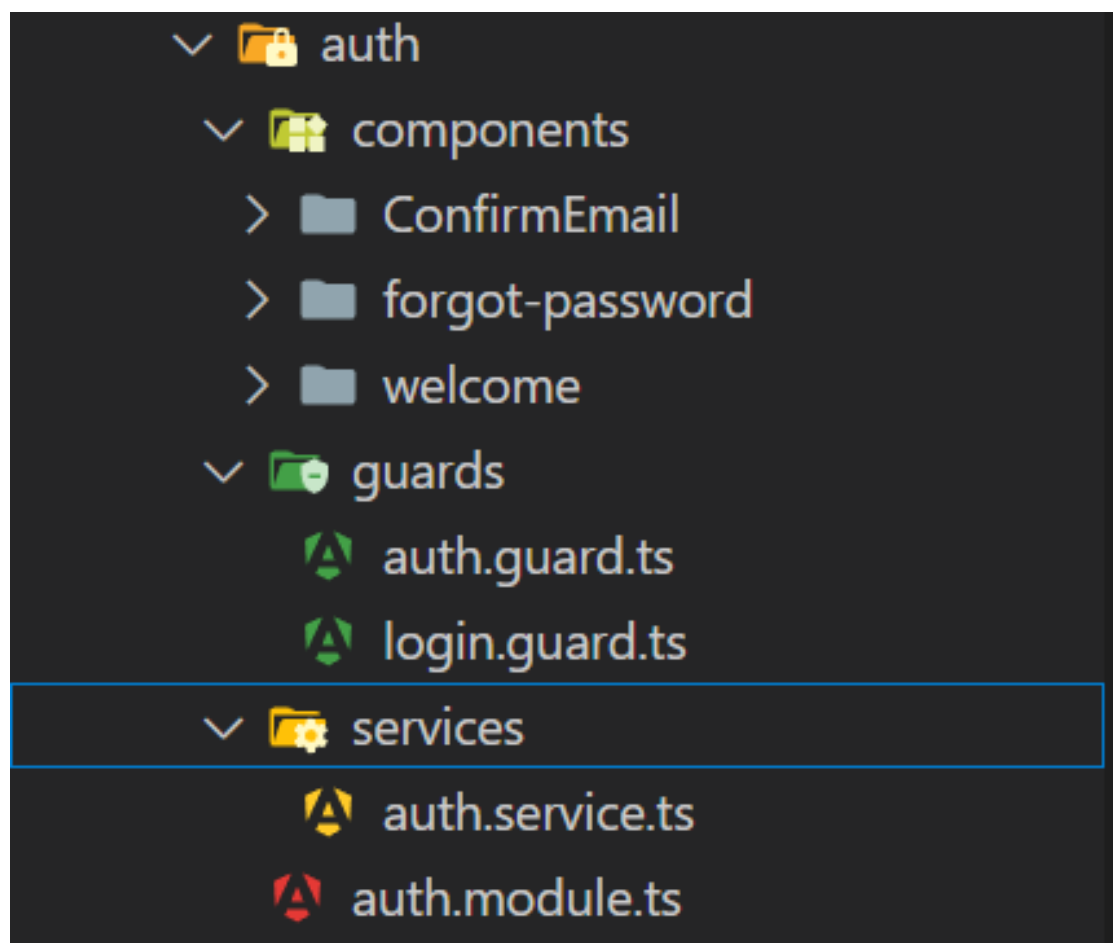


FIGURE 13.4: Auth Module Architecture

- **Blogs Module:** Manages blog-related functionalities, including:
 - **Components:**
 - * **blogs Component:** Displays all blogs and allows any user to add a blog.
 - * **blog-details Component:** Displays a single blog where users can read and add comments, and the owner can update or delete the blog.
 - **Interfaces:**
 - * **blog-post Interface:** Represents the structure of a blog post.
 - **Services:** Handles various blog-related API interactions, including:
 - * `getCurrentPage` API
 - * `blogsSearch` API
 - * `getCategories` API
 - * `getSubjects` API
 - * `filterByCategoryId` API
 - * `getBlogDetails` API
 - * `addComment` API
 - * `deleteComment` API
 - * `updateComment` API
 - * `shareBlog` API
 - * `deleteBlog` API
 - * `postBlog` API
 - * `editBlog` API

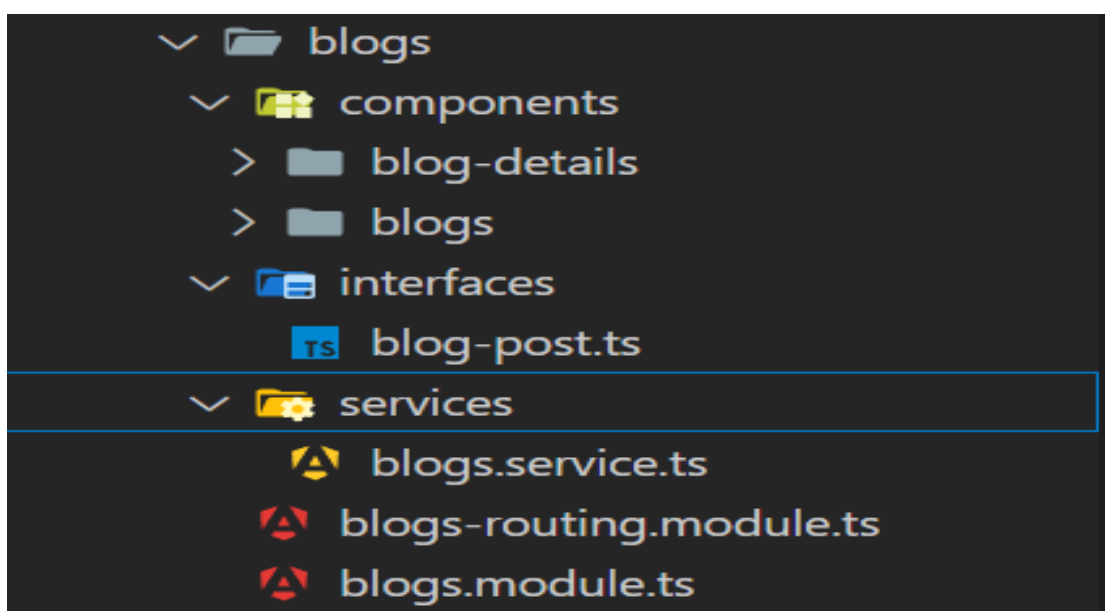


FIGURE 13.5: Blog Module Architecture

- **Cart Module:** Manages the shopping cart functionalities, including:
 - **Components:**
 - * **cart Component:** Displays the courses the user wants to purchase and handles purchasing them.
 - * **failed Component:** Handles errors that occur during the purchasing process.
 - **Services:** Handles cart-related API interactions, including:
 - * checkout API

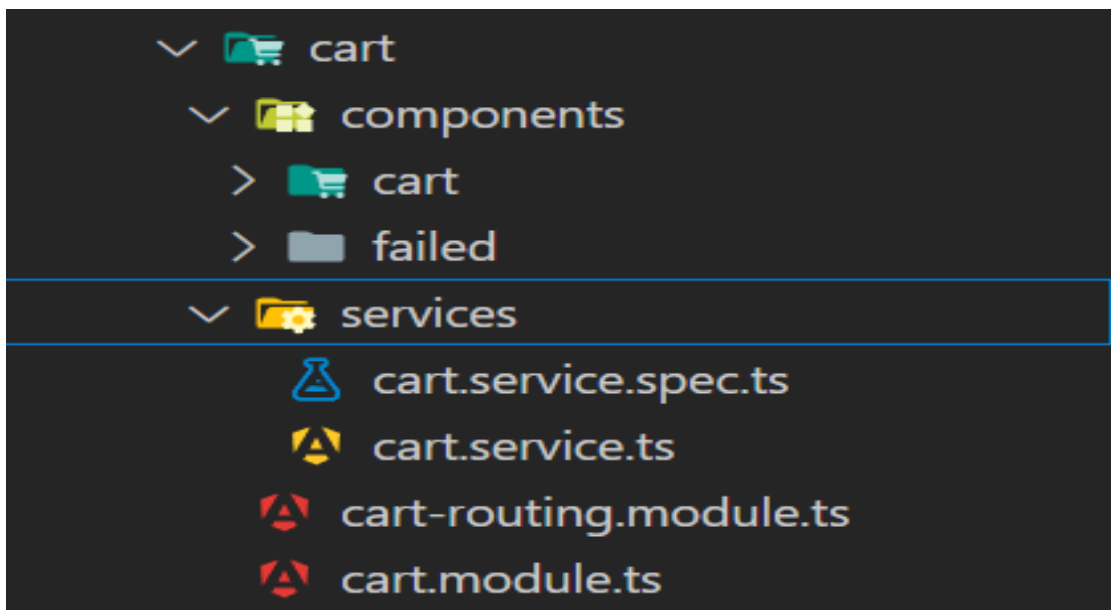


FIGURE 13.6: Cart Module Architecture

- **Course Module:** Manages individual course functionalities, including:
 - **Components:**
 - * **Main Component:** Displays the main page for the content of the course after being purchased.
 - * **Children Components:**
 - **Notes:** Displays notes for each lecture.
 - **Question-Details:** Displays details for each question.
 - **Questions:** Displays questions for each course.
 - **Quiz:** Displays quizzes for the course.
 - **Services:** Handles course-related API interactions, including:
 - * `getCourseDetails` API
 - * `getCourseSectionsDetails` API

- * `getCourseLectureDetails` API
- * `getAllNotes` API
- * `addNotes` API
- * `deleteNote` API
- * `updateNote` API
- * `getAllQuestions` API
- * `addQuestions` API
- * `deleteQuestion` API
- * `updateQuestion` API
- * `getQuestionDetails` API
- * `addAnswer` API
- * `getAnswers` API
- * `updateAnswer` API
- * `deleteAnswer` API

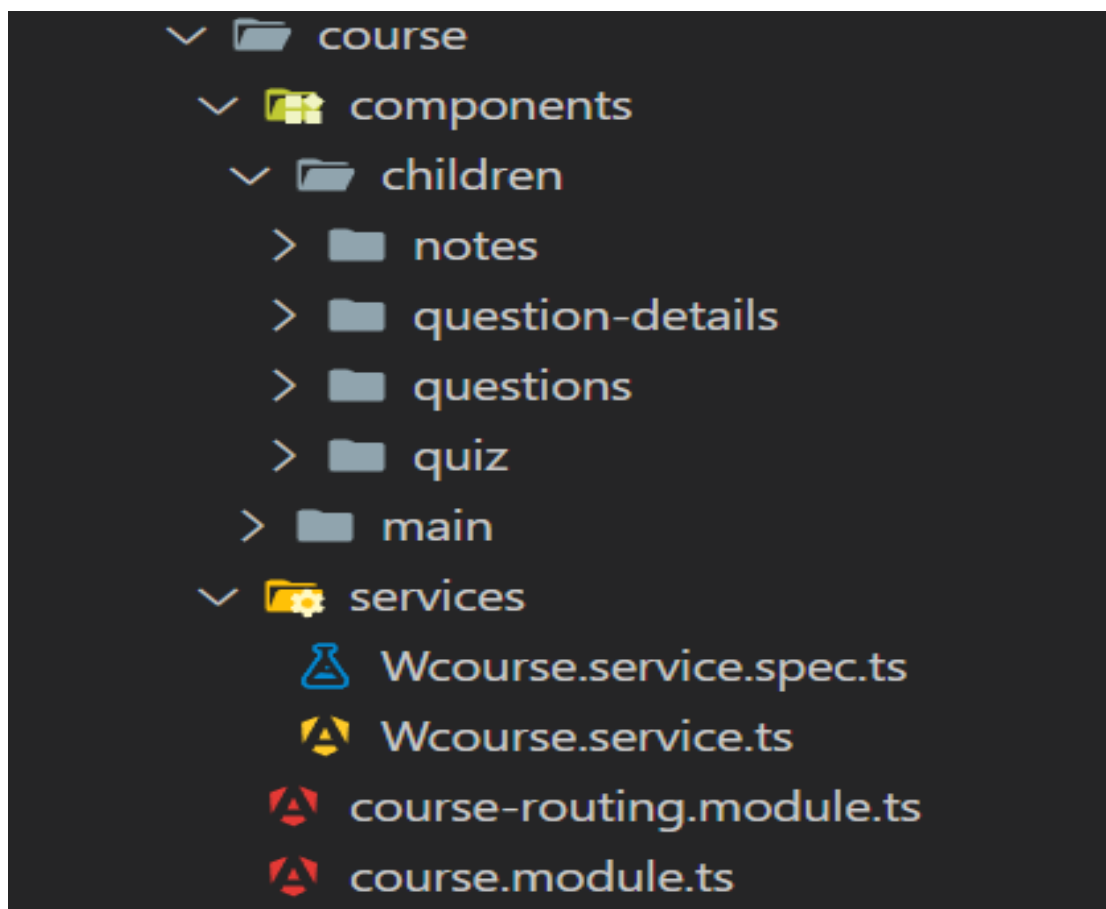


FIGURE 13.7: Course Module Architecture

- **Courses Module:** Manages the display and organization of multiple courses, including:

- **Components:**
 - * **course-details Component:** Displays each course with full details.
 - * **courses Component:** Displays all courses and allows filtering.
 - * **curriculum Component:** Allows adding, updating, and deleting content of each course.
 - * **home Component:** Serves as the home page after signing in.
- **Interfaces:**
 - * course Interface
 - * curriculum Interface
 - * quiz-question Interface
 - * reviews Interface
- **Services:** Handles various course-related API interactions, including:
 - * course Services
 - * curriculum-item Services
 - * curriculum-lecture Services
 - * curriculum-quiz-questions Services
 - * curriculum-quiz Services
 - * curriculum-section Services
 - * curriculum-show Services
 - * reviews Services

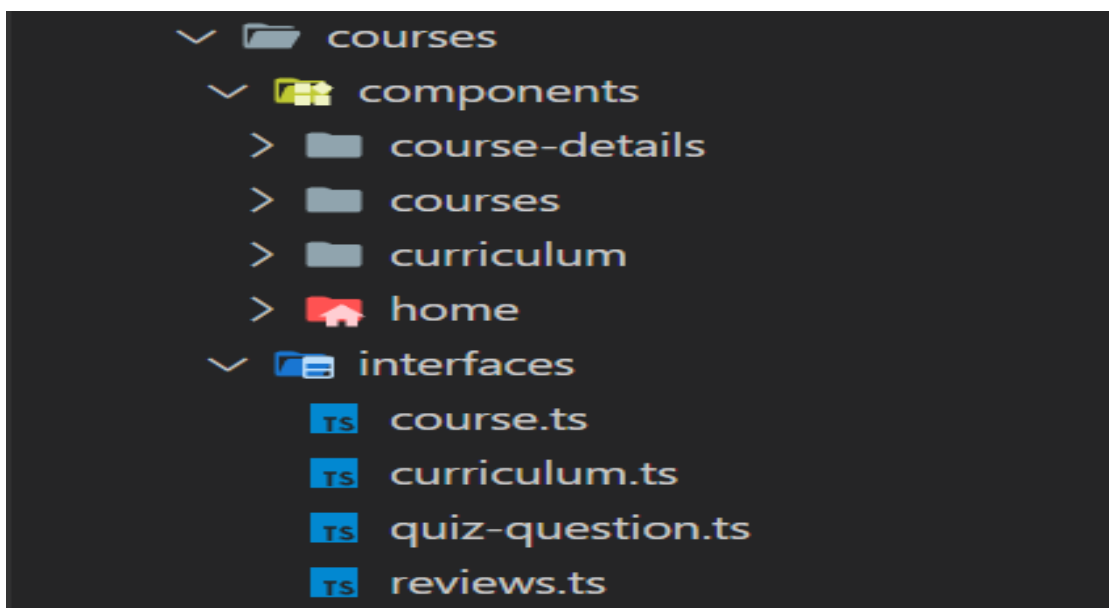


FIGURE 13.8: Courses Module Architecture

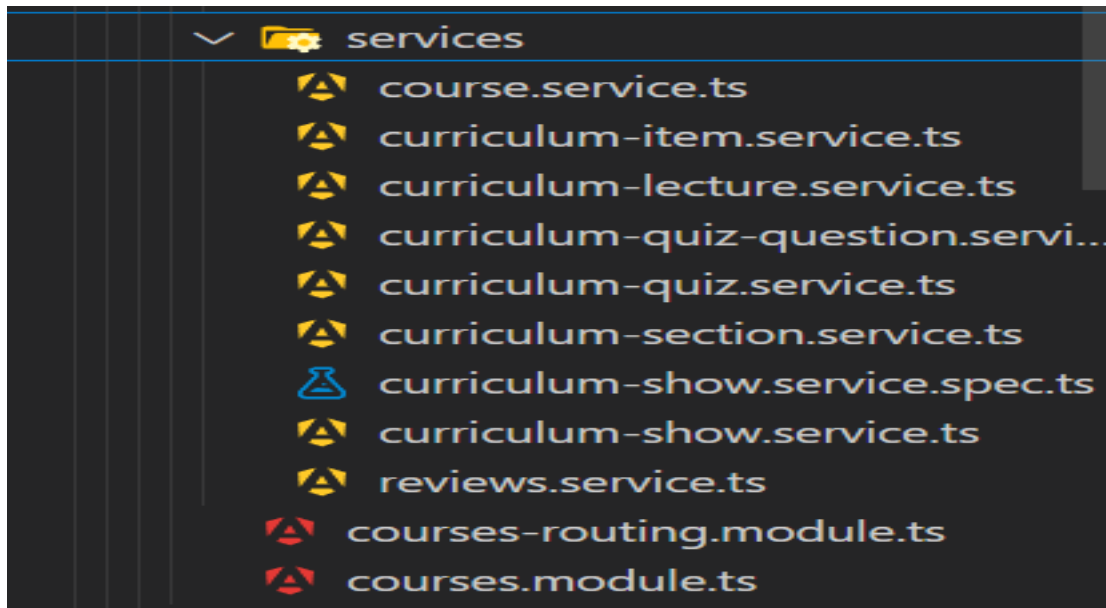


FIGURE 13.9: Courses Module Architecture

- **Dashboard Module:** Provides an interface for users to manage their profiles, courses, and other settings. The dashboard module is structured to accommodate different user roles:

- **Student Users Components:**

- * **dashboard Component:** Displays the main dashboard interface for students.
- * **my-profile Component:** Allows students to view and manage their profile information.
- * **enrolled-courses Component:** Displays courses that the student is currently enrolled in.
- * **purchase-history Component:** Shows the history of course purchases made by the student.
- * **wish-list Component:** Allows students to add courses to their wish list.
- * **reviews Component:** Manages and displays reviews submitted by the student.
- * **settings Component:** Provides settings options for the student's account.
- * **myquiz-attempts Component:** Displays quizzes attempted by the student.

- **Instructor Users Components:**

- * **i-announcements Component:** Allows instructors to post announcements.
- * **i-mycourse Component:** Manages courses taught by the instructor.
- * **i-mystudents Component:** Displays students enrolled in courses taught by the instructor.
- * **i-qand-a Component:** Manages questions and answers related to courses taught by the instructor.
- * **i-quiz-attempts Component:** Displays quiz attempts for courses taught by the instructor.
- * **i-withdrawals Component:** Handles withdrawal requests from students enrolled in courses taught by the instructor.
- **Admin Users Components:**
 - * **admin Component:** Provides administrative functionalities for managing the entire platform.
- **Interfaces:**
 - * `categories` Interface
 - * `eventcountry` Interface
 - * `eventCategory` Interface
 - * `meetings` Interface
 - * `subject` Interface
 - * `user` Interface
- **Services:** Handles various functionalities related to categories, events, and subjects, including:
 - * `category` Services
 - * `eventCategory` Services
 - * `eventCountry` Services
 - * `meetingsCategory` Services
 - * `subject` Services

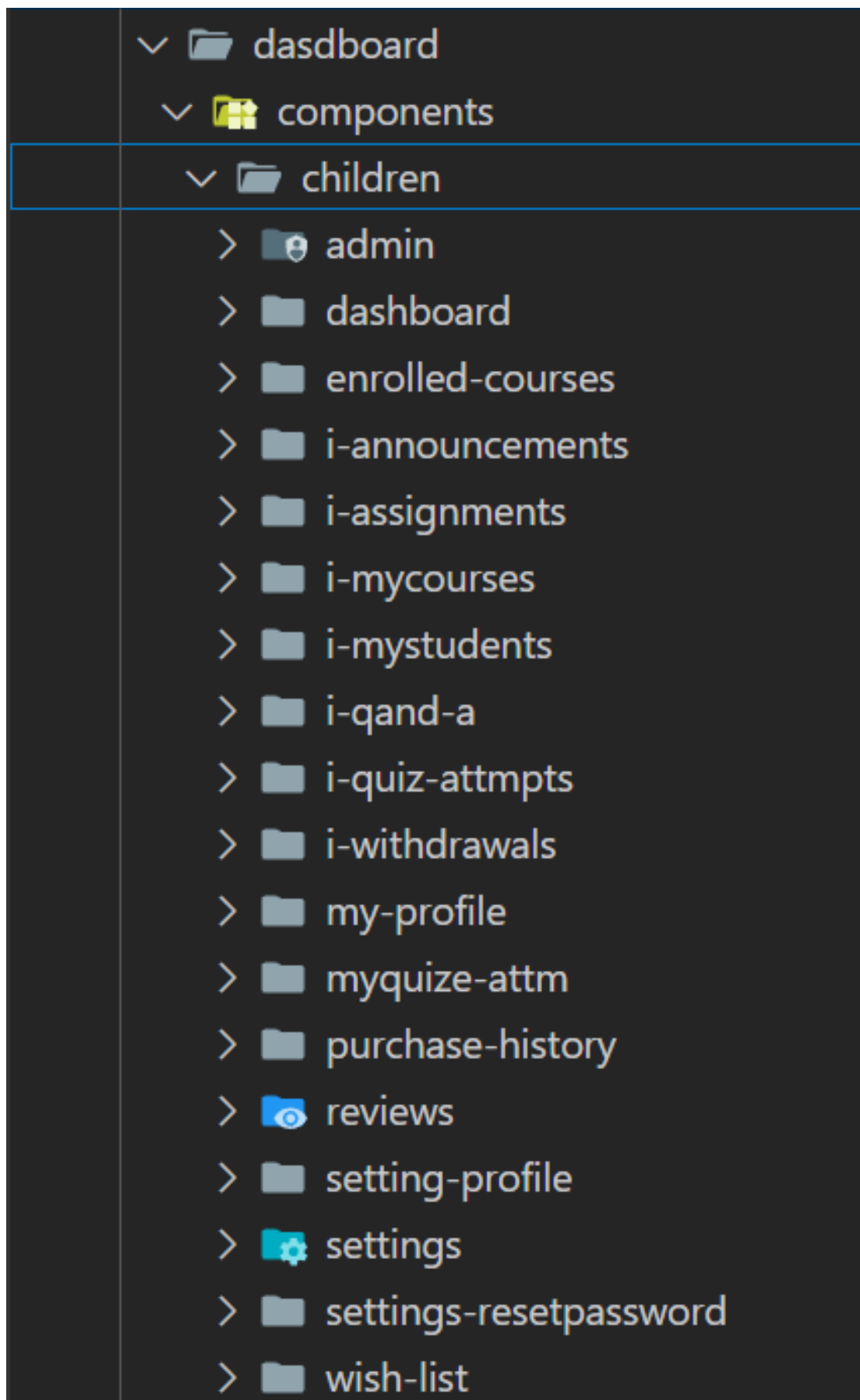


FIGURE 13.10: Dashboard Module Architecture

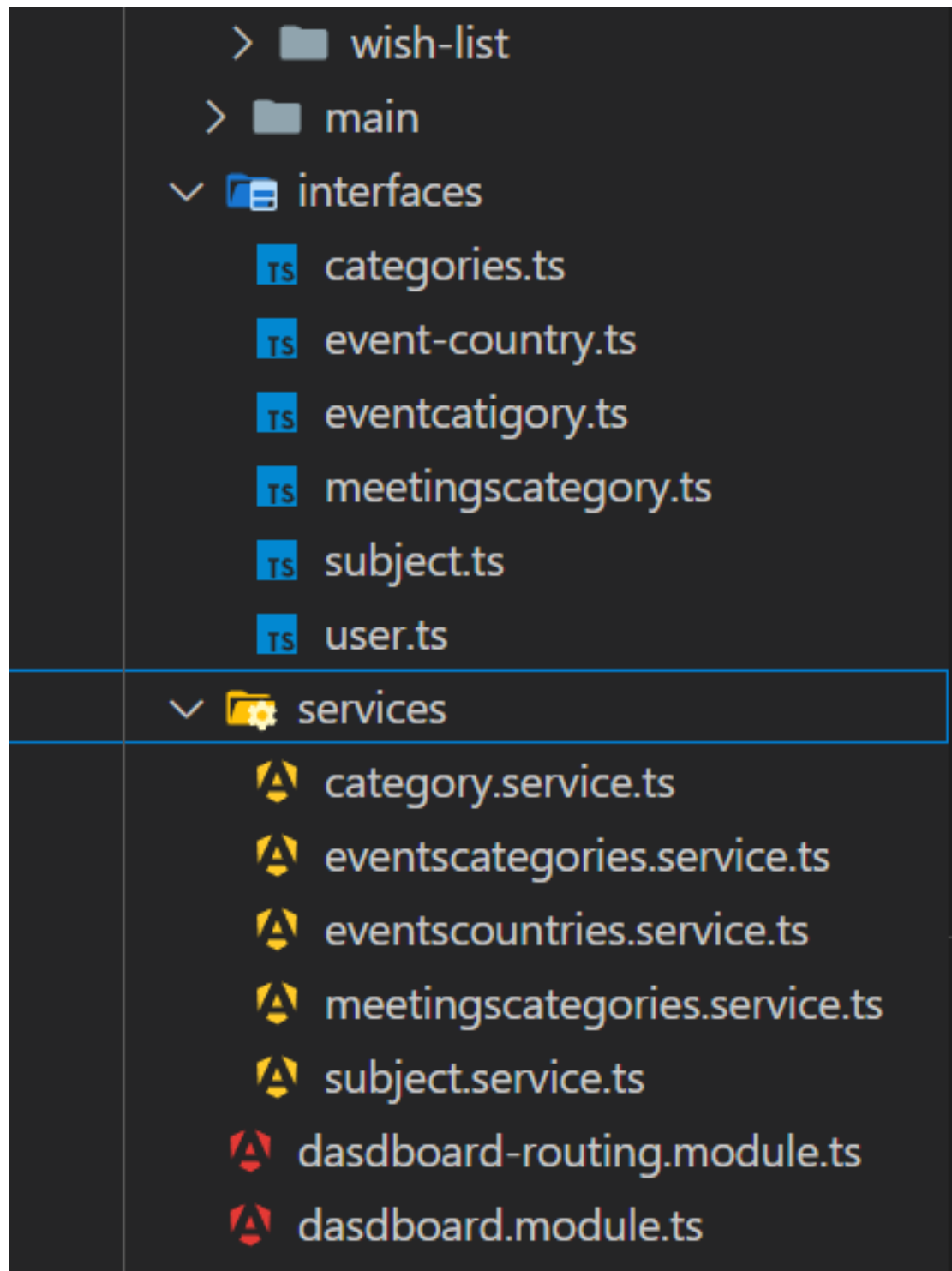


FIGURE 13.11: Dashboard Module Architecture

- **Pages Module:** Contains static and dynamic pages that do not fit into the other specific modules. It includes:
 - **chatbot Component:** Manages interactions with a chatbot using AI.
 - **translation Component:** Handles translations using AI.

- **translate Services:** Provides services for translation functionalities.

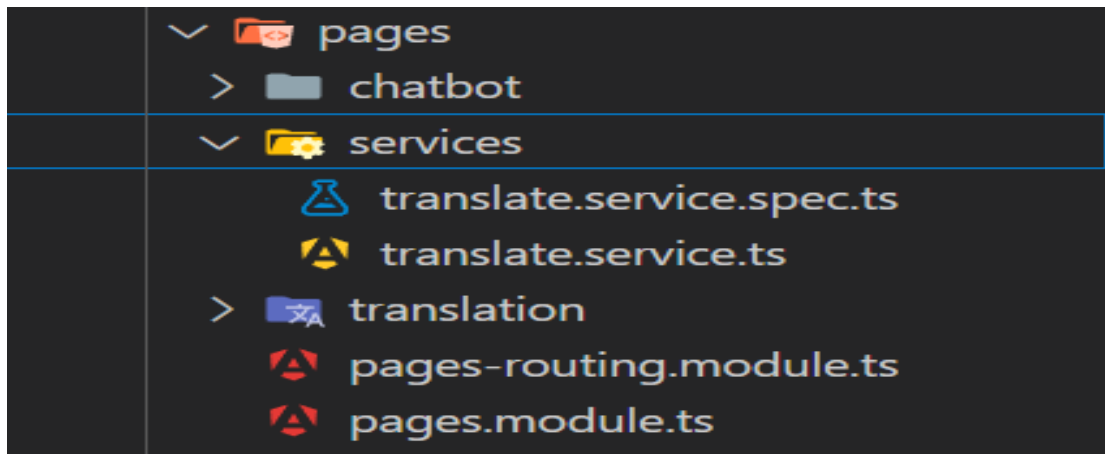


FIGURE 13.12: Pages Module Architecture

- **Event Module:** Manages events, including their creation, display, and registration functionalities. It includes:
 - **events Component:** Displays all events and allows adding new events.
 - **event-details Component:** Displays detailed information about a specific event.
 - **event Service:** Handles API interactions for events, including:
 - * addEvent API
 - * updateEvent API
 - * deleteEvent API
 - * getEvent API

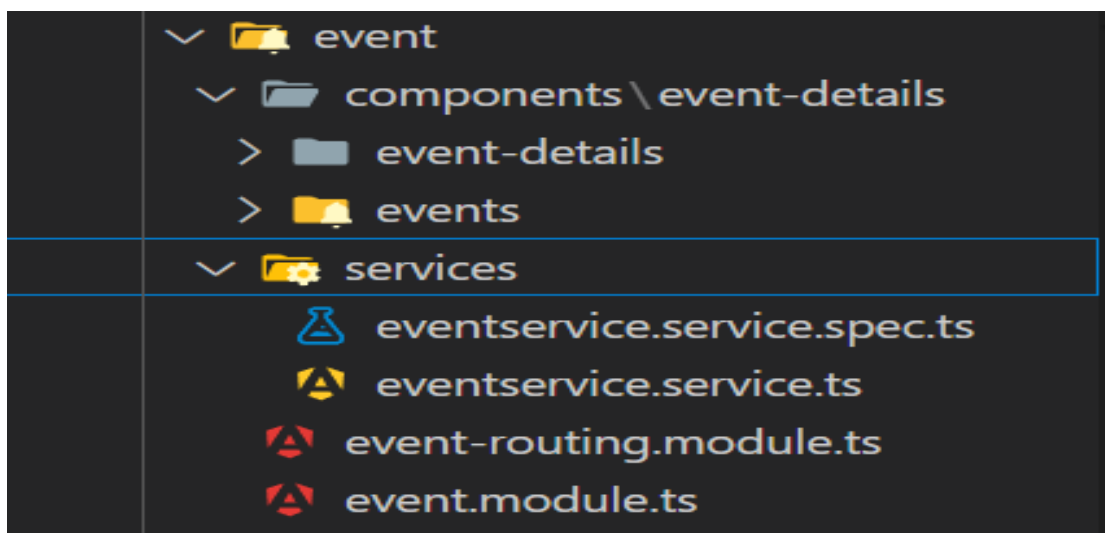


FIGURE 13.13: Event Module Architecture

- **Shared Module:** The shared module is utilized across the entire project and contains reusable components and services that are shared among different modules. It includes:
 - **Navbar Component:** Displays the navigation bar across the application, providing consistent navigation options for users.
 - **Not-Found404 Component:** Displays an error page (404) when a user enters an incorrect path, improving user experience by handling invalid routes gracefully.

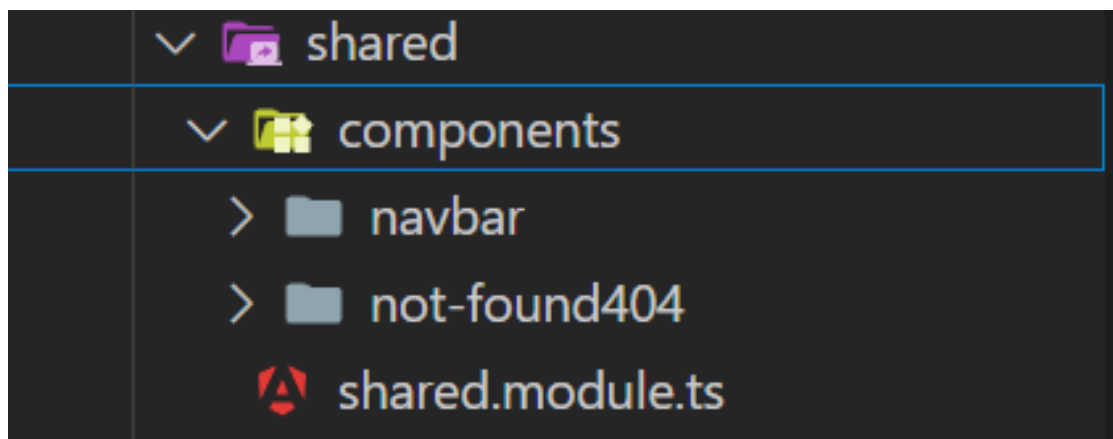


FIGURE 13.14: Shared Module Architecture

- **Zoom Module:** Manages Zoom meeting functionalities, facilitating interaction and management of online meetings. It includes:
 - **zoom-meetings Component:** Displays all Zoom meetings and allows adding new meetings.
 - **zoom-meetings-details Component:** Displays detailed information about a specific Zoom meeting.
 - **Interfaces:**
 - * meeting Interface
 - **Services:** Handles Zoom-related API interactions, including:
 - * zoom Services

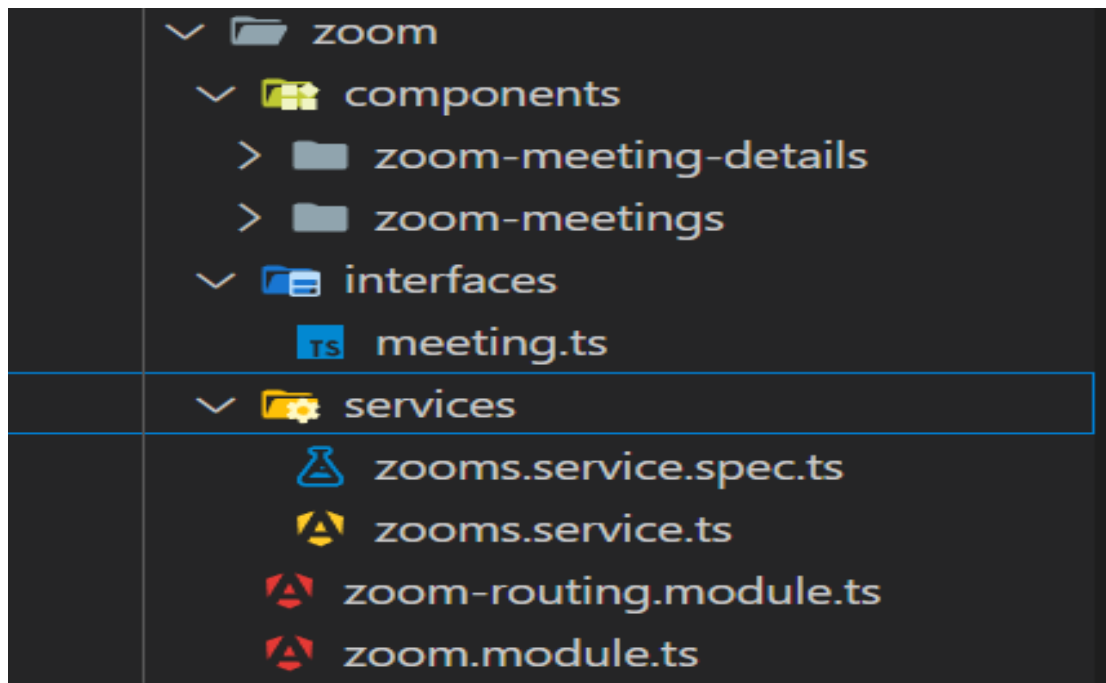


FIGURE 13.15: Zoom Module Architecture

13.5 Integration of Angular and .NET

The integration of Angular frontend with the .NET backend is crucial for the functionality of the e-learning platform. The following points outline the integration:

- **RESTful APIs:** The .NET backend provides RESTful APIs that Angular consumes for data operations. These APIs handle business logic, data validation, and interaction with the database.
- **HTTP Services:** Angular's HttpClient module makes HTTP requests to the .NET APIs, including GET, POST, PUT, and DELETE requests for various resources.
- **Authentication:** Angular's Auth Service manages user authentication by interacting with the .NET backend, which handles user sessions and security tokens.
- **Data Binding and State Management:** Angular's data binding ensures that the frontend UI reflects the current state of backend data. User actions that modify data are sent to the backend via APIs.

13.6 Design Patterns and Principles

The e-learning platform adheres to several design patterns and principles to ensure code quality and maintainability:

- **Modular Architecture:** Divides the application into reusable modules, each responsible for a specific feature set.
- **Separation of Concerns:** Ensures that each component and service has a single responsibility, making the codebase cleaner and easier to maintain.
- **Dependency Injection:** Utilizes Angular's dependency injection system to manage service dependencies, enhancing testability and scalability.
- **Lazy Loading:** Improves application performance by loading modules only when required, reducing initial load times.

13.7 Security Measures

Security is a top priority for the e-learning platform architecture:

- **Authentication Guards:** Protects routes to ensure only authenticated users can access certain parts of the application.
- **Form Validation:** Uses Angular's reactive forms for strong client-side validation, preventing invalid data submissions.
- **Secure APIs:** The .NET backend employs token-based authentication and authorization to secure API endpoints, ensuring data privacy and integrity.

13.8 Database and API Interactions

The backend interacts with a database to persist and retrieve data. Angular communicates with the backend via APIs for CRUD operations:

- **Data Storage:** The .NET backend manages data storage using a relational database, storing user profiles, course content, and other essential information.
- **API Operations:** Angular's services interact with backend APIs to perform operations such as creating, reading, updating, and deleting data, ensuring seamless data management and user interaction.

Chapter 14

Libraries Used in the Front-End Project

14.1 Font Awesome



FIGURE 14.1: FontAwesome

Font Awesome is a popular icon toolkit used to add scalable vector icons to web projects. It provides a wide range of icons, including those for social media, user interface elements, and various other graphics, making it a versatile choice for enhancing the visual appeal and functionality of web applications.

Font Awesome icons are implemented as fonts, which means they can be styled using CSS, resized without losing quality, and manipulated like any other text element. This flexibility allows developers to integrate icons seamlessly into their designs and ensure consistent appearance across different devices and screen sizes.

14.1.1 Description of Font Awesome

Font Awesome offers an extensive collection of free and paid icons, covering a wide range of use cases. These icons are vector-based, meaning they are resolution-independent and look sharp at any size. Font Awesome also supports accessibility features, making it easier to create inclusive web applications.

Key features of Font Awesome include:

- **Scalability:** Icons can be scaled to any size without losing quality.
- **Customizability:** Icons can be styled using CSS, allowing for changes in color, size, and effects.
- **Variety:** A vast library of icons covering various categories, such as web application icons, social media icons, and more.
- **Accessibility:** Built-in support for screen readers and other accessibility tools.
- **Ease of Use:** Simple integration with HTML, making it easy to add icons to any web project.

14.1.2 Usage in Our Project

In our e-learning platform, we used Font Awesome to enhance the user interface by providing visually appealing and intuitive icons. The icons were employed in various parts of the application to improve user interaction and navigation. Here are some examples of how Font Awesome was utilized:

- **Navigation Bar:** Icons were used in the navigation bar to represent different sections of the application, making it easier for users to understand the purpose of each link at a glance.
- **Buttons and Actions:** Icons were added to buttons and action links to provide visual cues about their functions, such as adding, editing, or deleting items.
- **User Feedback:** Icons were used to display feedback messages, such as success, error, and warning notifications, helping users quickly recognize the type of message being conveyed.
- **Course Content:** Icons were included in the course content to represent different types of learning materials, such as videos, quizzes, and assignments, enhancing the visual organization of the content.

By integrating Font Awesome into our project, we were able to create a more engaging and user-friendly interface, contributing to a better overall user experience.

14.2 Bootstrap



FIGURE 14.2: Bootstrap

Bootstrap is a widely-used front-end framework for developing responsive and mobile-first web applications. It provides a comprehensive set of CSS and JavaScript components, including typography, forms, buttons, navigation, and other interface elements. Bootstrap's predefined classes and grid system make it easy to create layouts and designs that work seamlessly across different devices and screen sizes.

14.2.1 Description of Bootstrap

Bootstrap is known for its ease of use and extensive documentation, making it a popular choice among developers for quickly building and styling web applications. The framework follows a modular approach, allowing developers to include only the components they need. Bootstrap's grid system is particularly powerful for creating responsive layouts, as it allows for flexible and consistent design across various screen sizes.

Key features of Bootstrap include:

- **Responsive Grid System:** A flexible and scalable grid system that adapts to different screen sizes and devices.

- **Predefined Components:** A wide range of reusable components such as buttons, forms, modals, and navigation bars.
- **Customizable:** Easily customizable through CSS and Sass variables to fit the specific needs of a project.
- **Cross-Browser Compatibility:** Ensures consistent appearance and functionality across different browsers.
- **Extensive Documentation:** Comprehensive documentation and examples that help developers quickly learn and implement Bootstrap components.

14.2.2 Usage in Our Project

In our e-learning platform, we used Bootstrap to enhance the visual style and responsiveness of the application. By leveraging Bootstrap's predefined classes and components, we were able to create a consistent and attractive user interface. Here are some examples of how Bootstrap was utilized:

- **Classes for Styling:** Bootstrap classes were used to style various elements, ensuring a uniform and modern look throughout the application.
- **Forms:** Bootstrap forms were used to create user-friendly and accessible forms for registration, login, and other user interactions.
- **Modals:** Bootstrap modals were implemented for displaying dialogs and pop-up messages, enhancing user engagement and interactivity.
- **Inputs and Buttons:** Predefined input and button styles were used to ensure consistency and improve the usability of interactive elements.
- **Responsive Layouts:** The Bootstrap grid system was employed to create responsive layouts that adapt to different screen sizes, providing an optimal viewing experience on all devices.

By incorporating Bootstrap into our project, we were able to streamline the development process, achieve a polished look and feel, and ensure a responsive design that enhances the overall user experience.

14.3 jQuery



FIGURE 14.3: JQuery

jQuery is a fast, small, and feature-rich JavaScript library. It simplifies things like HTML document traversal and manipulation, event handling, animation, and Ajax with an easy-to-use API that works across a multitude of browsers. jQuery has changed the way that millions of people write JavaScript, making it more accessible and easier to develop complex functionalities with less code.

14.3.1 Description of jQuery

jQuery was created to simplify client-side scripting of HTML. It abstracts many of the complexities of dealing with raw JavaScript and provides a more concise and readable syntax. The library is lightweight and efficient, offering a wide range of functionalities that enhance interactivity and responsiveness in web applications.

Key features of jQuery include:

- **DOM Manipulation:** Simplifies the process of selecting and manipulating HTML elements and attributes.
- **Event Handling:** Provides a robust framework for handling events, making it easier to create interactive web pages.
- **Animations:** Built-in methods for creating rich animations and effects, enhancing user experience.

- **Ajax Support:** Simplifies asynchronous HTTP requests, enabling dynamic content updates without reloading the page.
- **Cross-Browser Compatibility:** Ensures consistent behavior across different browsers.
- **Extensible:** Supports plugins, allowing developers to extend the library with additional functionalities.

14.3.2 Usage in Our Project

In our e-learning platform, we used jQuery to add animations and enhance interactivity, making the website more dynamic and engaging. The simplicity and power of jQuery allowed us to implement these features efficiently. Here are some examples of how jQuery was utilized:

- **Animations:** We used jQuery functions such as `slideUp`, `slideDown`, `fadeIn`, and `fadeOut` to create smooth transitions and effects that improve the user experience.
- **Toggle Functionality:** The `toggle` function was used to show or hide elements, providing interactive features like expandable menus and collapsible sections.
- **Scroll to Top:** We implemented a `scrollTop` function to create a scroll-to-top button, allowing users to easily navigate back to the top of the page with a single click.
- **Event Handling:** jQuery's event handling capabilities were used to manage user interactions, such as clicks and form submissions, making the application more responsive to user inputs.

By integrating jQuery into our project, we were able to enhance the visual appeal and interactivity of the website, making it more engaging and user-friendly.

14.4 AOS.js



FIGURE 14.4: AOS.js

AOS (Animate On Scroll) is a small library that allows you to animate elements as you scroll down, and up, the page. It is simple to use, highly customizable, and provides a wide range of animations to enhance the visual appeal and interactivity of web applications. AOS.js is particularly useful for creating engaging and dynamic user experiences by adding animations to elements when they first appear in the viewport.

14.4.1 Description of AOS.js

AOS.js is designed to animate elements based on their scroll position, making it easy to implement scroll-triggered animations without writing complex JavaScript code. The library offers various animation effects, such as fade, slide, zoom, flip, and more. These animations can be configured to trigger at different points during the scroll, providing flexibility in how and when elements animate.

Key features of AOS.js include:

- **Ease of Use:** Simple initialization and configuration with HTML data attributes.
- **Customizable Animations:** A wide range of built-in animations with options to customize duration, delay, and easing.
- **Lightweight:** Minimal impact on page load times due to its small size.

- **Responsive:** Works well across different devices and screen sizes, ensuring consistent animations.
- **No Dependencies:** Standalone library that does not require jQuery or any other dependencies.

14.4.2 Usage in Our Project

In our e-learning platform, we used AOS.js to add animations that enhance the user experience by animating elements when they first appear in the viewport. This approach helps to draw attention to key content and makes the interface more engaging. Here are some examples of how AOS.js was utilized:

- **Element Entry Animations:** Elements such as course cards, blog posts, and sections of the dashboard were animated with effects like fade-in, slide-up, and zoom-in when they first became visible to the user.
- **Enhanced Interactivity:** Animations were added to interactive components, such as buttons and icons, to provide visual feedback and improve user engagement.
- **User Engagement:** Key elements that we wanted to highlight, such as new features or important announcements, were animated to capture the user's attention effectively.
- **Smooth Transitions:** The use of AOS.js ensured smooth transitions between different sections of the page, creating a cohesive and visually appealing browsing experience.

By integrating AOS.js into our project, we were able to create a more dynamic and visually engaging interface, improving the overall user experience.

14.5 Parallax.js



FIGURE 14.5: Parallax.Js

Parallax.js is a lightweight JavaScript library used to create parallax scrolling effects on background images or other elements in web pages. It allows you to animate elements at different speeds and directions based on the user's scroll or cursor movement, enhancing the visual appeal and interactivity of the website.

14.5.1 Description of Parallax.js

Parallax.js simplifies the implementation of parallax scrolling effects without requiring complex calculations or extensive JavaScript code. The library provides a straightforward API for creating smooth animations that respond to user actions, such as scrolling or mouse movements. This makes it ideal for adding depth and visual interest to backgrounds or other elements on web pages.

Key features of Parallax.js include:

- **Ease of Use:** Simple integration with HTML elements using data attributes or JavaScript initialization.
- **Customizable Effects:** Supports parallax effects for background images, foreground elements, or entire sections of a webpage.

- **Responsive Design:** Ensures that parallax effects adapt smoothly to different screen sizes and devices.
- **Performance Optimization:** Lightweight and optimized for performance to minimize impact on page loading times.
- **Interactive Animations:** Allows for interactive parallax effects triggered by user actions such as scrolling or cursor movement.

14.5.2 Usage in Our Project

In our e-learning platform, we used Parallax.js to create engaging and interactive parallax effects on background images. These effects were designed to respond to the user's cursor movements, adding a layer of visual depth and dynamism to the website.

Here are some examples of how Parallax.js was utilized:

- **Background Animations:** Parallax.js was applied to background images on landing pages, course sections, or promotional banners, creating a sense of immersion and visual interest.
- **Cursor Interaction:** Background elements were animated to move at different speeds and directions relative to the user's cursor, providing a playful and interactive experience.
- **Scroll Triggered Effects:** Parallax.js effects were synchronized with scrolling actions to reveal hidden content or highlight key features dynamically.
- **User Engagement:** The use of parallax scrolling helped to capture and maintain user attention, making the website more engaging and memorable.

By incorporating Parallax.js into our project, we enhanced the visual appeal and interactivity of the e-learning platform, creating a more immersive and enjoyable user experience.

14.6 Perfect Scrollbar

Perfect Scrollbar is a lightweight and customizable scrollbar plugin for web applications that enhances the scrolling experience by replacing the default browser scrollbar with a more aesthetic and feature-rich alternative. It provides smooth scrolling behavior and supports custom styling and functionality, making it suitable for applications that require a polished and user-friendly interface.

14.6.1 Description of Perfect Scrollbar

Perfect Scrollbar improves the user experience by offering a scrollbar solution that is highly customizable and responsive to user interactions. Unlike the standard browser scrollbar, Perfect Scrollbar allows developers to style the scrollbar to match the overall design aesthetics of their application. It also includes additional features such as touch support and smooth scrolling animations, ensuring consistent performance across different devices and browsers.

Key features of Perfect Scrollbar include:

- **Customizable Styles:** Allows developers to customize the scrollbar's appearance using CSS, including colors, sizes, and gradients.
- **Smooth Scrolling:** Provides a smooth scrolling experience with options for easing and acceleration, enhancing user navigation through content.
- **Responsive Design:** Adapts seamlessly to different screen sizes and devices, maintaining functionality and usability across platforms.
- **Touch Support:** Supports touch gestures for scrolling on mobile devices, making it suitable for responsive web applications.
- **Performance Optimization:** Optimized for performance to minimize overhead and ensure fast rendering of scrollable content.

14.6.2 Usage in Our Project

In our e-learning platform, we integrated Perfect Scrollbar to improve the scrolling experience within various components and pages. This plugin was particularly useful for sections of the application that required scrolling through lists of courses, blog posts, or user notifications. Here are some examples of how Perfect Scrollbar was utilized:

- **Course Listings:** Perfect Scrollbar was applied to course listings to provide a smoother and more visually appealing scrolling experience when users browse through available courses.
- **Blog Sections:** Scrollbars in blog sections were customized using Perfect Scrollbar to ensure consistent styling and functionality across different content areas.

- **User Dashboard:** Implemented Perfect Scrollbar in the user dashboard to enhance navigation through personalized content, such as enrolled courses, purchase history, and notifications.
- **Custom Styling:** Utilized CSS customization options to match the scrollbar's appearance with the overall theme of the e-learning platform, maintaining a cohesive design.

By incorporating Perfect Scrollbar into our project, we improved the usability and aesthetics of scrollable content, contributing to a more enjoyable and efficient user experience.

14.7 Swiper

Swiper is a modern, mobile-friendly JavaScript library used for creating responsive, touch-enabled sliders and carousels. It is highly flexible and offers a wide range of features, making it a popular choice for web developers looking to implement smooth and interactive sliders in their applications.

14.7.1 Description of Swiper

Swiper is designed to provide a seamless sliding experience across all devices and browsers. It supports horizontal and vertical slides, multiple slide effects, and is fully customizable. Swiper is highly performant, leveraging modern CSS and JavaScript features to ensure smooth animations and responsive behavior.

Key features of Swiper include:

- **Responsive Design:** Automatically adjusts to different screen sizes and orientations, providing a consistent experience on both desktop and mobile devices.
- **Touch and Gesture Support:** Includes touch and gesture recognition, allowing users to swipe through slides on touch-enabled devices.
- **Multiple Slide Effects:** Supports a variety of slide effects, such as fade, cube, coverflow, and flip, which can be customized to fit the application's design.
- **Modular Architecture:** Offers a modular architecture where only the necessary components are included, optimizing performance.

- **Customization and Theming:** Extensive API and options for customizing the look and behavior of sliders, including navigation controls, pagination, and scrollbar.

14.7.2 Usage in Our Project

In our e-learning platform, we used Swiper to create interactive and visually appealing sliders for various components, enhancing the user experience by making content more accessible and engaging. Here are some examples of how Swiper was utilized:

- **Course Previews:** Implemented Swiper for course preview carousels, allowing users to swipe through featured courses and view quick summaries.
- **Testimonial Slides:** Used Swiper to display user testimonials in a slider format, providing a dynamic way to showcase feedback and reviews.
- **Image Galleries:** Integrated Swiper for image galleries within course content and blog posts, enabling users to swipe through images seamlessly.
- **Promotional Banners:** Added Swiper to the homepage and other sections to cycle through promotional banners and announcements, drawing attention to key content.
- **Custom Navigation:** Customized the navigation controls and pagination to match the platform's design, ensuring a consistent and branded user interface.

By incorporating Swiper into our project, we were able to deliver a more interactive and user-friendly experience, making it easier for users to navigate and engage with content on the e-learning platform.

14.8 jQuery PowerTip

jQuery PowerTip is a robust and flexible tooltip plugin for jQuery that provides a highly customizable way to display tooltips. It is designed to offer advanced positioning options, theming, and interactive content within tooltips, making it a versatile tool for enhancing user interfaces.

14.8.1 Description of jQuery PowerTip

jQuery PowerTip enhances the default tooltip functionality with a wide array of features. It supports complex configurations and customizations, allowing developers to create tooltips that match the style and behavior requirements of their applications. PowerTip offers precise positioning options, event handling, and theming capabilities, ensuring that tooltips are both functional and visually appealing.

Key features of jQuery PowerTip include:

- **Advanced Positioning:** Offers precise control over tooltip positioning with smart placement and collision detection to ensure tooltips are displayed optimally.
- **Customizable Themes:** Allows the creation of custom themes for tooltips using CSS, enabling consistent styling with the rest of the application.
- **Interactive Content:** Supports HTML content within tooltips, making it possible to include interactive elements such as links, images, and forms.
- **Event Handling:** Provides event hooks for customizing tooltip behavior based on user interactions, such as hover, click, or focus events.
- **Performance Optimization:** Optimized for performance to handle large numbers of tooltips efficiently without impacting page load times.

14.8.2 Usage in Our Project

In our e-learning platform, we integrated jQuery PowerTip to enhance the user experience by providing informative and interactive tooltips. These tooltips were used to give users additional context and information without cluttering the interface. Here are some examples of how jQuery PowerTip was utilized:

- **Course Information:** Implemented tooltips to display detailed information about courses when users hover over course titles or icons, providing quick access to key details.
- **Navigation Help:** Used tooltips to offer guidance on navigation elements, helping users understand the functionality of various buttons and links.
- **Form Assistance:** Integrated tooltips within forms to provide helpful hints and validation messages, enhancing the user experience during form completion.

- **Interactive Elements:** Added interactive content such as links and images within tooltips to engage users and provide additional resources or actions.
- **Custom Styling:** Customized the appearance of tooltips to match the platform's design, ensuring a cohesive and professional look throughout the application.

By incorporating jQuery PowerTip into our project, we improved the overall usability and accessibility of the e-learning platform, making it easier for users to find and understand relevant information.

14.9 JWT-decode

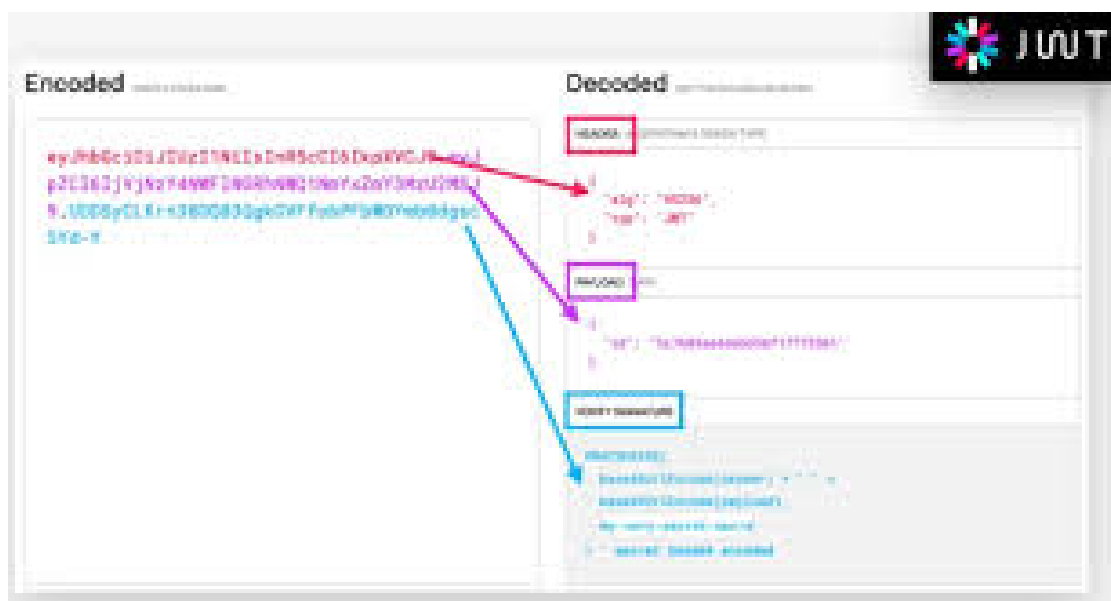


FIGURE 14.6: JWT-Code

JWT-decode is a JavaScript library used to decode JSON Web Tokens (JWTs) without requiring server-side interaction. It simplifies the process of extracting and interpreting the payload data from JWTs, which is useful for client-side authentication and authorization processes.

14.9.1 Description of JWT-decode

JWT-decode provides a straightforward way to decode JWTs and retrieve the embedded information. JWTs are commonly used for securely transmitting information between parties as a JSON object. This library is particularly valuable for client-side applications that need to access and utilize the data stored in JWTs.

Key features of JWT-decode include:

- **Simple Decoding:** Allows developers to decode JWTs with a single function call, extracting the payload data without any server-side dependencies.
- **No Verification:** Focuses on decoding the token rather than verifying its signature, making it lightweight and fast for client-side usage.
- **Small Footprint:** The library is small and efficient, minimizing the impact on the application's performance and load time.
- **Compatibility:** Supports decoding JWTs in various environments, including browsers and Node.js, ensuring versatility in different project setups.
- **Ease of Use:** Offers a user-friendly API that simplifies the integration process and reduces the complexity of working with JWTs.

14.9.2 Usage in Our Project

In our e-learning platform, we used JWT-decode to handle client-side decoding of JWTs received from the backend. This allowed us to read the data contained in the tokens and utilize it for various authentication and authorization tasks. Here are some examples of how JWT-decode was utilized:

- **User Authentication:** Decoded JWTs to extract user information, such as user ID and roles, enabling personalized content delivery and access control.
- **Session Management:** Used the decoded data to manage user sessions, ensuring that users remained authenticated and their sessions were valid.
- **Authorization Checks:** Implemented client-side authorization checks by decoding JWTs and verifying user permissions for accessing specific features or pages.
- **Profile Information:** Retrieved user profile data from the token payload to display personalized information on the dashboard and other components.
- **API Integration:** Integrated JWT-decode with other libraries and services that required decoded token information to function correctly, streamlining the development process.

By incorporating JWT-decode into our project, we enhanced the security and efficiency of our authentication system, providing a seamless experience for users while maintaining robust access control mechanisms.

14.10 Angular CDK

Angular CDK (Component Dev Kit) is a library developed by the Angular team to provide a set of behavior-driven tools and utilities for building Angular components. It provides foundational APIs for common component behaviors, such as accessibility, drag-and-drop, and scrolling, enabling developers to build sophisticated UIs without reinventing the wheel.

14.10.1 Description of Angular CDK

Angular CDK offers a suite of tools that make it easier to develop custom components and interactions in Angular applications. It is designed to work seamlessly with Angular Material but can be used independently in any Angular project.

Key features of Angular CDK include:

- **Accessibility:** Tools for building accessible components, ensuring that applications are usable by all users, including those with disabilities.
- **Overlay:** Services for creating floating panels, dialogs, and other overlay-based components.
- **Drag-and-Drop:** APIs for implementing drag-and-drop functionality, including sorting and reordering items.
- **Scrolling:** Utilities for managing virtual scrolling, making it easier to handle large lists of items efficiently.
- **Portal:** Tools for dynamically rendering components in different parts of the DOM.

14.10.2 Usage in Our Project

In our e-learning platform, we used Angular CDK to implement drag-and-drop functionality, making it easy for instructors to reorder the content of their courses. This enhanced the user experience and streamlined the course management process.

Here are some examples of how Angular CDK was utilized:

- **Course Content Management:** Implemented drag-and-drop functionality using Angular CDK to allow instructors to easily reorder course sections, lectures, and quizzes.

- **Interactive UI Components:** Utilized Angular CDK to create dynamic and interactive user interface elements that enhance user interaction.
- **Efficient Scrolling:** Leveraged the scrolling utilities to efficiently manage and display large lists of items, ensuring smooth performance.
- **Overlay Components:** Used the overlay services to create custom dialogs and floating panels, providing additional functionality and information to users.
- **Enhanced User Experience:** Improved the overall user experience by integrating accessible and interactive components that adhere to modern design principles.

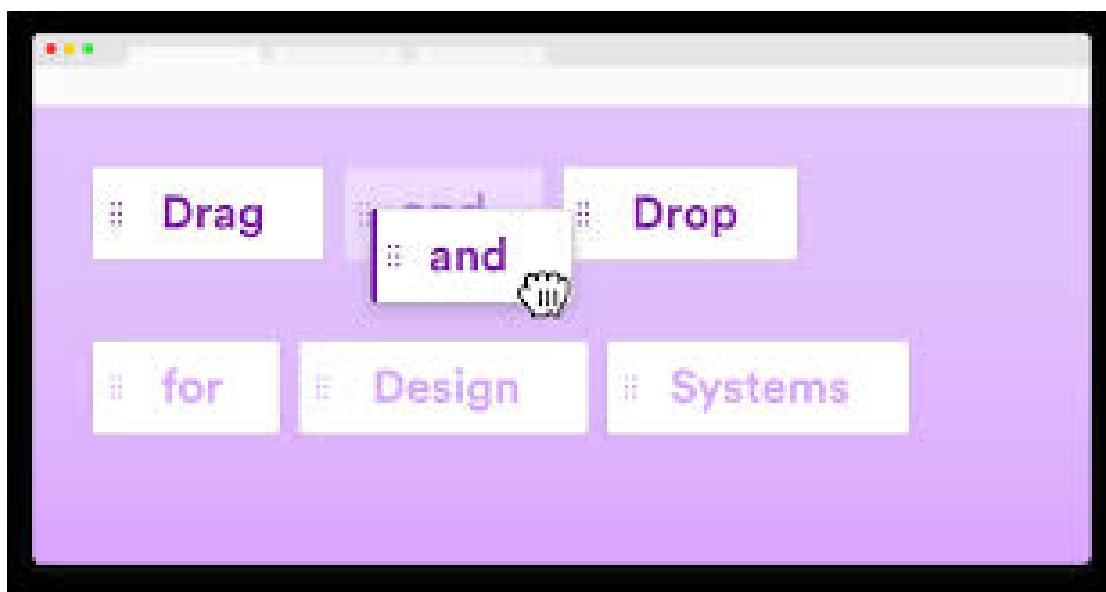


FIGURE 14.7: Drag Drop

By incorporating Angular CDK into our project, we were able to build a more interactive, user-friendly, and visually appealing e-learning platform, enhancing both the instructor and student experiences.

14.11 @abacritt/angularx-social-login

`@abacritt/angularx-social-login` is an Angular library that provides a simple and configurable way to integrate social login functionality into an Angular application. It supports multiple social providers, including Google and Facebook, allowing users to authenticate using their existing social media accounts.

14.11.1 Description of @abacritt/angularx-social-login

@abacritt/angularx-social-login makes it easy to add social login capabilities to an Angular application. It abstracts the complexity of interacting with social media APIs, providing a straightforward API for developers to use. The library handles the OAuth flow, token management, and user profile retrieval, simplifying the integration process.

Key features of @abacritt/angularx-social-login include:

- **Multiple Providers:** Supports popular social login providers, including Google and Facebook.
- **Easy Integration:** Simple setup and configuration process, allowing quick integration into existing Angular applications.
- **User Profile Retrieval:** Automatically retrieves user profile information from the social provider, such as name, email, and profile picture.
- **Token Management:** Manages OAuth tokens, ensuring secure authentication and authorization.
- **Configurable:** Offers customization options for the login process, such as scopes and response types.

14.11.2 Usage in Our Project

In our e-learning platform, we used @abacritt/angularx-social-login to enable users to log in or register using their Google or Facebook accounts. This provided a convenient and secure way for users to access the platform without needing to create a separate account.

- **Google and Facebook Login:** Integrated Google and Facebook login functionality, allowing users to authenticate using their social media accounts.
- **User Registration:** Enabled new users to register on the platform using their existing Google or Facebook accounts, streamlining the sign-up process.
- **Profile Retrieval:** Automatically retrieved user profile information, such as name and email, from the social provider, reducing the need for users to enter their details manually.

- **Token Management:** Managed OAuth tokens securely, ensuring safe authentication and authorization.
- **Improved User Experience:** Enhanced the overall user experience by providing a fast and convenient login option, reducing the friction associated with account creation and login.

By incorporating `@abacritt/angularx-social-login` into our project, we were able to provide a seamless and user-friendly authentication experience, encouraging more users to join and engage with our e-learning platform.

14.12 Mermaid

Mermaid is a JavaScript-based diagramming and charting tool that allows users to create complex diagrams and visualizations using a simple text-based syntax. It supports a wide variety of diagram types, including flowcharts, sequence diagrams, class diagrams, and more. Mermaid is particularly useful for generating dynamic diagrams that can be rendered directly in the browser.

14.12.1 Description of Mermaid

Mermaid simplifies the process of creating diagrams and visualizations by allowing users to describe their diagrams using a straightforward, human-readable syntax. The library then converts this textual representation into a visual diagram that can be embedded in web applications.

Key features of Mermaid include:

- **Wide Range of Diagram Types:** Supports various types of diagrams, including flowcharts, sequence diagrams, class diagrams, state diagrams, Gantt charts, and more.
- **Text-Based Syntax:** Allows users to create diagrams using a simple, text-based syntax, making it easy to generate and update diagrams.
- **Dynamic Diagrams:** Enables the creation of dynamic diagrams that can be updated and rendered in real-time in the browser.
- **Integration-Friendly:** Can be easily integrated into web applications, documentation, and other projects.

- **Customizable:** Offers various customization options to style and configure diagrams according to specific needs.

14.12.2 Usage in Our Project

In our e-learning platform, we used Mermaid to convert response codes from the AI API into visual diagrams that could be displayed on the website. This helped in providing a clear and interactive way for users to understand the data and processes.

- **AI API Response Visualization:** Used Mermaid to generate diagrams from AI API response codes, allowing users to visualize data and processes.
- **Flowcharts and Sequence Diagrams:** Created flowcharts and sequence diagrams to illustrate various processes and workflows within the platform.
- **Dynamic Rendering:** Implemented dynamic rendering of diagrams, enabling real-time updates and interactions based on user input and API responses.
- **Enhanced User Experience:** Improved the user experience by providing clear, visual representations of complex information, making it easier to understand and engage with the content.
- **Documentation:** Used Mermaid diagrams in the platform's documentation to visually explain features, processes, and workflows.

By integrating Mermaid into our project, we were able to offer a more engaging and interactive way for users to visualize and comprehend information, enhancing both the usability and appeal of our e-learning platform.

Chapter 15

Building a Content-Based Recommendation System

15.1 Introduction to Recommendation Systems

Recommendation systems are essential tools in the era of information overload. They help users discover items of interest by filtering and suggesting content based on various criteria. In the context of online learning platforms, recommendation systems play a crucial role in enhancing the user experience by suggesting courses that align with learners' interests and educational needs.

There are several types of recommendation systems:

- **Collaborative Filtering:** This method relies on user interactions, such as ratings and clicks, to recommend items. It assumes that users who agreed in the past will agree in the future. Collaborative filtering can be user-based or item-based.
- **Content-Based Filtering:** This approach uses the content information of items and users' preferences to recommend similar items. It compares the features of items to the user's profile.
- **Hybrid Methods:** These systems combine collaborative and content-based filtering to leverage the strengths of both approaches and mitigate their weaknesses.

In our project, we chose to build a content-based recommendation system because it does not require user interaction data, which may be sparse or unavailable in new or less frequently used courses. Instead, it relies on the content of the courses to find similarities and recommend relevant courses to users.

15.2 Data Sources

The data used to build the recommendation system comes from three tables:

- **Courses Table:** Stores information such as course ID, course title, course subject, and learning objectives.
- **Subjects Table:** Contains subject IDs and names.
- **Instructors Table:** Includes data about instructors, such as instructor names, IDs, and areas of expertise.

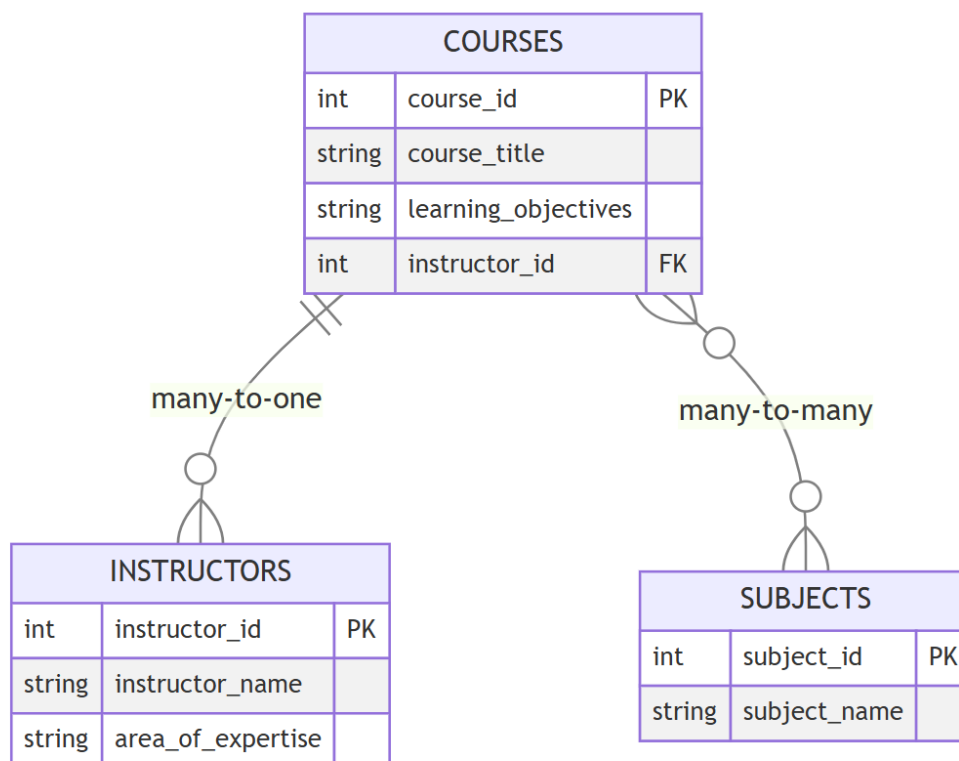


FIGURE 15.1: Schema of the Data Tables Used for the Recommendation System

15.3 Feature Selection

To determine the similarity between courses, several features were selected:

- **Course Title**
- **Course Learning Objectives**

- **Course Language**
- **Course Level**
- **Course Subject**
- **Instructor Name**
- **Instructor Expertise**

These features provide a comprehensive representation of each course, ensuring that the recommendation system can accurately identify similar courses.

15.4 Text Vectorization with TF-IDF

To process the textual data, Term Frequency-Inverse Document Frequency (TF-IDF) vectorization was used. TF-IDF is a statistical measure that evaluates the importance of a word in a document relative to a collection of documents (corpus). It consists of two components:

- **Term Frequency (TF)**: Measures how frequently a word appears in a document. The term frequency of a word t in a document d is given by:

$$\text{TF}(t, d) = \frac{\text{Number of times } t \text{ appears in } d}{\text{Total number of words in } d}$$

- **Inverse Document Frequency (IDF)**: Measures how important a word is across all documents. It is given by:

$$\text{IDF}(t) = \log \left(\frac{\text{Total number of documents}}{\text{Number of documents containing } t} \right)$$

The TF-IDF score for a word t in a document d is then calculated as:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

This method transforms the text data into numerical vectors, which can then be used to calculate similarities.

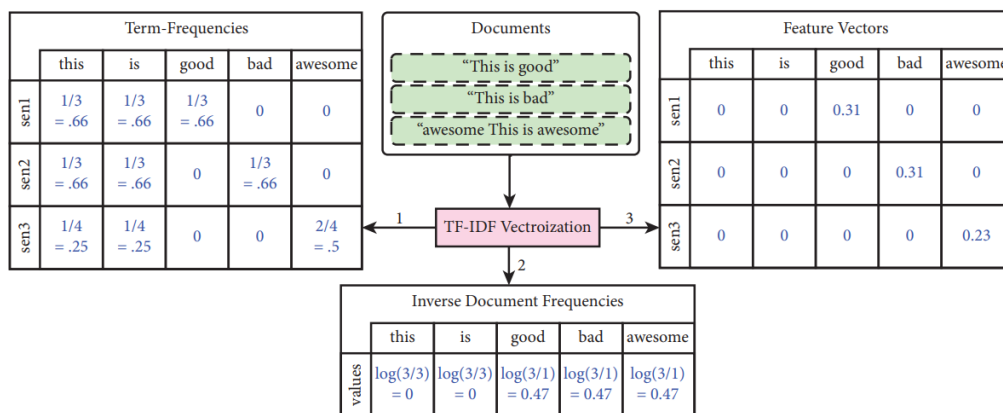


FIGURE 15.2: TF-IDF Vectorization Process

15.5 Calculating Similarity with Cosine Similarity

Cosine similarity was employed to compute the similarity between the TF-IDF vectors.

Cosine similarity measures the cosine of the angle between two vectors, providing a metric that ranges from -1 to 1. The cosine similarity between two vectors A and B is given by:

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

where $A \cdot B$ is the dot product of vectors A and B , and $\|A\|$ and $\|B\|$ are the magnitudes of A and B respectively. In this context, a value of 1 indicates identical courses, while a value of 0 indicates no similarity.

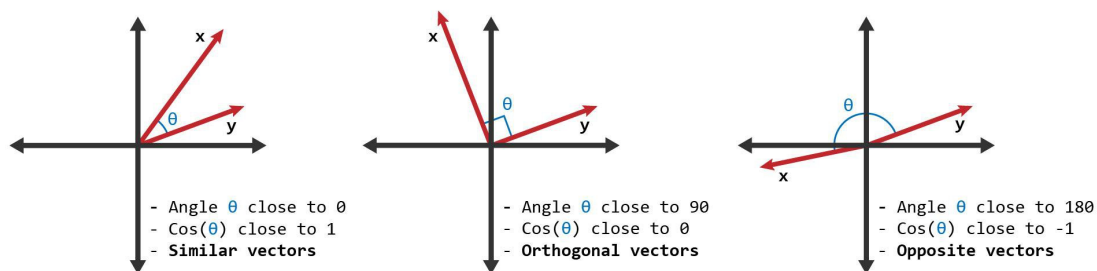


FIGURE 15.3: Cosine Similarity

15.6 Building the Recommendation System

All components were integrated into a single function that takes a course name as input and returns a list of the 10 most similar courses. The steps involved are:

1. Load the data from the tables.

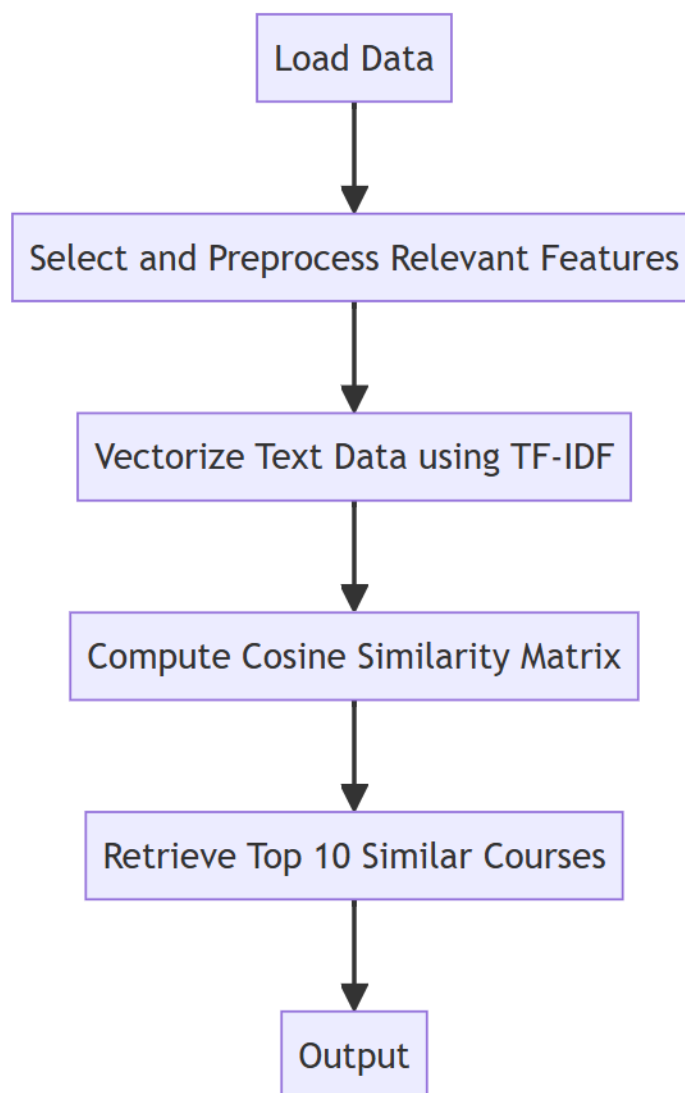


FIGURE 15.4: Recommendation system flowchart

2. Select and preprocess the relevant features.
3. Vectorize the text data using TF-IDF.
4. Compute the cosine similarity matrix.
5. Retrieve the top 10 similar courses for the given course.

15.7 Deployment with FastAPI and Docker

The recommendation system was deployed using FastAPI, a modern web framework for building APIs with Python. The API was containerized using Docker and deployed on Hugging Face Spaces, a platform for hosting machine learning models and applications.

15.7.1 FastAPI Implementation

FastAPI was chosen for its ease of use, performance, and automatic documentation generation. The API exposes an endpoint that takes a course name and returns the top 10 similar courses. Here is a brief overview of how FastAPI was implemented:

1. **Setup:** Install FastAPI and Uvicorn.
2. **Define the API:** Create endpoints for the recommendation functionality.
3. **Integrate the Recommendation System:** Call the recommendation function within the API endpoint.
4. **Run the API:** Use Uvicorn to run the FastAPI application.

15.7.2 Containerization with Docker

Docker was used to create a containerized environment for the API, ensuring consistency across different deployment platforms. The Docker setup involves:

1. **Dockerfile:** Create a Dockerfile to define the image.
2. **Build the Image:** Use Docker to build the image from the Dockerfile.
3. **Run the Container:** Start the container from the built image.

15.7.3 Hosting on Hugging Face Spaces

The containerized API was deployed on Hugging Face Spaces, providing a scalable and accessible interface for the recommendation system. The steps to deploy on Hugging Face Spaces include:

1. **Create a Space:** Set up a new Space on Hugging Face.
2. **Configure the Space:** Set the Space to use Docker.
3. **Upload Docker Image:** Push the Docker image to the Hugging Face Space.
4. **Run the Space:** Start the Space, making the API accessible online.

15.8 Conclusion

This chapter detailed the development and deployment of a content-based recommendation system for an online learning platform. By leveraging TF-IDF vectorization and cosine similarity, the system effectively recommends courses similar to a given course. The deployment using FastAPI, Docker, and Hugging Face Spaces ensures the system is robust, scalable, and accessible.

Chapter 16

Visual Learning: AI-Powered Explanations and Diagrams

16.1 Introduction to Visual Learning

Visual learning is a method that uses visual aids to facilitate understanding and retention of information. In the context of computer science and programming, visualizations such as diagrams and charts can simplify complex concepts, making them more accessible and easier to grasp. This chapter introduces an AI tool designed to explain computer science concepts and generate various diagrams using the Mermaid diagramming language.

16.2 Types of Diagrams Generated by the Tool

The AI tool supports the generation of several types of diagrams, each serving unique purposes in the context of computer science and programming:

16.2.1 Flowcharts

Flowcharts are used to represent algorithms and workflows. They help in understanding the sequence of steps in a process.

Example of a Flowchart

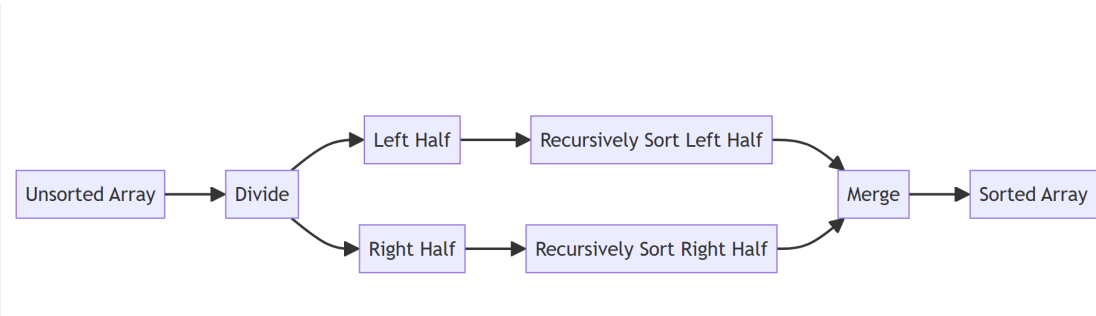


FIGURE 16.1: A flowchart of the merge sort algorithm

16.2.2 Sequence Diagrams

Sequence diagrams illustrate how objects interact in a particular sequence of time. They are useful for understanding the flow of messages between objects in a system.

Example of a Sequence Diagram

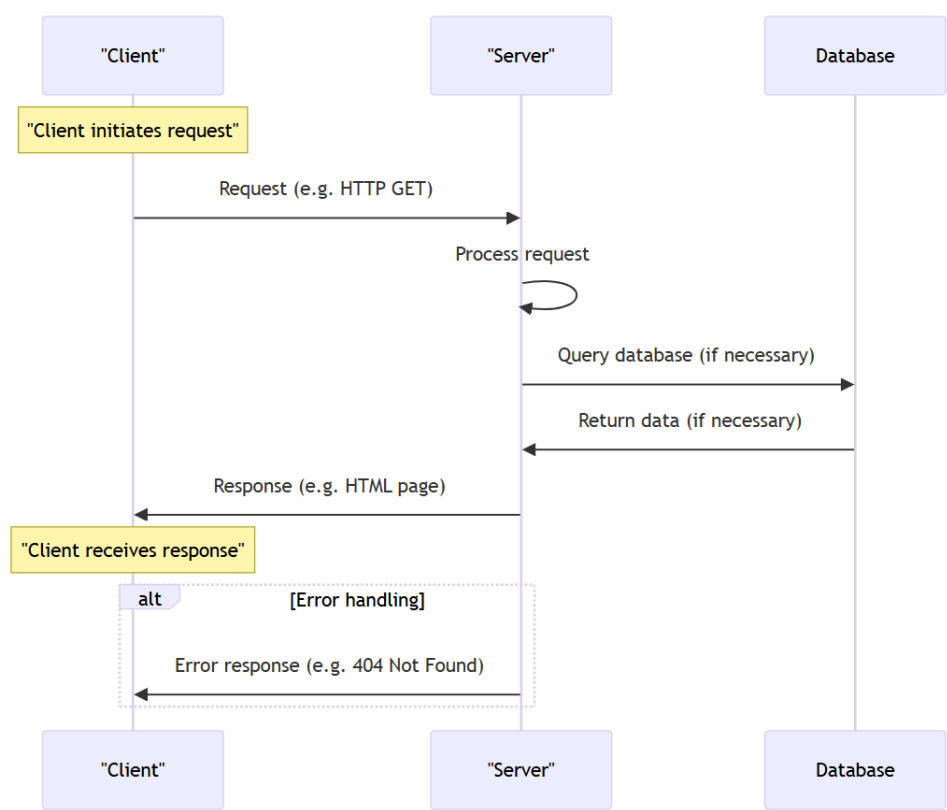


FIGURE 16.2: A sequence diagram for the client server architecture

16.2.3 Mind Maps

Mind maps are used to visually organize information. They are particularly useful for brainstorming and structuring complex concepts.

Example of a Mind Map

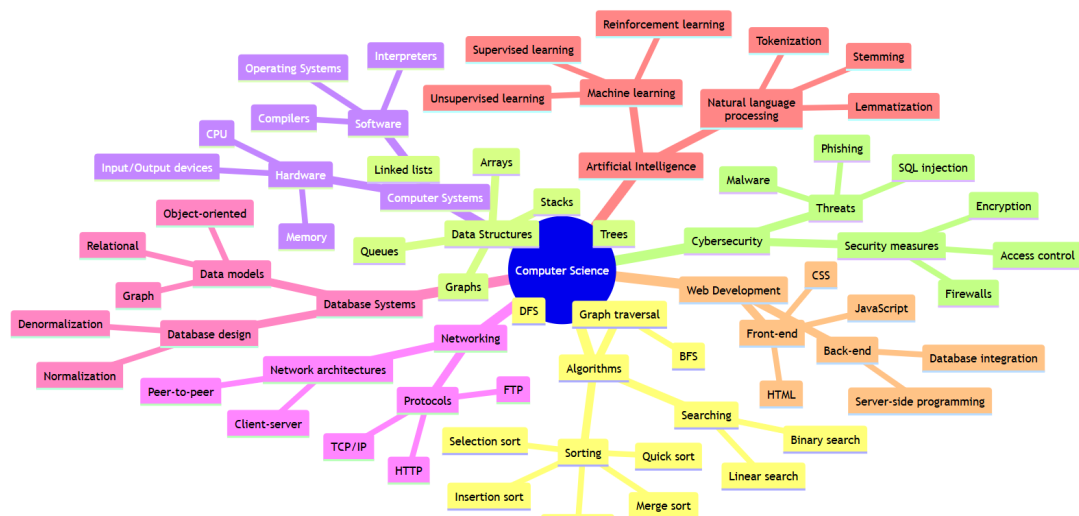


FIGURE 16.3: A mind map of important computer science concepts

16.2.4 Class Diagrams

Class diagrams represent the static structure of a system by showing its classes, attributes, methods, and relationships among objects. Figure 17.4 shows an example of a class diagram that models a simple banking system

16.2.5 ER Diagrams

Entity-Relationship (ER) diagrams are used to model the data structure of a database. They illustrate the relationships between entities in a database. Figure 17.5 shows an example of a simple ER diagram

16.2.6 Gantt Charts

Gantt charts are used for project management, showing the timeline of tasks and their dependencies. Figure 17.6 shows an example of a simple Gantt diagram

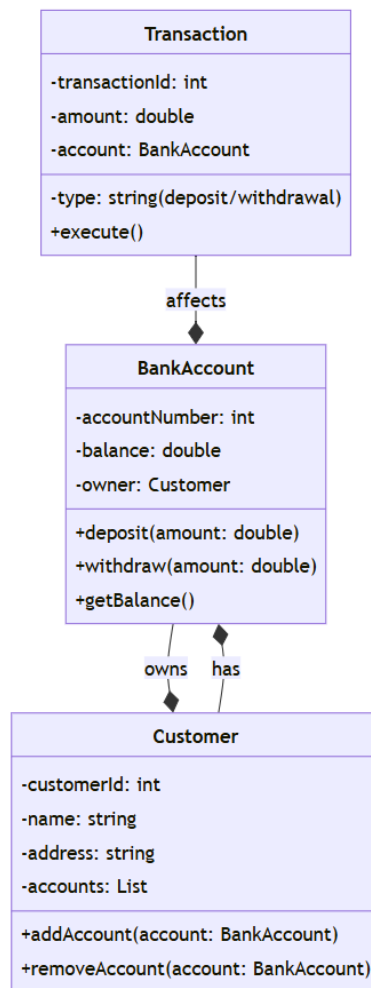


FIGURE 16.4: A class diagram modelling a simple banking system

16.2.7 State Diagrams

State diagrams represent the states and transitions of a system, particularly useful in modeling the behavior of objects. Figure 17.7 shows an example of a simple state diagram

16.3 Mermaid Diagramming Language

Mermaid is a powerful diagramming language that allows users to create various types of diagrams and visualizations using a simple and intuitive syntax. It is widely used in documentation, educational materials, and project management tools to create clear and informative visual representations.

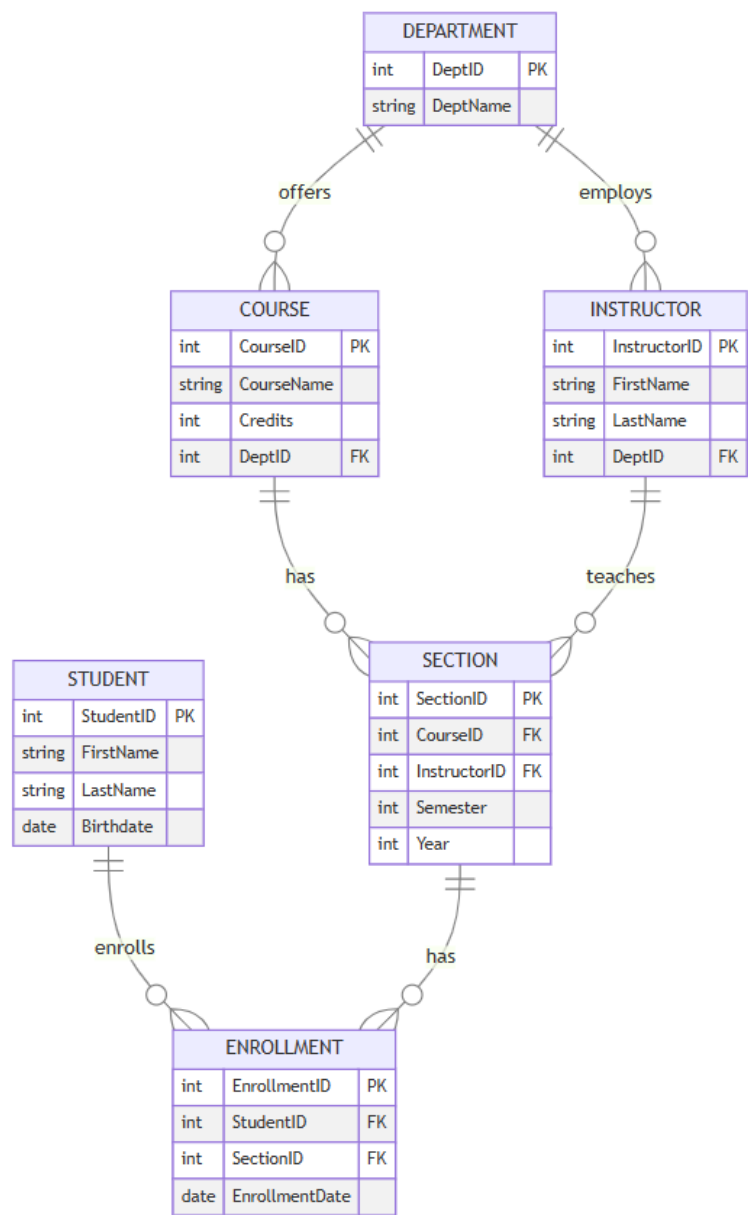


FIGURE 16.5: A simple ER diagram for

16.3.1 Overview of Mermaid Syntax

Mermaid uses a text-based syntax to define diagrams. Users can create diagrams by writing code in a plain text format, which is then rendered into graphical representations by the Mermaid parser. The simplicity and readability of Mermaid syntax make it accessible to both technical and non-technical users.

Example of Mermaid Syntax for a Flowchart

graph TD

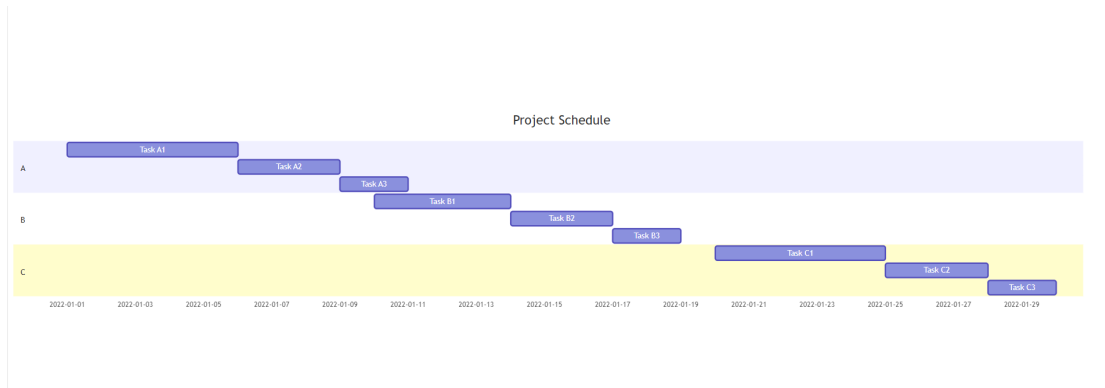


FIGURE 16.6: Gantt Chart Example

```

A[Start] --> B{Decision}
B -->|Yes| C[Action 1]
B -->|No| D[Action 2]
C --> E[End]
D --> E

```

16.3.2 Integration with AI Tools

Mermaid's text-based syntax makes it an ideal candidate for integration with AI tools.

In our project, the AI tool generates Mermaid code based on natural language descriptions provided by the user. This code is then rendered into diagrams, providing a visual representation of the requested concept.

By leveraging the power of the LLaMA3-70B model and the simplicity of Mermaid syntax, our AI tool can generate detailed and accurate diagrams that enhance the understanding of complex computer science concepts.

16.4 Building the AI Tool

The AI tool leverages advanced techniques in prompt engineering and few-shot learning to generate accurate explanations and diagrams from natural language inputs.

16.4.1 Prompt Engineering

Prompt engineering is the process of designing and refining the prompts given to the

AI model to elicit the desired responses. This involves crafting instructions and examples that guide the model to produce accurate and relevant outputs. In our tool,

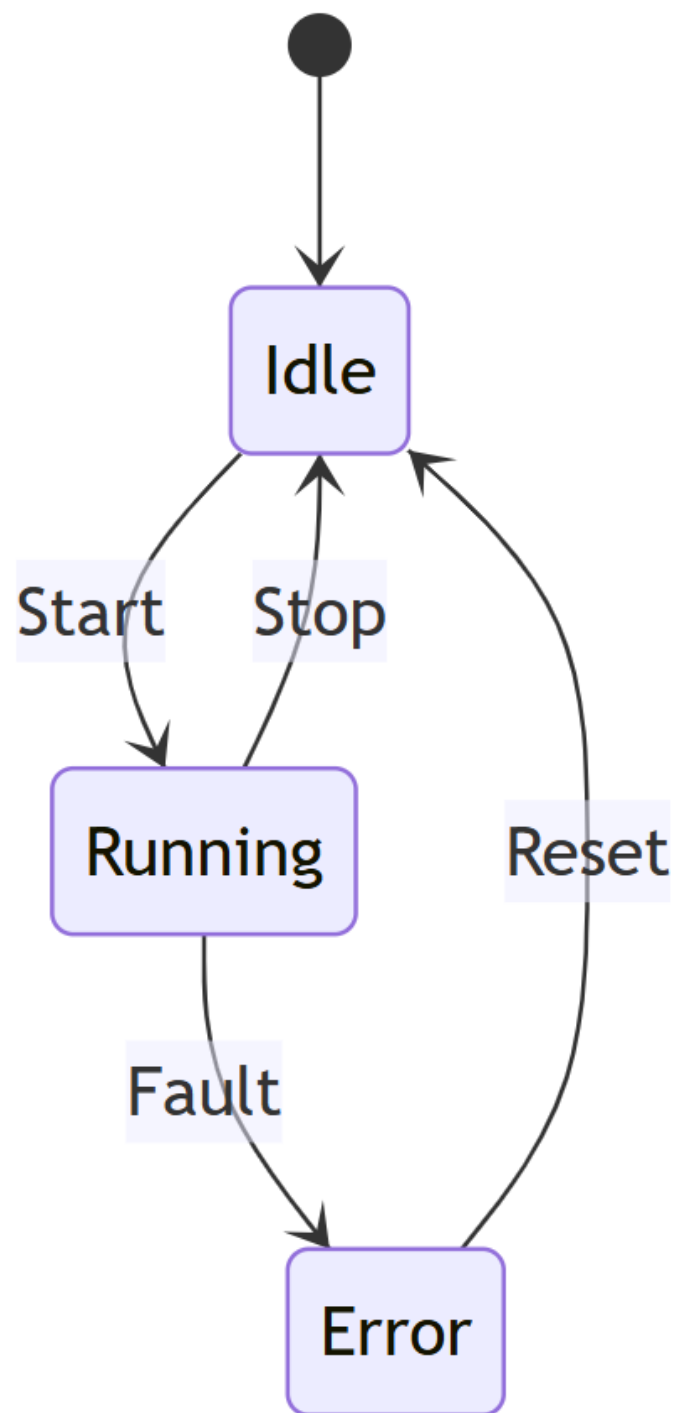


FIGURE 16.7: A state diagram for a simple system

we use the Llama3-70b model and provide it with a system message that instructs it to act as an expert in generating Mermaid diagrams from natural language descriptions.

System Message Example

"You are an expert in generating Mermaid diagrams from natural language

descriptions. Given a description of a computer science concept or a request for a specific type of diagram, generate the appropriate Mermaid code and a brief explanation."

16.4.2 Few-Shot Learning

To refine the tool's performance, the system message is supplemented with example inputs and outputs. These examples demonstrate how to translate user requests into

Mermaid diagrams and explanations, this technique is known as few-shot learning. Few-shot learning is a training approach where the model is provided with only a few examples to learn from. Unlike traditional machine learning models that require large datasets, few-shot learning leverages a limited number of examples to understand the task at hand. This approach is particularly useful for our tool because it allows the model to quickly adapt to generating various types of diagrams from diverse natural language inputs.

In the context of our tool, few-shot learning is implemented by providing the model with a small set of input-output pairs. Each pair consists of a user request and the corresponding Mermaid diagram code and explanation. For example, if a user requests a class diagram for a simple banking system, the input-output pair might look like this:

Example Input and Output for Few-Shot Learning

Input: "Generate a class diagram for a simple banking system with classes for Account, Customer, and Transaction."

Output: "Here is a class diagram for a simple banking system:

'''

classDiagram

```
class Account {
    -int accountNumber
    -float balance
    +deposit(amount)
    +withdraw(amount)
}
```

```
class Customer {
    -int customerID
    -String name
    +openAccount()
    +closeAccount()
}
```

```
class Transaction {  
    -int transactionID  
    -date transactionDate  
    -float amount  
    +process()  
}  
  
Customer --> Account  
Account --> Transaction  
'''
```

By learning from these examples, the AI model can generalize and generate accurate diagrams for a wide range of requests.

16.5 The LLaMA3-70B Model

The LLaMA3-70B model is a state-of-the-art large language model developed by Meta AI. It is part of the LLaMA (Large Language Model Meta AI) series, which focuses on creating powerful, efficient, and versatile AI models for various applications.

LLaMA3-70B, with 70 billion parameters, represents one of the largest and most capable models in the series.

16.5.1 Architecture and Design

The LLaMA3-70B model is based on the transformer architecture, which has become the standard for modern natural language processing (NLP) models. Transformers use self-attention mechanisms to process input data in parallel, allowing for faster and more efficient handling of large datasets. The model consists of multiple layers of attention and feed-forward networks, enabling it to learn and generate complex patterns in the data.

16.5.2 Training and Capabilities

LLaMA3-70B was trained on a diverse and extensive dataset, covering a wide range of topics and languages. This comprehensive training enables the model to understand and generate human-like text across various domains. The model's large size, with 70 billion parameters, allows it to capture intricate details and nuances in language, making it highly effective for tasks such as text generation, translation, summarization, and question-answering.

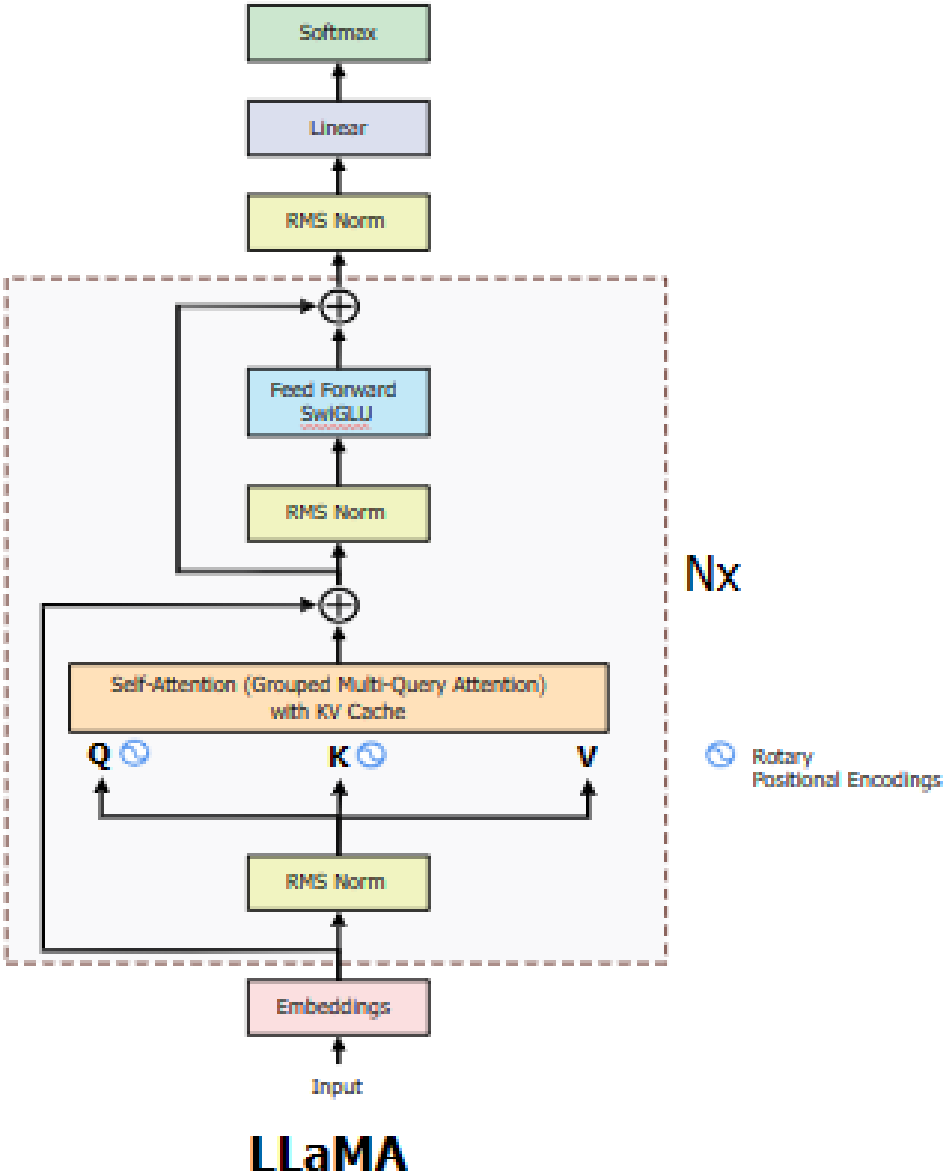


FIGURE 16.8: LLaMA Architecture

16.5.3 Applications in Visual Learning

In the context of visual learning, the LLaMA3-70B model plays a crucial role in generating accurate and detailed explanations of computer science concepts. By leveraging its advanced language understanding capabilities, the model can interpret user requests, generate relevant Mermaid diagrams, and provide clear and concise explanations. This makes it an invaluable tool for enhancing the learning experience through visual aids.

16.5.4 Langchain Library

The Langchain library is a powerful tool in the development of applications that interact with large language models (LLMs). It provides a framework for integrating LLMs into various workflows, allowing developers to leverage the capabilities of these models effectively. In our AI tool, Langchain is used to implement the prompt engineering concepts and to manage the interaction between the AI model and the diagram generation process.

Key Features of Langchain

- **Prompt Management:** Langchain allows developers to define and manage prompts efficiently, ensuring that the AI model receives clear and concise instructions.
- **Integration with LLMs:** Langchain seamlessly integrates with various LLMs, including the Llama3-70b model used in our tool. This integration is facilitated through the `Langchain-groq` library, which provides a direct interface to the Groq API.
- **Workflow Orchestration:** Langchain enables the orchestration of complex workflows involving multiple steps and interactions with the AI model. This is particularly useful for handling user requests, processing natural language inputs, and generating the corresponding Mermaid diagrams.

16.5.4.1 Prompt Management

Langchain's prompt management features allow us to define detailed and structured prompts for the AI model. This includes the system message, example inputs and outputs, and any additional instructions required to generate accurate diagrams.

16.5.4.2 Integration with Groq API

The Groq API is known for its fast AI inference, making it an ideal choice for real-time applications. The `Langchain-groq` library integrates Langchain with the Groq API, enabling efficient processing of user requests and generation of Mermaid diagrams.

16.5.4.3 Workflow Orchestration

Langchain provides tools for orchestrating the entire workflow from receiving user requests to generating and displaying the diagrams. This includes parsing the natural

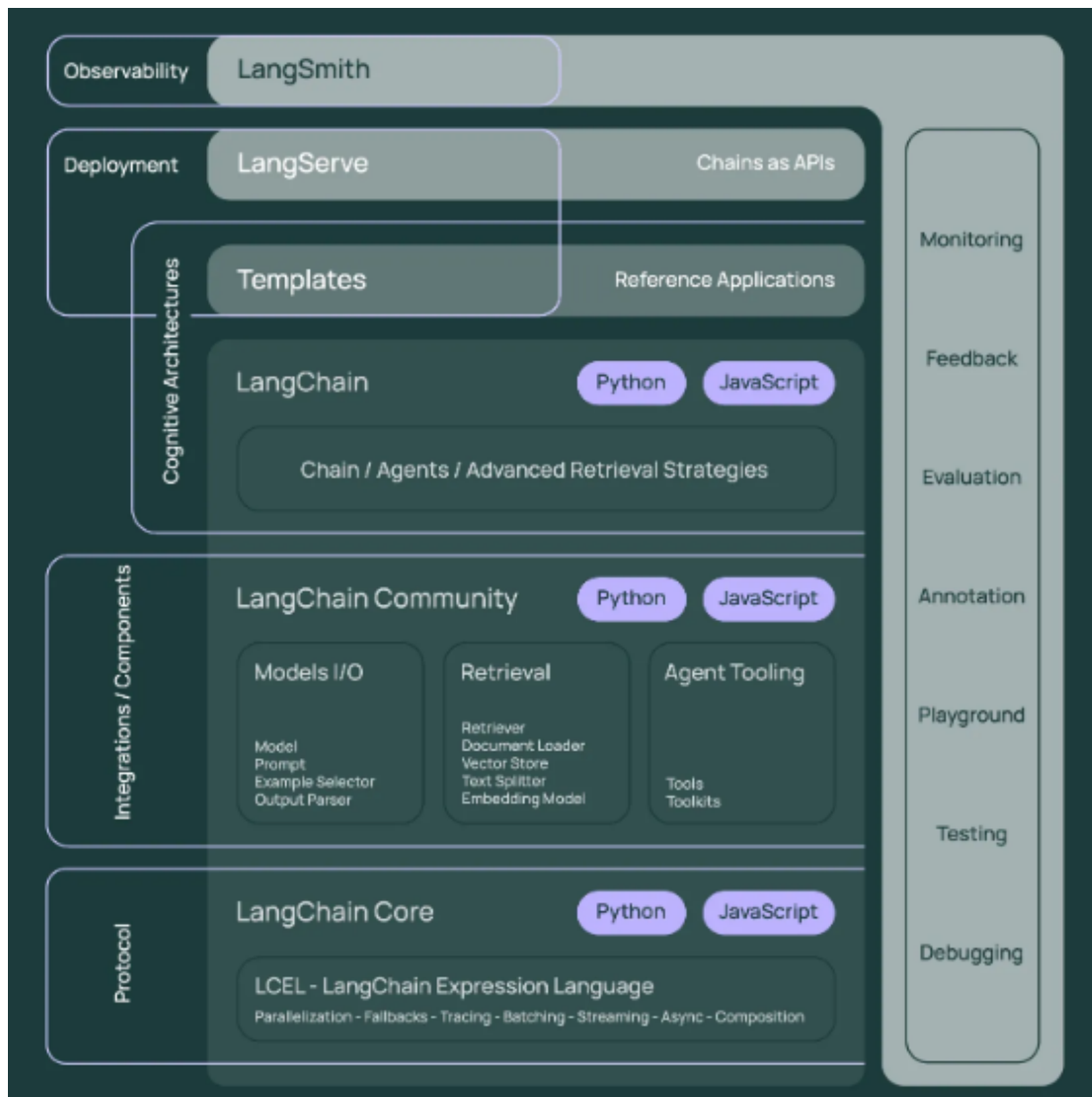


FIGURE 16.9: Langchain Architecture

language input, generating the corresponding Mermaid code, and formatting the output for the user.

16.6 Deployment with FastAPI and Docker

The recommendation system was deployed using FastAPI, a modern web framework for building APIs with Python. The API was containerized using Docker and deployed on Hugging Face Spaces, a platform for hosting machine learning models and applications.

16.6.1 FastAPI Implementation

FastAPI was chosen for its ease of use, performance, and automatic documentation generation. The API exposes an endpoint that takes a course name and returns the top 10 similar courses. Here is a brief overview of how FastAPI was implemented:

1. **Setup:** Install FastAPI and Uvicorn.
2. **Define the API:** Create endpoints for the recommendation functionality.
3. **Integrate the Recommendation System:** Call the recommendation function within the API endpoint.
4. **Run the API:** Use Uvicorn to run the FastAPI application.

16.6.2 Containerization with Docker

Docker was used to create a containerized environment for the API, ensuring consistency across different deployment platforms. The Docker setup involves:

1. **Dockerfile:** Create a Dockerfile to define the image.
2. **Build the Image:** Use Docker to build the image from the Dockerfile.
3. **Run the Container:** Start the container from the built image.

16.6.3 Hosting on Hugging Face Spaces

The containerized API was deployed on Hugging Face Spaces, providing a scalable and accessible interface for the recommendation system. The steps to deploy on Hugging Face Spaces include:

1. **Create a Space:** Set up a new Space on Hugging Face.
2. **Configure the Space:** Set the Space to use Docker.
3. **Upload Docker Image:** Push the Docker image to the Hugging Face Space.
4. **Run the Space:** Start the Space, making the API accessible online.

16.7 Conclusion

Visual learning through AI-powered explanations and diagrams provides an effective way to understand complex computer science and programming concepts. The AI tool developed leverages advanced technologies and methodologies to offer users intuitive and accurate visualizations, enhancing their learning experience.

Chapter 17

Conclusion and Future Work

17.1 Conclusion

MoltaqaTech has successfully established itself as a comprehensive e-learning platform tailored for the tech industry. By integrating multiple modules such as courses, blogs, events, Zoom meetings, and a robust security system, MoltaqaTech provides a holistic learning and community experience for tech students and professionals.

The core feature of the platform, the Courses module, allows instructors to create detailed and structured courses with various interactive elements such as quizzes, lectures, and progress tracking. This ensures that students have a clear learning path and can track their progress effectively. The integration with payment systems like Stripe and functionalities like course reviews and wishlists enhance the user experience by providing a seamless and engaging interface for learning.

The BlogPosts module fosters a community of knowledge sharing, where users can post articles, comment, and engage in discussions. This module supports the continuous growth and exchange of ideas within the tech community.

The Events module enables users to announce and participate in tech-related events, providing networking and learning opportunities beyond the traditional course structure. The integration with Zoom for live meetings further extends the platform's capabilities by allowing real-time interaction and collaboration.

The platform's security is ensured through robust authentication and authorization mechanisms, utilizing JWT and token management, which are essential for protecting user data and maintaining trust.

Overall, MoltaqaTech is not just an e-learning platform but a vibrant community for tech enthusiasts to learn, share, and grow together. Its comprehensive feature set and user-friendly interface make it a valuable resource for both students and instructors.

17.2 Future Work

While MoltaqaTech has made significant strides in providing a comprehensive e-learning platform, there are several areas for future improvement and expansion:

17.2.1 Enhanced Analytics and Reporting

Future developments could include more advanced analytics and reporting features for both students and instructors. For students, this could mean more detailed insights into their learning patterns, strengths, and areas for improvement. For instructors, enhanced analytics could provide deeper insights into course performance, student engagement, and content effectiveness.

17.2.2 Personalized Learning Paths

Implementing AI-driven personalized learning paths can significantly enhance the user experience. By analyzing student behavior and performance, the platform can recommend tailored courses, sections, or additional resources that align with the student's learning pace and interests.

17.2.3 Gamification

Introducing gamification elements such as badges, leaderboards, and achievement tracking can increase student engagement and motivation. These features can encourage continuous learning and participation by rewarding students for their progress and accomplishments.

17.2.4 Mobile Application

Developing a mobile application for MoltaqaTech can provide users with greater flexibility and access to the platform. A mobile app can enhance the learning experience by allowing students to access courses, participate in discussions, and manage their accounts on-the-go.

17.2.5 Integration with Other Tools

Further integration with other educational and productivity tools can expand the platform's functionality. For example, integrating with project management tools, additional video conferencing solutions, or learning management systems can provide users with a more seamless and comprehensive experience.

17.2.6 Advanced Security Features

Enhancing the platform's security features by implementing multi-factor authentication (MFA), biometric authentication, and continuous security monitoring can further protect user data and ensure the platform's integrity.

17.2.7 Expanded Payment Options

Adding more payment options, such as cryptocurrency or regional payment methods, can cater to a broader audience and make the platform more accessible to users from different regions.

17.2.8 Collaborative Learning Features

Introducing collaborative learning features such as group projects, peer reviews, and collaborative note-taking can enhance the interactive and social aspects of the learning experience. These features can help students learn from each other and develop teamwork skills.

17.2.9 Content Localization

Expanding the platform's reach by localizing content for different languages and regions can attract a more diverse user base. Providing courses and support in multiple languages can make the platform more inclusive and accessible.

17.2.10 Continuous Improvement and Feedback Loops

Establishing continuous feedback loops with users to gather insights and suggestions for improvement can help keep the platform aligned with user needs and expectations.

Regular updates and improvements based on user feedback can ensure that MoltaqaTech remains a leading e-learning platform.

By focusing on these future developments, MoltaqaTech can continue to grow and adapt to the evolving needs of the tech education community, providing even greater value and support to its users.

Appendix A

Appendix

A.1 Source Code Links

- Text Summarization Model: <https://github.com/Anwar11234/text-summarization-AI-m>
- Content-Based Recommender System: <https://github.com/Anwar11234/contetn-based-recommender-system>
- Autocomplete Model: https://github.com/mohammedalalimy/Autocomlete_model
- Machine Translation between Arabic and English: <https://github.com/mohammedalalimy/Machine-Translation-between-Arabic-and-English-by-Transformers>
- Autocorrect Spelling Correction: <https://github.com/mohammedalalimy/autocorrect-spelling-correction>
- Chatbot: <https://github.com/1Mou3az/ChatBot>
- Quiz Generation: https://github.com/1Mou3az/Quiz_Generation
- text2diagram: <https://github.com/Anwar11234/text2diagram>
- Front-end: <https://github.com/Omar-Nabil/MultaqaTech>
- Back-end: <https://github.com/Ahmed58Usama2001/MultaqaTech.Solution>

A.2 References

1. <https://medium.com/expedia-group-tech/onion-architecture-deed8a554423>

2. <https://medium.com/@alessandro.traversi/understanding-onion-architecture-an-example-for>
3. <https://dotnet.microsoft.com/en-us/apps/aspnet/apis>
4. <https://medium.com/@sajadshafi/jwt-authentication-in-c-net-core-7-web-api-b825b3aee11d>
5. <https://redis.io/solutions/caching/>
6. <https://medium.com/@pauloluwasen/implementing-social-authentication-in-net-web-api-215>
7. <https://devdocs.io/angular~16/>
8. <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/write?view=aspnetcore-8.0>
9. <https://learn.microsoft.com/en-us/aspnet/web-api/overview/data/using-web-api-with-entity-framework-part-5>
10. <https://developers.zoom.us/docs/api/>
11. <https://material.angular.io/>
12. <https://getbootstrap.com/docs/4.1/getting-started/introduction/>
13. <https://docs.stripe.com/payments/payment-intents>
14. <https://medium.com/data-at-the-core/how-llms-work-explained-in-9-steps-transformer-architecture-3b138e>
15. <https://medium.com/@javaid.nabi/all-you-need-to-know-about-llm-text-generation-03b138e>
16. <https://medium.com/@AIBites/lora-low-rank-adaptation-of-llms-paper-explained-5ae866871>
17. <https://towardsdatascience.com/introduction-to-recommender-systems-1-971bd274f421>
18. <https://medium.com/@prateekgaurav/step-by-step-content-based-recommendation-system-823>
19. <https://medium.com/@cmukesh8688/tf-idf-vectorizer-scikit-learn-dbc0244a911a>
20. <https://www.machinelearningplus.com/nlp/cosine-similarity/>
21. <https://www.youtube.com/watch?v=kCc8FmEb1nY&list=PLAqIrjkxbuWI23v9cThsA9GvCAUhRvKZ&index=7>
22. <https://jalammar.github.io/illustrated-transformer/>
23. <https://arxiv.org/abs/1706.03762>
24. <https://arxiv.org/abs/2106.09685>
25. <https://huggingface.co/learn/nlp-course/chapter7/5?fw=pt>

26. <https://medium.com/@sthanikamsanthosh1994/understanding-bleu-and-rouge-score-for>
27. <https://huggingface.co/blog/HemanthSai7/deploy-applications-on-huggingface-space>
28. <https://learnprompting.org/docs/intro>
29. <https://www.promptingguide.ai/>
30. <https://mermaid.js.org/intro/getting-started.html>
31. <https://arxiv.org/abs/2401.14423>
32. <https://medium.com/@akriti.upadhyay/optimizing-performance-with-peft-a-deep-dive>
33. <https://towardsdatascience.com/fine-tuning-large-language-models-llms-23473d763b>
34. <https://piyush0511.medium.com/autocorrect-spell-check-using-attention-model-9aa2>
35. https://www.geeksforgeeks.org/sentence-autocomplete-using-pytorch/?ref=header_search